

UNIVERSIDADE FEDERAL DE SANTA CATARINA

CAMPUS FLORIANÓPOLIS

CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Tális Breda

ESTUDO DA FAMÍLIA DE EMPARELHAMENTO EM GRAFOS

Florianópolis

2026

Tális Breda

ESTUDO DA FAMÍLIA DE EMPARELHAMENTO EM GRAFOS

Trabalho de Conclusão de Curso submetido ao Curso de Graduação em Ciência da Computação do Centro Tecnológico da Universidade Federal de Santa Catarina para obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Rafael de Santiago, Dr.

Florianópolis

2026

[Ficha catalográfica a ser gerada em <http://portalbu.ufsc.br/ficha>]

Tális Breda

ESTUDO DA FAMÍLIA DE EMPARELHAMENTO EM GRAFOS

Este Trabalho de Conclusão de Curso foi julgado adequado para obtenção do título de Bacharel em Ciência da Computação e aprovado em sua forma final pelo Curso de Graduação em Ciência da Computação.

Florianópolis, [dia] de [mês] de 2026.

_____

Profa. Lúcia Helena Martins Pacheco, Dra.
Coordenadora do Curso

Banca Examinadora:

_____

Prof. Rafael de Santiago, Dr.
Orientador
Universidade Federal de Santa Catarina

_____

[Nome], Dr(a).
Avaliador(a)
[Instituição]

_____

[Nome], Dr(a).
Avaliador(a)
[Instituição]

RESUMO

Um emparelhamento em um grafo $G = (V, E)$ é definido como um conjunto de arestas $M \subseteq E$ tal que nenhum par de arestas em M compartilha um vértice comum. Os problemas de emparelhamento em grafos têm diversas variações, e suas aplicações variam de áreas da computação como visão computacional até áreas fora dela, como a biologia. Eles têm utilidade principalmente em tarefas que envolvem a correspondência de elementos, como a comparação de elementos em conjuntos distintos, por exemplo em análise de dados estruturados. Enquanto as formulações clássicas são solúveis eficientemente, variações com restrições adicionais pertencem frequentemente à classe NP-Difícil, e suas soluções podem ser computacionalmente custosas. Devido à existência de vários problemas, propriedades, e algoritmos relacionados a emparelhamento de grafos, este trabalho busca levantar, comparar e classificar métodos computacionais, destacando suas aplicações e limitações, com o objetivo de fornecer insights úteis para pesquisadores e profissionais na escolha de soluções eficazes para cenários práticos.

Palavras-chave: Emparelhamento em grafos; algoritmos; grafos.

ABSTRACT

A matching in a graph $G = (V, E)$ is defined as a set of edges $M \subseteq E$ such that no two edges in M share a common vertex. Graph matching problems have several variations, and their applications range from areas of computing, such as computer vision, to areas outside it, such as biology. They are mainly useful in tasks that involve the correspondence of elements, such as the comparison of elements across distinct sets, for instance in the analysis of structured data. While the classical formulations are efficiently solvable, variations with additional constraints often belong to the NP-hard class, and their solutions can be computationally expensive. Given the many problems, properties, and algorithms related to graph matching, this work aims to survey, compare, and classify computational methods, highlighting their applications and limitations, with the goal of providing useful insights to researchers and practitioners when choosing effective solutions for practical scenarios.

Keywords: Graph matching; algorithms; graphs.

LISTA DE ILUSTRAÇÕES

Figura 1 – Tempo de execução em escala log-log para o grafo denso (cardinalidade máxima bipartida). A inclinação de cada reta corresponde ao expoente empírico de crescimento; a de Edmonds-Karp, próxima de 3, destaca-se das demais. 122
Figura 2 – Tempo de execução em escala log-log para a matriz densa (atribuição linear). As duas implementações de núcleo compilado situam-se bem abaixo das interpretadas, mas com inclinação semelhante. 123

LISTA DE TABELAS

Tabela 1 – Algoritmos para emparelhamento de cardinalidade máxima em grafos bipartidos 51
Tabela 2 – Algoritmos para problema de atribuição 63
Tabela 3 – Algoritmos para problema de atribuição gargalo 73
Tabela 4 – Algoritmos para problema de emparelhamento de cardinalidade máxima em grafos gerais 83
Tabela 5 – Algoritmos para problema de emparelhamento ponderado máximo em grafos gerais 91
Tabela 6 – Algoritmos para problema de emparelhamento induzido máximo em grafos gerais 97
Tabela 7 – Interface padrão de entrada e saída por problema 99
Tabela 8 – Famílias de instâncias por problema 100
Tabela 9 – Especificação da máquina de medição 108
Tabela 10 – Proveniência dos algoritmos avaliados 110
Tabela 11 – Tempo de execução em Python — Grafo 1 — Esparso | SEM perfeito | MCM = 80% de n (mediana, ms) 112
Tabela 12 – Qualidade da solução — Grafo 1 — Esparso | SEM perfeito | MCM = 80% de n 112
Tabela 13 – Tempo de execução em Python — Grafo 2 — Esparso | COM perfeito | MCM = n (mediana, ms) 113
Tabela 14 – Qualidade da solução — Grafo 2 — Esparso | COM perfeito | MCM = n113
Tabela 15 – Tempo de execução em Python — Grafo 3 — Denso | COM perfeito | MCM = n (mediana, ms) 113
Tabela 16 – Qualidade da solução — Grafo 3 — Denso | COM perfeito | MCM = n 113
Tabela 17 – Tempo de execução em Python — Matriz 1 — Densa uniforme | custo in [1,100] (mediana, ms) 114
Tabela 18 – Qualidade da solução — Matriz 1 — Densa uniforme | custo in [1,100]114
Tabela 19 – Tempo de execução em Python — Matriz 2 — Permutação barata | ótimo = n (mediana, ms) 114
Tabela 20 – Qualidade da solução — Matriz 2 — Permutação barata | ótimo = n 115
Tabela 21 – Tempo de execução em Python — Matriz 3 — Estruturada | custo = |i-j|+1 (mediana, ms) 115
Tabela 22 – Qualidade da solução — Matriz 3 — Estruturada | custo = |i-j|+1 . . 115
Tabela 23 – Tempo de execução em Python — Matriz 1 — Densa uniforme | custo in [1,100] (mediana, ms) 116
Tabela 24 – Qualidade da solução — Matriz 1 — Densa uniforme | custo in [1,100]116

Tabela 25 – Tempo de execução em Python — Matriz 2 — Estruturada | custo = |i-j|+1 (mediana, ms) 116
Tabela 26 – Qualidade da solução — Matriz 2 — Estruturada | custo = |i-j|+1 . . 116
Tabela 27 – Tempo de execução em Python — Matriz 3 — Poucos valores | muitos empates (mediana, ms) 116
Tabela 28 – Qualidade da solução — Matriz 3 — Poucos valores | muitos empates116
Tabela 29 – Tempo de execução em Python — Perfil 1 — Aleatório | permutações uniformes (mediana, ms) 117
Tabela 30 – Qualidade da solução — Perfil 1 — Aleatório | permutações uniformes117
Tabela 31 – Tempo de execução em Python — Perfil 2 — Correlacionado | ranking de popularidade (mediana, ms) 117
Tabela 32 – Qualidade da solução — Perfil 2 — Correlacionado | ranking de popularidade 117
Tabela 33 – Tempo de execução em Python — Perfil 3 — Cíclico | deslocamento estruturado (mediana, ms) 118
Tabela 34 – Qualidade da solução — Perfil 3 — Cíclico | deslocamento estruturado118
Tabela 35 – Tempo de execução em Python — Cardinalidade — esparso (grau ~3) (mediana, ms) 118
Tabela 36 – Qualidade da solução — Cardinalidade — esparso (grau ~3) 118
Tabela 37 – Tempo de execução em Python — Cardinalidade — denso (p=0.3) (mediana, ms) 119
Tabela 38 – Qualidade da solução — Cardinalidade — denso (p=0.3) 119
Tabela 39 – Tempo de execução em Python — Peso — esparso (grau ~3) (mediana, ms) 119
Tabela 40 – Qualidade da solução — Peso — esparso (grau ~3) 119
Tabela 41 – Tempo de execução em Python — Peso — denso (p=0.3) (mediana, ms) 119
Tabela 42 – Qualidade da solução — Peso — denso (p=0.3) 119
Tabela 43 – Tempo de execução em Python — Grafo esparso (grau ~3) (mediana, ms) 120
Tabela 44 – Qualidade da solução — Grafo esparso (grau ~3) 120
Tabela 45 – Tempo de execução em Python — Grafo médio (grau ~6) (mediana, ms) 120
Tabela 46 – Qualidade da solução — Grafo médio (grau ~6) 120
Tabela 47 – Tempo de execução em Python — Grafo denso (grau ~12) (mediana, ms) 121
Tabela 48 – Qualidade da solução — Grafo denso (grau ~12) 121
Tabela 49 – Expoente empírico de crescimento — Grafo 3 — Denso | COM perfeito | MCM = n 123

Tabela 50 – Expoente empírico de crescimento — Matriz 1 — Densa uniforme | custo in [1,100] 124
Tabela 51 – Expoente empírico de crescimento — Perfil 1 — Aleatório | permutações uniformes 124
Tabela 52 – Expoente empírico de crescimento — Matriz 1 — Densa uniforme | custo in [1,100] 124
Tabela 53 – Expoente empírico de crescimento — Peso — esparso (grau 3) . . 124
Tabela 54 – Tempo de execução em compilado (C/C++) — Grafo 1 — Esparso | SEM perfeito | MCM = 80% de n (mediana, ms) 131
Tabela 55 – Tempo de execução em compilado (C/C++) — Grafo 2 — Esparso | COM perfeito | MCM = n (mediana, ms) 131
Tabela 56 – Tempo de execução em compilado (C/C++) — Grafo 3 — Denso | COM perfeito | MCM = n (mediana, ms) 131
Tabela 57 – Tempo de execução em compilado (C/C++) — Matriz 1 — Densa uniforme | custo in [1,100] (mediana, ms) 131
Tabela 58 – Tempo de execução em compilado (C/C++) — Matriz 2 — Permutação barata | ótimo = n (mediana, ms) 132
Tabela 59 – Tempo de execução em compilado (C/C++) — Matriz 3 — Estruturada | custo = |i-j|+1 (mediana, ms) 132

LISTA DE ALGORITMOS

1 Algoritmo de Dijkstra (CORMEN et al., 2009) 25

2 Algoritmo de Bellman-Ford (CORMEN et al., 2009) 26

3 Algoritmo de Ford-Fulkerson 29

4 Algoritmo de caminhos aumentantes (MCBM) (HALIM; HALIM, 2013) . . 38

5 Algoritmo de Edmonds-Karp (HALIM; HALIM, 2013) 41

6 Algoritmo de Hopcroft-Karp: laço principal e grafo em camadas (BURKARD; DELL'AMICO; MARTELLO, 2009) 44

7 Algoritmo de Hopcroft-Karp: busca dos caminhos aumentantes disjuntos (BURKARD; DELL'AMICO; MARTELLO, 2009) 45

8 Método guloso simples (VAZIRANI, 2001) 47

9 Algoritmo de Karp-Sipser (KARP; SIPSER, 1981) 49

10 Algoritmo Húngaro (JUNGNICKEL, 2008) 52

11 Método AUGMENT (JUNGNICKEL, 2008) 53

12 Algoritmo de Jonker-Volgenant (LAPJV), em alto nível (JONKER; VOLGENANT, 1987) 56

13 Algoritmo de caminhos de custo mínimo sucessivos (JUNGNICKEL, 2008) 58

14 Adaptação do algoritmo 13 utilizando Dijkstra com técnica de potenciais 60

15 Algoritmo Threshold para o LBAP (BURKARD; DELL'AMICO; MARTELLO, 2009) 64

16 Dual_LBAP: algoritmo dual para o LBAP (BURKARD; DELL'AMICO; MARTELLO, 2009) 66

17 LBAP usando caminhos aumentantes (BURKARD; DELL'AMICO; MARTELLO, 2009) 68

18 Dijkstra modificado para caminhos aumentantes em LBAP (BURKARD; DELL'AMICO; MARTELLO, 2009) 70

19 Algoritmo de Gale-Shapley para o problema do emparelhamento estável (KLEINBERG; TARDOS, 2005) 74

20 Procedimento MAXMATCH (JUNGNICKEL, 2008) 79

21 Algoritmo de BLOSSOM (JUNGNICKEL, 2008) 80
22 Procedimento CONTRACT (JUNGNICKEL, 2008) 81
23 Procedimento AUGMENT (JUNGNICKEL, 2008) 81
24 Esquema primal-dual do algoritmo de Edmonds ponderado, em alto nível (LAWLER, 1976) 86
25 Algoritmo Path Growing (DRAKE; HOUGARDY, 2003) 89
26 Emparelhamento induzido máximo exato, via conjunto independente máximo em $L(G)^2$ (CAMERON, 1989) 93
27 Heurísticas gulosas para emparelhamento induzido: aleatória e de grau mínimo (DUCKWORTH; WORMALD; ZITO, 2002; HALLDóRSSON; RADHAKRISHNAN, 1994) 95
28 Geradores de grafos bipartidos para cardinalidade máxima 101
29 Geradores de matrizes de custo para atribuição linear e gargalo 103
30 Geradores de perfis de preferência para o casamento estável 104
31 Gerador de grafos gerais aleatórios no modelo $G(n, p)$ 105

LISTA DE ABREVIATURAS E SIGLAS

BFS	Busca em largura (*Breadth-First Search*)
DFS	Busca em profundidade (*Depth-First Search*)
FIFO	Primeiro a entrar, primeiro a sair (*First In, First Out*)
LAPJV	Algoritmo de Jonker-Volgenant para o LAP
LBAP	Problema de atribuição gargalo linear (*Linear Bottleneck Assignment Problem*)
LIFO	Último a entrar, primeiro a sair (*Last In, First Out*)
MCBM	Emparelhamento bipartido de cardinalidade máxima (*Maximum Cardinality Bipartite Matching*)
MCM	Emparelhamento de cardinalidade máxima (*Maximum Cardinality Matching*)
MIM	Emparelhamento induzido máximo (*Maximum Induced Matching*)
MIS	Conjunto independente máximo (*Maximum Independent Set*)
MVC	Cobertura mínima de vértices (*Minimum Vertex Cover*)
NP	Tempo polinomial não determinístico (*Nondeterministic Polynomial time*)
P	Tempo polinomial (*Polynomial time*)
PL	Programação linear
QAP	Problema de atribuição quadrática (*Quadratic Assignment Problem*)
SMP	Problema do casamento estável (*Stable Marriage Problem*)
SSP	Caminhos mínimos sucessivos (*Successive Shortest Paths*)

LISTA DE SÍMBOLOS

$G = (V, E)$	Grafo com conjunto de vértices V e conjunto de arestas E
$D = (V, A)$	Grafo direcionado com conjunto de arcos A
$\{u, v\}$	Aresta não direcionada entre os vértices u e v
(u, v)	Arco direcionado de u para v
V_1, V_2	Classes de cor (partes) de um grafo bipartido
$w(e)$	Peso (ou custo) da aresta e
M	Emparelhamento
$\|M\|$	Cardinalidade do emparelhamento M
$v(G)$	Cardinalidade máxima de um emparelhamento em G
$M \ominus P$	Diferença simétrica entre os conjuntos de arestas M e P
$\delta(u, v)$	Peso de um caminho mínimo de u a v
$c(u, v)$	Capacidade do arco (u, v) em uma rede de fluxo
G_f	Rede residual associada ao fluxo f
$c_f(u, v)$	Capacidade residual do arco (u, v)
s, t	Vértices fonte e sorvedouro de uma rede de fluxo
$L(G)$	Grafo de linha de G
$L(G)^2$	Quadrado do grafo de linha de G
$O(\cdot)$	Limite assintótico superior
n	Tamanho da instância
k	Expoente empírico de crescimento
R^2	Coeficiente de determinação do ajuste do expoente empírico

SUMÁRIO

1 Introdução 18
1.1 Objetivos 19
1.1.1 Objetivo geral 19
1.1.2 Objetivos específicos 19
1.2 Delimitação do estudo 19
1.3 Organização do trabalho 20
2 Fundamentação teórica 21
2.1 Definições sobre grafos 21
2.1.1 Grafo direcionado 21
2.1.2 Grafo ponderado 22
2.1.3 Grafo bipartido 22
2.2 Busca em grafos 22
2.2.1 Representação computacional 23
2.2.2 Busca em largura 23
2.2.3 Busca em profundidade 23
2.3 Caminhos mais curtos 24
2.3.1 Algoritmo de Dijkstra 24
2.3.2 Algoritmo de Bellman-Ford 26
2.4 Redes de fluxo em grafos 27
2.4.1 Definições básicas 27
2.4.2 O problema do fluxo máximo 27
2.4.2.1 Corte 27
2.4.2.2 Redes residuais 28
2.4.2.3 Caminhos aumentantes 28
2.4.3 Fluxo de custo mínimo 30
2.4.3.1 Condições de otimalidade e ciclos negativos 30
2.4.3.2 Potenciais de vértices e custos reduzidos 31
2.5 Problema Geral do Emparelhamento 31
2.5.1 Matching em grafos bipartidos 32
2.5.2 Formulação geral (emparelhamento e otimização) 32
2.6 Métodos exatos, aproximados e heurísticos 32
3 Problemas de emparelhamento em grafos bipartidos 34
3.1 Emparelhamento de cardinalidade máxima 34
3.1.1 Propriedades 34
3.1.1.1 Emparelhamento máximo vs maximal 35
3.1.1.2 Caminhos aumentantes 35

3.1.1.3 Teorema de Kőnig . . . 35
3.1.1.4 Teorema de Hall . . . 36
3.1.1.5 Teorema de Gallai e Conjunto Independente Máximo . . 36
3.1.2 Algoritmos exatos . . . 37
3.1.2.1 Algoritmo do caminho aumentante . . . 37
3.1.2.2 Redução ao problema de fluxo máximo . . . 39
3.1.2.3 Algoritmo de Edmonds-Karp . . . 40
3.1.2.4 Algoritmo de Dinic . . . 43
3.1.2.5 Algoritmo de Hopcroft-Karp . . . 43
3.1.3 Algoritmos aproximados . . . 46
3.1.3.1 Método guloso simples . . . 46
3.1.3.2 Algoritmo de Karp-Sipser . . . 48
3.2 Problema de atribuição (Assignment Problem) . . . 50
3.2.1 Algoritmos exatos . . . 50
3.2.1.1 Método Húngaro . . . 50
3.2.1.2 Jonker-Volgenant . . . 54
3.2.1.3 Redução para problema de fluxo de custo mínimo . . . 58
3.2.2 Heurísticas . . . 61
3.2.2.1 Heurística gulosa ordenada . . . 61
3.3 Problema de atribuição gargalo . . . 62
3.3.1 Soluções exatas . . . 62
3.3.1.1 Métodos de threshold . . . 62
3.3.1.2 Método dual . . . 65
3.3.1.3 Método de caminhos aumentantes . . . 67
3.3.2 Heurísticas . . . 71
3.3.2.1 Heurística Construtiva Gulosa . . . 71
3.4 Problema do emparelhamento estável . . . 72
3.4.1 Propriedades . . . 72
3.4.2 Algoritmo de Gale-Shapley . . . 74
4 Problemas de emparelhamento em grafos gerais . . . 76
4.1 Emparelhamento de cardinalidade máxima . . . 76
4.1.1 Propriedades . . . 76
4.1.1.1 Ciclos ímpares . . . 76
4.1.1.2 Flor (Blossom) . . . 77
4.1.2 O algoritmo de Edmonds . . . 77
4.1.2.1 Teorema de Tutte-Berge . . . 81
4.1.3 Algoritmos aproximados . . . 82
4.1.3.1 Algoritmo de crescimento de caminho (Path Growing) . 82
4.2 Emparelhamento ponderado máximo . . . 84

4.2.1 Formulação poliédrica e Restrições de Blossom 84
4.2.2 A abordagem Primal-Dual 84
4.2.3 O algoritmo de Edmonds Ponderado 85
4.2.4 Melhorias 88
4.2.5 Algoritmos aproximados 88
4.2.5.1 Algoritmo de Crescimento de Caminho (Path Growing) . 88
4.3 Emparelhamento induzido máximo 90
4.3.1 Propriedades 90
4.3.2 Complexidade computacional 92
4.3.3 Redução ao Problema do Conjunto Independente 92
4.3.4 Abordagens de solução e heurísticas 92
4.3.4.1 Algoritmo exato 93
4.3.4.2 Heurísticas gulosas 94
4.3.4.3 Metaheurísticas 96
5 Metodologia 98
5.1 Arquitetura do ambiente de testes 98
5.2 Interface padrão dos problemas 99
5.3 Geradores de instância 99
5.3.1 Grafos bipartidos 100
5.3.2 Matrizes de custo 102
5.3.3 Perfis de preferência 104
5.3.4 Grafos gerais aleatórios 105
5.4 Protocolo de medição 106
5.5 Critérios de comparação 107
5.6 Ambiente de execução e reprodutibilidade 108
5.7 Origem, adaptação e verificação dos algoritmos 109
6 Resultados e discussão 112
6.1 Cardinalidade máxima em grafos bipartidos 112
6.2 Atribuição linear 114
6.3 Atribuição gargalo 116
6.4 Casamento estável 117
6.5 Emparelhamento em grafos gerais 118
6.6 Emparelhamento induzido máximo 120
6.7 Defeitos em implementações públicas 121
6.8 Crescimento empírico frente à complexidade teórica 122
7 Trabalhos futuros 125

Referências 127

Apêndices **130**
APÊNDICE A Tempos das implementações de núcleo compilado 131
A.1 Cardinalidade máxima em grafos bipartidos 131
A.2 Atribuição linear 131

1 INTRODUÇÃO

Na teoria dos grafos e otimização combinatória, um emparelhamento em um grafo não-dirigido $G = (V, E)$ é definido formalmente como um subconjunto de arestas $M \subseteq E$ tal que nenhuma aresta em M compartilhe um vértice comum. Em termos precisos, para todo vértice $v \in V$, o grau de v no subgrafo induzido por M é no máximo 1. Esta definição fundamental é apresentada por Lovász e Plummer (1986) em sua obra seminal sobre a teoria dos emparelhamentos, onde também se estabelece a distinção clássica entre emparelhamentos em grafos bipartidos — cujos vértices podem ser divididos em dois conjuntos disjuntos U e V — e em grafos gerais.

Os problemas de emparelhamento têm aplicações significativas em várias áreas, como biologia computacional, onde podem ser empregados na diferenciação e classificação de estruturas proteicas (TAYLOR, 2002); redes sociais, para identificação de comunidades ou grupos (FAN, 2012); e visão computacional, para correspondências entre elementos de conjuntos distintos, como pontos em imagens ou vértices em malhas tridimensionais (HALLER et al., 2022).

É crucial, no entanto, fazer uma distinção precisa quanto à complexidade computacional desses problemas. Ao contrário do que uma análise superficial poderia sugerir, o problema de encontrar um emparelhamento de cardinalidade ou peso máximo pertence à classe de complexidade **P** (tempo polinomial), sendo solúvel por algoritmos clássicos como o Método Húngaro ou o Algoritmo de *Blossom* de Edmonds (SCHRIJVER, 2004). O verdadeiro desafio computacional, frequentemente classificado como **NP-Difícil**, surge em variações estruturais mais complexas, como o Emparelhamento Tridimensional ou o Problema da Atribuição Quadrática (QAP). Segundo Garey e Johnson (1990) e Burkard, Dell'Amico e Martello (2009), o QAP modela situações onde o custo depende da interação entre os pares escolhidos, tornando métodos exatos inviáveis para grandes instâncias e exigindo o uso de relaxações ou heurísticas avançadas.

Diante desse cenário, este trabalho visa sistematizar os principais problemas e métodos computacionais associados ao emparelhamento em grafos. Ele busca oferecer uma comparação detalhada entre suas aplicações e limitações, contribuindo para o entendimento aprofundado dos algoritmos disponíveis e identificando oportunidades para o desenvolvimento de novas soluções. Essa abordagem torna-se ainda mais relevante à luz da importância prática desses algoritmos para resolver problemas reais em grande escala, conforme evidenciado por estudos recentes (SUSSMAN; LI, 2021).

1.1 OBJETIVOS

Os objetivos deste trabalho dividem-se em um objetivo geral, que expressa o propósito do estudo como um todo, e em objetivos específicos, que o decompõem nas etapas necessárias para alcançá-lo.

1.1.1 Objetivo geral

Este trabalho tem como objetivo principal levantar e organizar os principais problemas de emparelhamento em grafos e os métodos computacionais presentes na literatura para resolvê-los, implementando esses métodos e comparando-os experimentalmente quanto ao desempenho e à qualidade das soluções.

1.1.2 Objetivos específicos

- Mapear as variações e características dos principais problemas de emparelhamento em grafos presentes na literatura.
- Classificar esses problemas e os métodos computacionais que os resolvem, segundo critérios como a estrutura do grafo, o objetivo de otimização e o tipo de método (exato, aproximado ou heurístico).
- Implementar os algoritmos selecionados, adaptando implementações públicas ou, quando não houver implementação adequada, partindo da descrição bibliográfica.
- Criar um ambiente de testes unificado e, nele, comparar os algoritmos quanto ao tempo de execução, à complexidade assintótica de tempo e espaço e à qualidade das soluções encontradas.
- Organizar os resultados em um formato que facilite o acesso e a compreensão dos tópicos abordados por pesquisadores e profissionais.

1.2 DELIMITAÇÃO DO ESTUDO

Os problemas de emparelhamento em grafos podem ser classificados de diversas maneiras, como sendo em grafos bipartidos ou não-bipartidos, ponderados ou não-ponderados, direcionados ou não-direcionados, problemas em multigrafos, problemas em hipergrafos, entre outros. Este trabalho focará em problemas que envolvem grafos menos complexos, excluindo multigrafos e hipergrafos. Também ficam fora do escopo outras variações do emparelhamento em grafos gerais, como o b-emparelhamento, o emparelhamento em matroides e o problema dos colegas de quarto estáveis (*Stable Roommates Problem*).

1.3 ORGANIZAÇÃO DO TRABALHO

O restante deste trabalho está organizado da seguinte forma:

- **Capítulo 2:** apresenta a fundamentação teórica: definições sobre grafos, algoritmos de busca e de caminhos mais curtos, redes de fluxo, o problema geral do emparelhamento e a classificação dos métodos em exatos, aproximados e heurísticos.
- **Capítulo 3:** descreve os problemas de emparelhamento em grafos bipartidos (cardinalidade máxima, atribuição, atribuição gargalo e emparelhamento estável), com suas propriedades, seus algoritmos em pseudocódigo e uma tabela-resumo por problema.
- **Capítulo 4:** faz o mesmo para os problemas em grafos gerais (cardinalidade máxima, peso máximo e emparelhamento induzido máximo).
- **Capítulo 5:** descreve o ambiente de testes construído para comparar os algoritmos: arquitetura, interface padrão, geradores de instâncias, protocolo de medição, critérios de comparação e origem de cada implementação.
- **Capítulo 6:** apresenta e discute os resultados obtidos no ambiente de testes, quanto ao tempo de execução, à qualidade das soluções e ao crescimento empírico frente à complexidade teórica.
- **Capítulo 7:** aponta as extensões deste trabalho que ficaram como trabalhos futuros.
- **Apêndice A:** reúne as tabelas de tempo das implementações de núcleo compilado, usadas como referência.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo reúne os conceitos de teoria dos grafos e de otimização combinatória sobre os quais os capítulos seguintes se apoiam. Inicia-se pelas definições básicas sobre grafos e suas variações (direcionados, ponderados e bipartidos). Em seguida, apresentam-se os algoritmos de busca em largura e em profundidade, usados pelos algoritmos de emparelhamento para encontrar caminhos aumentantes; os algoritmos de caminhos mais curtos, usados nas reduções do problema de atribuição a fluxo de custo mínimo; e as redes de fluxo, por meio das quais o emparelhamento bipartido pode ser resolvido. Por fim, define-se o problema geral do emparelhamento e a classificação dos métodos de solução em exatos, aproximados e heurísticos, usada ao longo do trabalho.

2.1 DEFINIÇÕES SOBRE GRAFOS

Um **grafo** G é uma estrutura definida por um par $G = (V, E)$, consistindo de um conjunto finito V de elementos chamados **vértices** (ou **nós** ou **pontos**) e um conjunto (ou família) E de pares não ordenados de vértices, chamados de **arestas** (JUNGNICKEL, 2008). O conjunto V é o conjunto de vértices de G e E é a família de arestas de G.

No contexto deste projeto, assumimos que as arestas $e = \{u, v\}$ são pares não ordenados de vértices distintos. A notação de par ordenado (u, v) fica reservada aos arcos de grafos direcionados (Seção 2.1.1). Se uma aresta e conecta os vértices a e b, diz-se que a e b são **adjacentes** ou **vizinhos**, e que a aresta e é **incidente** a a e b (SCHRIJVER, 2004).

O conceito de grafo é amplamente utilizado como uma representação abstrata concisa para estruturas complexas e para codificar relações binárias entre um conjunto de objetos.

2.1.1 Grafo direcionado

Se as relações entre os vértices forem assimétricas, utilizamos um **grafo direcionado** ou **dígrafo**, $D = (V, A)$. Neste caso, A é uma família de pares ordenados de vértices, chamados **arcos** (ou arestas direcionadas). Para um arco $e = (u, v)$, u é chamado o vértice inicial ou **cauda** (tail), e v é o vértice final ou **cabeça** (head) (MANBER, 1989)

2.1.2 Grafo ponderado

Um grafo $G = (V, E)$ é chamado **grafo ponderado** (ou com pesos) se uma **função de peso** (ou função de custo, ou função de comprimento) $w : E \to R$ está associada às arestas. Formalmente: $G = (V, E, w)$, onde $w : E \to R$. Geralmente, o peso $w(e)$ de uma aresta e é um valor real não negativo que representa o custo, comprimento ou capacidade associada àquela aresta (ROSEN; KREHER; STINSON, 1998).

Grafos ponderados podem ser representados por matrizes de adjacência de valor real A, onde a entrada A_{ij} é o peso w_{ij} da aresta (ZASLAVSKIY; BACH; VERT, 2009).

2.1.3 Grafo bipartido

Um grafo bipartido é um grafo não direcionado que pode ser facilmente colorido com apenas duas cores (MANBER, 1989).

Formalmente, um grafo $G = (V, E)$ é chamado **grafo bipartido** se o conjunto de vértices V puder ser particionado em dois subconjuntos disjuntos V_1 e V_2, chamados classes de cor, tais que:

$$V = V_1 \cup V_2 \quad e \quad V_1 \cap V_2 = \emptyset \tag{2.1}$$

e todas as arestas $e \in E$ conectam um vértice em V_1 a um vértice em V_2. Ou seja, não existem arestas que conectem vértices dentro do mesmo subconjunto (DASGUPTA; VAZIRANI, 2006).

Observações:

- Um grafo é bipartido se e somente se não contém ciclos de comprimento ímpar (SCHRIJVER, 2004)
- O grafo bipartido completo $K_{m,n}$ possui m vértices em V_1 e n vértices em V_2, e contém todas as arestas possíveis entre V_1 e V_2 (JUNGNICKEL, 2008)

2.2 BUSCA EM GRAFOS

A exploração sistemática dos vértices e arestas de um grafo é uma sub-rotina fundamental para a maioria dos algoritmos em redes, incluindo aqueles utilizados para encontrar emparelhamentos, fluxo máximo e componentes conexos. A busca em grafos consiste em seguir as arestas do grafo para visitar os vértices, permitindo descobrir a estrutura da rede e propriedades de conectividade (CORMEN et al., 2009). Neste trabalho, a busca em largura e a busca em profundidade são as sub-rotinas com

que os algoritmos de emparelhamento procuram caminhos aumentantes: o algoritmo de caminhos aumentantes (Algoritmo 4), o de Edmonds-Karp (Algoritmo 5), o de Hopcroft-Karp (Algoritmos 6 e 7) e o Blossom (Algoritmo 20).

2.2.1 Representação computacional

A complexidade das buscas depende de como o grafo $G = (V, E)$ é representado (AHO; HOPCROFT; ULLMAN, 1974). Na **matriz de adjacência**, uma matriz A de dimensão $|V| \times |V|$ tem $A_{ij} = 1$ se existe uma aresta $\{i, j\} \in E$, e 0 caso contrário; verificar a existência de uma aresta custa $O(1)$, mas o espaço é $\Theta(V^2)$, o que a torna ineficiente para grafos esparsos. Na **lista de adjacência**, cada vértice u tem uma lista $Adj[u]$ com todos os vértices v tais que $\{u, v\} \in E$, e o espaço é $\Theta(V + E)$ (CORMEN et al., 2009). Os algoritmos de busca descritos a seguir assumem listas de adjacência, o que torna sua complexidade linear no tamanho do grafo.

2.2.2 Busca em largura

A **busca em largura**, ou *Breadth-First Search* (BFS), explora o grafo em camadas: dado um vértice fonte s, visita primeiro todos os vizinhos de s (camada L_1), depois os vizinhos desses vizinhos (camada L_2), e assim sucessivamente (KLEINBERG; TARDOS, 2005). Para isso, guarda a fronteira de exploração em uma fila (FIFO). Como consequência, a BFS calcula o caminho mais curto, em número de arestas, de s a todos os vértices alcançáveis (DASGUPTA; VAZIRANI, 2006), propriedade usada pelos algoritmos de Edmonds-Karp e de Hopcroft-Karp para encontrar os caminhos aumentantes mais curtos. Sua complexidade é $O(V + E)$, pois cada vértice entra na fila no máximo uma vez e cada lista de adjacência é percorrida uma única vez (CORMEN et al., 2009).

2.2.3 Busca em profundidade

A **busca em profundidade**, ou *Depth-First Search* (DFS), explora a cada passo as arestas do vértice descoberto mais recentemente que ainda tenha arestas inexploradas: aprofunda-se no grafo tanto quanto possível e, quando não há mais para onde ir, retrocede (*backtracking*) para explorar outros caminhos (MANBER, 1989). Pode ser implementada de forma recursiva ou com uma pilha (LIFO). É a estratégia do algoritmo de caminhos aumentantes e da segunda fase do Hopcroft-Karp, que seguem um caminho até o fim antes de tentar outro. Assim como a BFS, tem complexidade $\Theta(V + E)$ com listas de adjacência (CORMEN et al., 2009).

2.3 CAMINHOS MAIS CURTOS

O problema de encontrar o caminho mais curto entre dois vértices em um grafo é um dos problemas de otimização combinatória mais estudados, servindo como sub-rotina para diversas aplicações, incluindo roteamento de redes e algoritmos de fluxo. Neste trabalho, os caminhos mais curtos aparecem na solução do problema de atribuição por fluxo de custo mínimo: o algoritmo de caminhos mínimos sucessivos (Algoritmo 13) busca, a cada iteração, um caminho de custo mínimo na rede residual, com o algoritmo de Dijkstra sobre custos reduzidos (Algoritmo 14) ou com o de Bellman-Ford, quando há custos negativos. O algoritmo de Dijkstra é também a base do procedimento de caminhos aumentantes do problema de atribuição gargalo (Algoritmo 18).

Seja $G = (V, E)$ um grafo ponderado com uma função de peso $w : E \to \mathbb{R}$. O peso de um caminho $p = \langle v_0, v_1 \ldots, v_k \rangle$ é a soma dos pesos de suas arestas constituintes: $w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$. O **caminho mais curto** de um vértice u a um vértice v é definido como qualquer caminho p com peso mínimo $\delta(u, v)$ (CORMEN et al., 2009).

2.3.1 Algoritmo de Dijkstra

Os algoritmos de caminho mínimo baseiam-se na propriedade de **subestrutura ótima**, segundo a qual subcaminhos de um caminho mais curto são, por si mesmos, caminhos mais curtos, e na técnica de **relaxamento** (KLEINBERG; TARDOS, 2005). Para cada vértice v, mantém-se uma estimativa $v.d$, que é um limite superior para o peso do caminho mais curto da fonte s até v, e um predecessor $v.\pi$. Relaxar um arco (u, v) consiste em testar se o caminho até u seguido do arco (u, v) melhora a estimativa de v e, se melhorar, atualizá-la:

$$\text{se } v.d > u.d + w(u, v), \text{ então } v.d \leftarrow u.d + w(u, v) \text{ e } v.\pi \leftarrow u \tag{2.2}$$

Nos pseudocódigos a seguir, essa operação aparece como Relax(u, v, w) e custa $O(1)$. Seguindo os predecessores a partir de um vértice qualquer até a fonte, obtém-se o caminho mais curto encontrado até o momento. Os algoritmos de Dijkstra e de Bellman-Ford diferem apenas na ordem e na frequência com que relaxam os arcos (CORMEN et al., 2009).

Para grafos onde os pesos das arestas são não-negativos ($w(u, v) \geq 0$), o Algoritmo de Dijkstra é a abordagem mais eficiente. Ele utiliza uma estratégia gulosa, mantendo um conjunto de vértices cujos pesos finais dos caminhos mais curtos já foram determinados (AHO; HOPCROFT; ULLMAN, 1974).

O algoritmo seleciona repetidamente o vértice u com a menor estimativa de caminho mais curto $d[u]$ de uma fila de prioridade, adiciona-o ao conjunto de visitados e relaxa todas as arestas que saem de u. Dasgupta e Vazirani (2006) destacam que a eficiência do Dijkstra depende da estrutura de dados utilizada. Com um Heap Binário, a complexidade é $O((V + E) \log V)$. Cormen et al. (2009) apontam que, ao utilizar um Heap de Fibonacci, a complexidade amortizada pode ser reduzida para $O(E + VlogV)$, o que é crucial para grafos densos.

Algoritmo 1: Algoritmo de Dijkstra (CORMEN et al., 2009)

```
(1)  Procedure Dijkstra(G, w, s)
(2)      Initialize-Single-Source(G, s)
(3)      S ← ∅
(4)      Q ← G.V
(5)      while Q ≠ ∅ do
(6)          u ← Extract-Min(Q)
(7)          S ← S ∪ {u}
(8)          foreach vertex v ∈ G.Adj[u] do
(9)              Relax(u, v, w)
(10)         end
(11)     end
```

O Algoritmo 1 funciona da seguinte forma (CORMEN et al., 2009):

- **Inicialização (linhas 2–4):** Initialize-Single-Source atribui $v.d = \infty$ e $v.\pi =$ NIL a todo vértice, e $s.d = 0$ à fonte. O conjunto S dos vértices com distância final já determinada começa vazio, e a fila de prioridade Q, ordenada pelo atributo d, começa com todos os vértices.

- **Escolha gulosa (linhas 5–7):** a cada iteração, retira-se de Q o vértice u de menor estimativa $u.d$ e ele é incluído em S. Como os pesos são não-negativos, nenhum caminho que passe por outro vértice de Q pode chegar a u com peso menor, e por isso $u.d = \delta(s, u)$ nesse momento.

- **Relaxamento (linhas 8–9):** todos os arcos que saem de u são relaxados (Equação 2.2), o que pode diminuir a estimativa dos vizinhos de u ainda em Q e, com isso, alterar suas posições na fila.

- **Complexidade:** cada vértice é retirado de Q exatamente uma vez, totalizando V operações Extract-Min, e cada arco é relaxado exatamente uma vez, quando sua origem é retirada, totalizando até E diminuições de chave na fila. Com um heap binário, as duas operações custam $O(\log V)$, o que resulta em $O((V + E) \log V)$. Com um heap de Fibonacci, a diminuição de chave custa $O(1)$ amortizado, o que resulta em $O(E + V \log V)$.

2.3.2 Algoritmo de Bellman-Ford

Quando o grafo contém arestas com pesos negativos, o algoritmo de Dijkstra não garante a corretude. Nesses casos, utiliza-se o Algoritmo de Bellman-Ford. Segundo Manber (1989), este algoritmo baseia-se em programação dinâmica, relaxando todas as arestas do grafo $|V| - 1$ vezes.

A propriedade fundamental do Bellman-Ford é sua capacidade de detectar ciclos negativos. Se, após $|V| - 1$ iterações, ainda for possível relaxar alguma aresta, o grafo contém um ciclo com peso total negativo acessível a partir da fonte, o que implica que $\delta(s, v) = -\infty$ para alguns vértices (KLEINBERG; TARDOS, 2005). Sua complexidade é $O(VE)$.

Algoritmo 2: Algoritmo de Bellman-Ford (CORMEN et al., 2009)

```
(1)  Procedure Bellman-Ford(G, w, s)
(2)      Initialize-Single-Source(G, s)
(3)      for i = 1 to |G.V| − 1 do
(4)          foreach edge (u, v) ∈ G.E do
(5)              Relax(u, v, w)
(6)          end
(7)      end
(8)      foreach edge (u, v) ∈ G.E do
(9)          if v.d > u.d + w(u, v) then
(10)             return false
(11)         end
(12)     end
(13)     return true
```

O Algoritmo 2 funciona da seguinte forma (CORMEN et al., 2009):

- **Inicialização (linha 2):** igual à do algoritmo de Dijkstra: $s.d = 0$ e $v.d = \infty$ para os demais vértices.
- **Relaxamentos (linhas 3–5):** todos os arcos são relaxados, em qualquer ordem, $|V| - 1$ vezes. Após a i-ésima passagem, todo vértice cujo caminho mais curto a partir de s tem no máximo i arcos já tem sua distância correta. Como um caminho mais curto sem ciclos tem no máximo $|V| - 1$ arcos, $|V| - 1$ passagens bastam quando não há ciclo negativo.
- **Detecção de ciclo negativo (linhas 8–10):** uma passagem extra verifica se algum arco ainda pode ser relaxado. Se puder, existe um ciclo de peso negativo alcançável a partir de s, e o algoritmo retorna **false**.
- **Retorno (linha 13):** caso contrário, retorna **true**, e os atributos $v.d$ e $v.\pi$ descrevem os caminhos mais curtos.

- **Complexidade:** a inicialização custa $O(V)$; o laço principal faz $|V| - 1$ passagens por todos os E arcos, com relaxamento de custo $O(1)$, totalizando $O(VE)$; a verificação final custa $O(E)$. O total é $O(VE)$.

2.4 REDES DE FLUXO EM GRAFOS

A teoria dos fluxos em redes é um ramo fundamental da otimização combinatória que modela problemas de transporte de recursos através de um sistema conectado. Esta seção define os conceitos preliminares e apresenta os teoremas centrais que sustentam os algoritmos utilizados neste trabalho. Neste trabalho, as redes de fluxo fundamentam duas reduções: a do emparelhamento bipartido de cardinalidade máxima a um problema de fluxo máximo, resolvido pelo algoritmo de Edmonds-Karp (Algoritmo 5), e a do problema de atribuição a um problema de fluxo de custo mínimo (Seção 3.2.1.3).

2.4.1 Definições básicas

Uma rede de fluxo é definida por um grafo direcionado $G = (V, E)$, onde V é o conjunto de vértices e E é o conjunto de arestas. Distinguem-se dois vértices especiais: a **fonte** $s \in V$. que produz o fluxo, e o **sorvedouro** $t \in V$, que o consome. Para cada aresta $(u, v) \in E$, associa-se uma capacidade não-negativa $c(u, v) \geq 0$, que limita a quantidade de fluxo que pode atravessar aquela aresta. Um **fluxo** em G é uma função $f : E \to \mathbb{R}^+$ que satisfaz as seguintes propriedades (CORMEN et al., 2009):

- **Restrição de capacidade:** O fluxo em uma aresta não pode exceder a sua capacidade.
$$0 \leq f(u, v) \leq c(u, v), \qquad \forall (u, v) \in E \tag{2.3}$$
- **Conservação de fluxo:** para todo vértice $v \in V - \{s, t\}$, a quantidade total de fluxo que entra deve ser igual à que sai
$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v) \tag{2.4}$$

2.4.2 O problema do fluxo máximo

O **problema do fluxo máximo** consiste em encontrar um fluxo f tal que $|f|$ seja o maior possível. Para entender a limitação desse fluxo, precisamos entender os conceitos de corte, redes residuais e caminhos aumentantes.

2.4.2.1 Corte

Um **corte** (S, T) é uma partição do conjunto de vértices V em dois subconjuntos disjuntos S e T, tal que $s \in S$ e $t \in T$. A **capacidade do corte**, denotada por $c(S, T)$, é a soma das capacidades das arestas que partem de S para T (CORMEN et al., 2009):

$$c(S,T) = \sum_{u \in S, v \in T} c(u,v) \tag{2.5}$$

O **corte mínimo** de uma rede N é um corte cuja capacidade é a menor entre todos os cortes possíveis nessa rede (CORMEN et al., 2009). Isso nos leva ao **teorema do fluxo máximo e corte mínimo (Max-flow Min-cut)**

> *O valor máximo do fluxo de uma rede N é igual à capacidade do corte mínimo em N (JUNGNICKEL, 2008)*

Este teorema é fundamental para a compreensão e corretude dos algoritmos de caminhos aumentantes.

2.4.2.2 Redes residuais

A maioria dos algoritmos de resolução de problemas de fluxo máximo se baseia no conceito de redes residuais. Uma **rede residual** G_f consiste em arestas com capacidades que representam como podemos alterar o fluxo das arestas em G. A capacidade das arestas da rede residual G_f corresponde à diferença entre a capacidade da aresta original de G e o fluxo f que está passando pela aresta (CORMEN et al., 2009). Ou seja:

$$c_f(u,v) = c(u,v) - f(u,v) \tag{2.6}$$

Quando a aresta (u,v) possui fluxo igual à sua capacidade, então $c_f(u,v) = 0$ e a aresta não é incluída no grafo residual G_f

2.4.2.3 Caminhos aumentantes

Dado um grafo $G = (V,E)$ e um fluxo f, um **caminho aumentante** é um caminho de s para t na rede residual G_f. Por definição, podemos aumentar o fluxo em uma aresta (u,v) de um caminho aumentante em até $c_f(u,v)$ sem violar a capacidade da aresta original em G (CORMEN et al., 2009).

O método de Ford-Fulkerson, que é base dos principais algoritmos de resolução do problema do fluxo máximo, iterativamente encontra um caminho aumentante, calcula sua capacidade residual e adiciona esse fluxo à solução atual até que não existam mais caminhos aumentantes (AHUJA; MAGNANTI; ORLIN, 1993).

Algoritmo 3: Algoritmo de Ford-Fulkerson

```
(1)  Procedure FORD-FULKERSON(G, s, t)
(2)      foreach edge (u, v) ∈ G.E do
(3)          (u, v).f ← 0
(4)      end
(5)      while there exists a path p from s to t in the residual network G_f do
(6)          c_f(p) ← min{c_f(u, v) : (u, v) is in p}
(7)          foreach edge (u, v) ∈ p do
(8)              if (u, v) ∈ E then
(9)                  (u, v).f ← (u, v).f + c_f(p)
(10)             else
(11)                 (v, u).f ← (v, u).f − c_f(p)
(12)             end
(13)         end
(14)     end
```

O Algoritmo 3 funciona da seguinte forma (CORMEN et al., 2009):

- **Inicialização (linhas 2–3):** o fluxo começa nulo em todos os arcos, o que satisfaz trivialmente a restrição de capacidade e a conservação de fluxo.
- **Busca de caminho aumentante (linha 5):** enquanto houver um caminho p de s a t na rede residual G_f, o fluxo atual não é máximo. O método não especifica como o caminho é encontrado; qualquer busca, como BFS ou DFS, serve.
- **Capacidade residual do caminho (linha 6):** $c_f(p)$ é a menor capacidade residual entre os arcos de p, ou seja, o máximo de fluxo que se pode enviar por p sem violar nenhuma capacidade.
- **Aumento (linhas 7–11):** para cada arco de p, se ele pertence à rede original (linha 8), o fluxo nele aumenta em $c_f(p)$ (linha 9); caso contrário, ele é um arco reverso da rede residual, e enviar fluxo por ele equivale a cancelar fluxo do arco original (v, u) (linha 11). Em ambos os casos, o valor $|f|$ do fluxo aumenta em $c_f(p)$.
- **Complexidade:** com capacidades inteiras, cada iteração aumenta $|f|$ em pelo menos uma unidade, de modo que há no máximo $|f^*|$ iterações, em que f^* é o fluxo máximo. Cada iteração encontra o caminho e atualiza o fluxo em $O(E)$, o que resulta em $O(E \cdot |f^*|)$. Quando o caminho é sempre o mais curto em número de arcos, encontrado por BFS, obtém-se o algoritmo de Edmonds-Karp (Algoritmo 5), com complexidade $O(VE^2)$, independente das capacidades.

2.4.3 Fluxo de custo mínimo

Uma outra variação dos problemas em redes de fluxo em grafos é o problema do fluxo de custo mínimo. Essa variação considera, além das capacidades disponíveis, uma dimensão econômica. Seja $G = (V, E)$ uma rede direcionada onde cada aresta (u, v) possui uma capacidade $c(u, v)$ e um custo unitário $w(u, v)$. O objetivo é transmitir uma quantidade de fluxo pré-determinada F da fonte s ao sorvedouro t com o menor custo total possível.

A formulação linear do problema é dada por Ahuja, Magnanti e Orlin (1993). O primeiro objetivo é minimizar o custo total do fluxo, que segue a fórmula:

$$\text{Minimizar o custo do fluxo:} \quad z(f) = \sum_{(u,v)\in A} w(u, v) \cdot f(u, v) \tag{2.7}$$

A função a seguir assegura que o nó fonte s gere exatamente a demanda F, e que o nó sorvedouro t absorva-a integralmente.

$$\sum_{i,j\in V} f(i, j) - \sum_{k,i\in V} f(k, i) = \begin{cases} F & \text{se } i = s \\ -F & \text{se } j = t \\ 0 & \text{caso contrário} \end{cases} \tag{2.8}$$

Além desta, é importante lembrar da equação 2.3, que define que o fluxo em uma aresta não pode exceder a sua capacidade.

2.4.3.1 Condições de otimalidade e ciclos negativos

O critério fundamental para verificar se uma solução é ótima é o **teorema dos ciclos negativos**:

> Um fluxo f é uma solução viável para o problema do fluxo de custo mínimo se e somente se ele satisfaz a condição de otimalidade de ciclo negativo: a rede residual $G(f)$ não contém um ciclo de custo negativo (AHUJA; MAGNANTI; ORLIN, 1993)

Este teorema é a base dos algoritmos de **cancelamento de ciclos** (Cycle Canceling), que iterativamente identificam ciclos negativos na rede residual e enviam fluxo através deles para reduzir o custo total da função objetivo até que nenhum ciclo negativo reste.

2.4.3.2 Potenciais de vértices e custos reduzidos

Embora o cancelamento de ciclos seja teoricamente sólido, a detecção de ciclos negativos (via Bellman-Ford) pode ser computacionalmente custosa. Para permitir o uso de algoritmos mais eficientes como o de Dijkstra, a literatura introduz o conceito de **potenciais de vértices**, baseando-se na teoria de dualidade da programação linear.

A ideia é manter um **potencial** para cada vértice v do grafo, chamado de $h(v)$, com valor inicial 0. Definimos o **custo reduzido** de uma aresta (u, v) como $\gamma'(u, v) = \gamma(u, v) + h(u) - h(v)$. Se os potenciais forem escolhidos de forma adequada, todos os custos reduzidos podem ser garantidos como não-negativos, permitindo o uso do algoritmo de Dijkstra. Além disso, é garantido que os caminhos de custo mínimo no grafo original e no grafo com custos reduzidos são os mesmos. (JUNGNICKEL, 2008).

O conceito de potenciais e de custo reduzido será utilizado em algoritmos como o de caminhos sucessivos para resolução do problema de atribuição na seção 3.2.1.3.

2.5 PROBLEMA GERAL DO EMPARELHAMENTO

O problema de **matching** (emparelhamento) em grafos é um problema fundamental na otimização combinatória. Esta seção define o emparelhamento e os conceitos associados a ele, usados em todo o restante do trabalho, e apresenta o caso bipartido, sua versão ponderada (o problema de atribuição) e uma formulação mais geral, de correspondência entre grafos.

Um **matching** M em um grafo não-direcionado $G = (V, E)$ é um subconjunto de arestas $M \subseteq E$ tal que nenhum par de arestas em M compartilha um vértice comum (KLEINBERG; TARDOS, 2005). Em outras palavras, cada nó aparece em no máximo uma aresta de M. A **cardinalidade** de um matching M é a quantidade de arestas contidas dentro dele, e um dos problemas que serão mostrados posteriormente trata de encontrar o **matching de cardinalidade máxima** (Maximum Matching) de um grafo G, que é o matching M com o maior número possível de arestas (CORMEN et al., 2009). A cardinalidade máxima de um matching de um grafo G é denominada por $v(G)$ (SCHRIJVER, 2004).

Um vértice é chamado **coberto** (*matched*) se for incidente a uma aresta em M; caso contrário, é **descoberto** (*unmatched* ou *exposed*) (CORMEN et al., 2009). Um **matching perfeito** é um matching em que todos os vértices do grafo estão cobertos (KLEINBERG; TARDOS, 2005).

O **problema de matching ponderado** (Weighted Matching Problem) envolve encontrar um matching para o qual a soma dos pesos das arestas é máxima. Em um grafo ponderado $G = (V, E, w)$, busca-se um $M \subseteq E$ que maximize $w(M)$ (MANDULAK et al., 2024).

2.5.1 Matching em grafos bipartidos

O problema de **Matching Bipartido** é o caso clássico de encontrar um matching de cardinalidade máxima em um grafo bipartido $G = (V, E)$, onde $V = X \cup Y$ (KLEINBERG; TARDOS, 2005). Pode ser utilizado para modelar situações de atribuição, como associar empregos (X) a máquinas (Y), ou professores (X) a cursos (Y), onde uma aresta indica uma capacidade de atribuição (KLEINBERG; TARDOS, 2005).

Quando aplicado em grafos ponderados, o matching bipartido é conhecido como **problema de atribuição** (LAWLER, 1976), que trata de encontrar uma combinação ótima de atribuições de forma a minimizar o custo total associado. Mais detalhes sobre esse problema serão vistos na Seção 3.2.

2.5.2 Formulação geral (emparelhamento e otimização)

Em um contexto mais amplo, o emparelhamento de grafos pode ser formalizado como um problema de otimização que busca maximizar a compatibilidade entre dois grafos G e G' (CAETANO et al., 2009). Esse problema é frequentemente abordado como um **problema de atribuição quadrática (QAP)**. Essa formulação busca maximizar uma função objetivo que combina termos de compatibilidade unária (nó-nó, $c_{ii'}$) e compatibilidade par a par (aresta-aresta, $d_{ii'jj'}$), sujeito a restrições de atribuição binária ($y_{ii'} \in \{0, 1\}$). O termo quadrático codifica a preservação das relações (arestas) entre os nós (CAETANO et al., 2009)

2.6 MÉTODOS EXATOS, APROXIMADOS E HEURÍSTICOS

Os métodos de solução apresentados nos capítulos seguintes são classificados em três tipos, de acordo com a garantia que oferecem sobre a qualidade da solução (VAZIRANI, 2001):

- **Exato:** sempre encontra uma solução ótima. É o caso, por exemplo, dos algoritmos de Hopcroft-Karp e Húngaro.
- **Aproximado:** encontra, em tempo polinomial, uma solução com qualidade garantida em relação ao ótimo. Em um problema de maximização, um algoritmo é dito uma ρ-aproximação, com $0 < \rho \leq 1$, se para toda instância a solução S encontrada satisfaz $\mathrm{val}(S) \geq \rho \cdot \mathrm{OPT}$, em que OPT é o valor da solução ótima. O método guloso simples para emparelhamento de cardinalidade máxima, por exemplo, é uma $\frac{1}{2}$-aproximação.
- **Heurístico:** encontra uma solução viável, geralmente de boa qualidade na prática, mas sem garantia provada de proximidade do ótimo.

A distinção entre aproximado e heurístico depende do problema, e não apenas do método: o método guloso ordenado tem garantia de $\frac{1}{2}$ no emparelhamento de peso máximo, mas não tem garantia na atribuição de custo mínimo, como discutido na Seção 3.2.2.1.

3 PROBLEMAS DE EMPARELHAMENTO EM GRAFOS BIPARTIDOS

Este capítulo apresenta os quatro problemas de emparelhamento em grafos bipartidos estudados neste trabalho. Todos compartilham a mesma estrutura de entrada, um grafo cujos vértices se dividem em dois conjuntos disjuntos, com arestas apenas entre eles, e diferem quanto ao objetivo da otimização:

- **Emparelhamento de cardinalidade máxima:** maximizar o número de arestas do emparelhamento, sem pesos;
- **Problema de atribuição:** encontrar o emparelhamento perfeito de menor custo total;
- **Problema de atribuição gargalo:** encontrar o emparelhamento perfeito cuja aresta mais cara seja a mais barata possível;
- **Emparelhamento estável:** encontrar um emparelhamento em que nenhum par prefira estar junto a permanecer com os parceiros atuais, a partir de listas de preferências.

Cada problema é apresentado na mesma sequência: a definição e as propriedades necessárias aos algoritmos, os algoritmos exatos, os algoritmos aproximados e as heurísticas, classificados conforme a Seção 2.6, e uma tabela-resumo com a complexidade, o tipo e uma breve descrição de cada método.

3.1 EMPARELHAMENTO DE CARDINALIDADE MÁXIMA

Dado um grafo não-direcionado $G = (V, E)$, o problema de emparelhamento de cardinalidade máxima (MCM) busca encontrar um emparelhamento $M \subseteq E$ tal que o número de arestas em M seja maximizado, ou seja, $|M|$ é o maior entre todos os emparelhamentos possíveis em G.

Um caso especial, mas importante, do MCM é o **Emparelhamento perfeito**, que é um emparelhamento onde todos os vértices do grafo são cobertos por exatamente uma aresta do emparelhamento. Para que um emparelhamento perfeito exista, o grafo deve ter um número par de vértices e o emparelhamento máximo deve ter cardinalidade igual a $|V|/2$ (JUNGNICKEL, 2008).

3.1.1 Propriedades

Antes de continuar para os algoritmos e soluções, é necessário entender algumas propriedades importantes

3.1.1.1 Emparelhamento máximo vs maximal

Um emparelhamento máximo é aquele que contém o maior número possível de arestas, enquanto um emparelhamento maximal é aquele que não pode ser aumentado adicionando mais arestas, mas não necessariamente é o maior possível (MANBER, 1989).

3.1.1.2 Caminhos aumentantes

Um caminho aumentante P em um grafo G é chamado de caminho aumentante se P tem tamanho ímpar, começa e termina em vértices livres (não cobertos por nenhuma aresta do emparelhamento atual) e alterna entre arestas que pertencem a M e arestas que não pertencem a M. A existência de um caminho aumentante indica que o emparelhamento atual não é máximo (SCHRIJVER, 2004)

3.1.1.3 Teorema de Kőnig

O teorema de Kőnig estabelece uma relação fundamental entre emparelhamentos e coberturas em grafos bipartidos. Ele afirma que, em qualquer grafo bipartido, o tamanho do maior emparelhamento é igual ao tamanho da menor cobertura de vértices (JUNGNICKEL, 2008).

A cobertura de vértices é um conjunto de vértices tal que cada aresta do grafo é incidente a pelo menos um vértice desse conjunto. O teorema de Kőnig é uma ferramenta poderosa para resolver problemas de emparelhamento, utilizado nos algoritmos de Hopcroft-Karp e no método Húngaro. A seguir será detalhado um método algorítmico para encontrar a cobertura de vértices mínima a partir de um emparelhamento máximo.

O método é definido pelos seguintes passos (JUNGNICKEL, 2008):

1. Identifica-se o conjunto de vértices livres (não saturados por M) na partição esquerda U.

2. A partir desses vértices, inicia-se uma travessia no grafo (utilizando Busca em Largura ou Profundidade) com regras estritas de alternância: transições do conjunto U para V devem ocorrer exclusivamente através de arestas que **não** pertencem ao emparelhamento ($E \setminus M$), enquanto as transições de V de volta para U devem utilizar obrigatoriamente as arestas do emparelhamento (M).

3. Registra-se em um conjunto Z todos os vértices que foram alcançados durante essa busca, tanto da partição U quanto da partição V.

4. A Cobertura Mínima de Vértices C é construída selecionando-se os vértices de U que **não** foram visitados e os vértices de V que **foram** visitados. Matematicamente, a cobertura é dada por $C = (U \setminus Z) \cup (V \cap Z)$.

Essa extração algorítmica possui complexidade linear em relação ao tamanho do grafo, $O(V + E)$, visto que requer apenas uma travessia simples após o cálculo do emparelhamento. Sua implementação é uma sub-rotina fundamental para métodos exatos de problemas mais complexos, como a atualização de limiares no Método Dual para o problema de atribuição gargalo (CORMEN et al., 2009).

3.1.1.4 Teorema de Hall

O Teorema de Hall, frequentemente referido como Teorema do Casamento de Hall, é outro resultado estrutural para grafos bipartidos. Enquanto o teorema de König determina a cardinalidade do emparelhamento máximo, o teorema de Hall estabelece a condição necessária e suficiente para a existência de um emparelhamento que cubra integralmente um dos subconjuntos da partição (SCHRIJVER, 2004).

Para um grafo bipartido $G = (U \cup V, E)$, o teorema afirma que existe um emparelhamento cobrindo todo o conjunto U se, e somente se, para qualquer subconjunto $S \subseteq U$, o tamanho de sua vizinhança $N(S)$ (o conjunto de vértices em V adjacentes a pelo menos um vértice em S) for maior ou igual ao tamanho de S. Matematicamente, exige-se que $|N(S)| \geq |S|$ para todo $S \subseteq U$ (JUNGNICKEL, 2008). Essa propriedade, conhecida como Condição de Hall, é essencial para provar matematicamente a viabilidade de atribuições e identificar gargalos estruturais que impedem a formação de um emparelhamento perfeito, relacionando-se intimamente com os conceitos de cortes mínimos em redes de fluxo.

3.1.1.5 Teorema de Gallai e Conjunto Independente Máximo

Um conjunto independente em um grafo é um subconjunto de vértices no qual não existem duas extremidades adjacentes, ou seja, nenhuma aresta conecta dois vértices desse conjunto. O problema de encontrar o Conjunto Independente Máximo (*Maximum Independent Set* - MIS) é classicamente conhecido por pertencer à classe NP-Difícil para grafos gerais (GAREY; JOHNSON, 1990). No entanto, quando o grafo é restrito à classe dos grafos bipartidos, o problema pode ser resolvido em tempo polinomial graças à sua profunda relação com os emparelhamentos e coberturas.

A base dessa resolução é o Teorema de Gallai, proposto em 1959. O teorema estabelece que, para qualquer grafo $G = (V, E)$ que não possua vértices isolados, a soma do tamanho da Cobertura Mínima de Vértices (MVC) com o tamanho do Conjunto

Independente Máximo (MIS) é sempre igual ao número total de vértices do grafo (SCHRIJVER, 2004).

Matematicamente:

$$|MVC| + |MIS| = |V| \tag{3.1}$$

Pelo Teorema de König, detalhado anteriormente, sabemos que em grafos bipartidos o tamanho da Cobertura Mínima de Vértices é exatamente igual ao tamanho do Emparelhamento Máximo ($|MVC| = |M|$). Consequentemente, o tamanho do Conjunto Independente Máximo pode ser deduzido diretamente do emparelhamento: $|MIS| = |V| - |M|$ (JUNGNICKEL, 2008).

Além da equivalência de cardinalidade, o Teorema de Gallai também fornece a dualidade estrutural: o Conjunto Independente Máximo é exatamente o complemento da Cobertura Mínima de Vértices. Portanto, para construir algoritmicamente o Conjunto Independente Máximo em um grafo bipartido, basta executar o seguinte procedimento:

1. Calcula-se o Emparelhamento Máximo utilizando algoritmos como o de Hopcroft-Karp.
2. Extrai-se a Cobertura Mínima de Vértices C através do método de busca por caminhos alternantes a partir dos vértices livres, definido na seção 3.1.1.3.
3. O Conjunto Independente Máximo I será composto por todos os vértices do grafo que não pertencem à cobertura. Ou seja, $I = V \setminus C$.

3.1.2 Algoritmos exatos

Por ser em um ambiente mais restrito, o problema de emparelhamento de cardinalidade máxima tem diversas soluções em tempo polinomial. A seguir listadas algumas delas:

3.1.2.1 Algoritmo do caminho aumentante

O algoritmo de caminho aumentante se baseia em um lema proposto por Claude Berge, que afirma que um grafo G é máximo se e somente se não existe nenhum caminho aumentante em G. Este algoritmo é uma implementação direta deste lema, onde se busca repetidamente por caminhos aumentantes e se aumenta o emparelhamento até que nenhum caminho aumentante possa ser encontrado (LAWLER, 1976).

Começamos com um emparelhamento vazio $M = \emptyset$. Escolhemos um vértice livre u no lado esquerdo do grafo bipartido e tentamos encontrar um caminho aumentante P que começa em u e termina em um vértice livre no lado direito do grafo, respeitando

a regra da alternância entre arestas em M e arestas fora de M. Se um caminho aumentante for encontrado, atualizamos o emparelhamento M invertendo as arestas ao longo do caminho P. Repetimos esse processo até que não seja possível encontrar mais caminhos aumentantes. O emparelhamento resultante será o emparelhamento de cardinalidade máxima (HALIM; HALIM, 2013).

Em seguida o algoritmo em pseudo-código:

Algoritmo 4: Algoritmo de caminhos aumentantes (MCBM) (HALIM; HALIM, 2013)

```
(1)  Global: match (vetor de emparelhamento), vis (vetor de visitados), AdjList

(2)  Procedure Aug(l)
(3)      if vis[l] = 1 then
(4)          return 0
(5)      end
(6)      vis[l] ← 1
(7)      for each r ∈ AdjList[l] do
(8)          if match[r] = −1 or Aug(match[r]) then
(9)              match[r] ← l
(10)             return 1 [comentário: encontrou 1 emparelhamento]
(11)         end
(12)     end
(13)     return 0 [comentário: nenhum emparelhamento]

(14) Procedure MCBM_Algorithm(n)
(15)     MCBM ← 0
(16)     match ← −1 (initialize for all V)
(17)     for l ← 0 to n − 1 do
(18)         vis ← 0 (reset before each recursion)
(19)         MCBM ← MCBM + Aug(l)
(20)     end
(21)     return MCBM
```

O pseudocódigo é composto de um procedimento recursivo de busca e de um procedimento principal, explicados a seguir com referência às linhas do Algoritmo 4. Os vértices do lado esquerdo são numerados de 0 a $n-1$, e $match[r]$ guarda o parceiro, no lado esquerdo, de cada vértice r do lado direito (-1 se r está livre):

- **Linhas 3–6:** O procedimento Aug(l) é uma busca em profundidade que tenta encontrar um caminho aumentante a partir do vértice esquerdo l. Se l já foi visitado na busca atual, a tentativa falha; caso contrário, l é marcado como visitado, o que impede que a busca entre em ciclo.
- **Linhas 7–8:** Para cada vizinho r de l, há duas formas de usar a aresta $\{l, r\}$: se r está livre, o caminho aumentante termina em r; se r já está emparelhado

com $match[r]$, a busca continua recursivamente a partir de $match[r]$, tentando realocá-lo para outro vértice. Essa recursão percorre o caminho alternante: a aresta $\{l, r\}$ está fora de M, e a aresta $\{r, match[r]\}$ está em M.

- **Linha 9:** Se uma das duas condições é satisfeita, r passa a ser parceiro de l. Como essa atribuição é feita na volta da recursão, em cada nível do caminho, todas as arestas do caminho aumentante trocam de papel, o que corresponde à diferença simétrica entre M e o caminho, e a cardinalidade de M aumenta em um.

- **Linha 13:** Se nenhum vizinho leva a um caminho aumentante, a busca a partir de l falha.

- **Linhas 16–21:** O procedimento principal começa com todos os vértices livres (linha 16) e tenta, uma única vez, encontrar um caminho aumentante a partir de cada vértice esquerdo (linha 17). Antes de cada tentativa, as marcas de visita são reiniciadas (linha 18). Cada busca bem-sucedida soma um ao tamanho do emparelhamento (linha 19), que é retornado ao final.

- **Complexidade:** Cada chamada da linha 19 visita cada vértice no máximo uma vez e examina cada aresta no máximo uma vez, com custo $O(V + E)$. Como há uma chamada por vértice do lado esquerdo, o tempo total é $O(V(V + E))$, ou seja, $O(VE)$ em grafos sem vértices isolados, onde V é o número de vértices e E é o número de arestas do grafo bipartido (HALIM; HALIM, 2013).

3.1.2.2 Redução ao problema de fluxo máximo

O problema de emparelhamento de cardinalidade máxima em grafos bipartidos pode ser eficientemente resolvido através de uma redução ao problema de fluxo máximo em redes. A ideia central é construir uma rede de fluxo a partir do grafo bipartido original, onde cada aresta do grafo bipartido é convertida em uma aresta com capacidade unitária na rede de fluxo. (LAWLER, 1976)

Primeiro, adicionamos um vértice fonte s e um vértice sumidouro t à rede. Conectamos o vértice fonte s a todos os vértices do conjunto esquerdo do grafo bipartido com arestas de capacidade 1. Em seguida, conectamos todos os vértices do conjunto direito do grafo bipartido ao vértice sumidouro t, também com arestas de capacidade 1. As arestas entre os conjuntos esquerdo e direito do grafo bipartido são mantidas na rede de fluxo, cada uma com capacidade 1 (HALIM; HALIM, 2013).

Com isso, temos agora um grafo de fluxo onde o objetivo é encontrar o fluxo máximo do vértice fonte s para o vértice sumidouro t. O valor do fluxo máximo encontrado nesta rede corresponde ao tamanho do emparelhamento máximo no grafo

bipartido original. (LAWLER, 1976). Para encontrar o fluxo máximo, podemos utilizar algoritmos clássicos como o de Edmonds-Karp.

3.1.2.3 Algoritmo de Edmonds-Karp

O algoritmo de **Edmonds-Karp** é o principal método para resolver esse tipo de problema. Ele é uma especialização do método de Ford-Fulkerson descrito na seção 2.4.2.3 que utiliza uma busca em largura (BFS) para encontrar sempre o caminho aumentante mais curto em número de arestas. Tem complexidade de tempo $O(VE^2)$:

Algoritmo 5: Algoritmo de Edmonds-Karp (HALIM; HALIM, 2013)

```
(1)  Global: res (matriz de capacidade residual), mf, f, s, t, p (vetor de pais da
       BFS)

(2)  Procedure BFS(s, t)
(3)      dist ← ∞;   dist[s] ← 0
(4)      Q ← {s};   p ← nil (init vector with -1)
(5)      while Q ≠ ∅ do
(6)          u ← Q.front(); Q.pop()
(7)          if u = t then
(8)              break (stop BFS)
(9)          end
(10)         for each v ∈ V do
(11)             if res_{u,v} > 0 and dist[v] = ∞ then
(12)                 dist[v] ← dist[u] + 1
(13)                 Q.push(v)
(14)                 p[v] ← u
(15)             end
(16)         end
(17)     end

(18) Procedure Augment(v, minEdge)
(19)     if v = s then
(20)         f ← minEdge
(21)         return
(22)     else
(23)         if p[v] ≠ −1 then
(24)             Augment(p[v], min(minEdge, res_{p[v],v}))
(25)             res_{p[v],v} ← res_{p[v],v} − f
(26)             res_{v,p[v]} ← res_{v,p[v]} + f
(27)         end
(28)     end

(29) Procedure EdmondsKarp(s, t)
(30)     mf ← 0
(31)     while true do
(32)         f ← 0
(33)         BFS(s, t); [comentário: preenche o vetor de pais p]
(34)         Augment(t, ∞); [comentário: encontra o menor peso de aresta f
               no caminho]
(35)         if f = 0 then
(36)             break (terminate)
(37)         end
(38)         mf ← mf + f
(39)     end
(40)     return mf
```

O pseudocódigo divide-se em três procedimentos, explicados a seguir com

referência às linhas do Algoritmo 5. A rede residual é representada pela matriz res, em que $res_{u,v}$ é a capacidade residual do arco (u, v):

- **BFS (linhas 3–14):** Busca em largura a partir de s na rede residual. Todas as distâncias começam infinitas, exceto a de s (linha 3), e a fila Q começa só com s, com o vetor de pais vazio (linha 4). Enquanto houver vértices na fila (linha 5), retira-se o primeiro, u; se ele é o sorvedouro t, o caminho já foi encontrado e a busca para (linha 7). Caso contrário, percorrem-se todos os vértices v (linha 10) e consideram-se apenas os arcos com capacidade residual positiva que levam a um vértice ainda não alcançado (linha 11). Cada vértice alcançado pela primeira vez recebe sua distância (linha 12), entra na fila e tem seu pai registrado em p (linha 14). Como a busca é em largura, o caminho registrado até t é o de menor número de arcos.
- **Augment (linhas 19–26):** Percorre o caminho encontrado de t até s, seguindo o vetor de pais, e carrega em minEdge a menor capacidade residual vista até o momento. Ao chegar em s (linha 19), esse mínimo é o gargalo do caminho e é guardado em f. Se um vértice não tem pai (linha 23), t não foi alcançado pela BFS e nada é feito, de modo que f continua 0. A chamada recursiva (linha 24) desce até s antes de qualquer atualização; na volta da recursão, f é subtraído da capacidade residual de cada arco no sentido do caminho (linha 25) e somado à do arco reverso (linha 26), o que permite a uma iteração futura desfazer parte desse fluxo.
- **EdmondsKarp (linhas 30–38):** O fluxo total mf começa em zero (linha 30). A cada iteração do laço (linha 31), f é zerado (linha 32), a BFS procura o caminho aumentante mais curto (linha 33) e o Augment envia por ele o fluxo do gargalo (linha 34). Quando a BFS não alcança t, nenhum fluxo é enviado ($f = 0$, linha 35) e o fluxo acumulado mf é máximo; caso contrário, f é somado a mf (linha 38).
- **Complexidade:** Como a BFS escolhe sempre o caminho aumentante mais curto, a distância de s a cada vértice na rede residual nunca diminui. A cada aumento, pelo menos um arco do caminho é saturado (o arco gargalo), e um mesmo arco só pode voltar a ser gargalo depois que a distância até ele aumenta em pelo menos 2. Como a distância é limitada por V, cada arco é gargalo $O(V)$ vezes, e o número total de aumentos é $O(VE)$. Com listas de adjacência, cada BFS custa $O(E)$, o que leva ao total de $O(VE^2)$. Na versão com matriz do Algoritmo 5, o laço da linha 10 percorre todos os vértices, e cada BFS custa $O(V^2)$, o que dá $O(V^3E)$ (HALIM; HALIM, 2013). Na rede obtida pela redução do emparelhamento bipartido, as capacidades são unitárias e o fluxo máximo é no máximo $V/2$; como

cada aumento envia uma unidade de fluxo, há no máximo $V/2$ aumentos, e o tempo cai para $O(VE)$ com listas de adjacência.

3.1.2.4 Algoritmo de Dinic

Outra opção para resolver o problema de fluxo máximo é o algoritmo de Dinic, que é mais eficiente em muitos casos práticos, especialmente em grafos densos. O algoritmo de Dinic utiliza uma combinação de buscas em largura (BFS) para construir níveis na rede residual e buscas em profundidade (DFS) para encontrar caminhos aumentantes dentro desses níveis. O tempo de execução do algoritmo de Dinic é $O(V^2E)$ em redes gerais e cai para $O(E\sqrt{V})$ em redes de capacidade unitária, como a obtida na redução do emparelhamento bipartido, o que o torna uma escolha preferida para muitos problemas de fluxo máximo (HALIM; HALIM, 2013).

Este trabalho não tem uma implementação do algoritmo de Dinic. Ao invés disso, será apresentado o algoritmo de Hopcroft-Karp, que é uma variação especializada do algoritmo de Dinic para o problema de emparelhamento em grafos bipartidos, oferecendo uma solução mais eficiente para este caso específico.

3.1.2.5 Algoritmo de Hopcroft-Karp

O algoritmo de Hopcroft-Karp é um algoritmo eficiente para encontrar o emparelhamento máximo em grafos bipartidos. Ele melhora o desempenho do algoritmo de caminhos aumentantes ao encontrar múltiplos caminhos aumentantes em cada iteração, em vez de apenas um. O tempo de execução do algoritmo de Hopcroft-Karp é $O(E\sqrt{V})$, tornando-o significativamente mais rápido do que o algoritmo de caminhos aumentantes simples, especialmente em grafos grandes (HALIM; HALIM, 2013).

O algoritmo consiste em duas fases principais: a fase de construção de níveis e a fase de busca de caminhos aumentantes. Na fase de construção de níveis, uma busca em largura (BFS) é realizada a partir dos vértices livres no lado esquerdo do grafo bipartido para construir uma camada de níveis que ajuda a identificar os caminhos aumentantes mais curtos. Na fase de busca de caminhos aumentantes, uma busca em profundidade (DFS) é realizada para encontrar todos os caminhos aumentantes possíveis dentro da camada de níveis construída na fase anterior. Esses caminhos são então usados para aumentar o emparelhamento (HALIM; HALIM, 2013).

A seguir, o pseudocódigo do algoritmo de Hopcroft-Karp, dividido em duas partes: o Algoritmo 6 traz o laço principal e a construção do grafo em camadas, e o Algoritmo 7, a busca dos caminhos aumentantes disjuntos.

Algoritmo 6: Algoritmo de Hopcroft-Karp: laço principal e grafo em camadas (BURKARD; DELL'AMICO; MARTELLO, 2009)

(1) **let** $G = (U, V; E)$ be a bipartite graph with initial (possibly empty) matching M;
(2) **let** U_0 contain all unmatched vertices of U;
(3) **let** V_0 contain all unmatched vertices of V;
(4) **while** $U_0 \neq \emptyset$ **do**
(5) | Construct_layered_graph;
(6) | Find_set_ΔM;
(7) | $M := M \ominus \Delta M$;
(8) | update sets U_0, V_0 of unmatched vertices
(9) **end**

(10) **Procedure** *Construct_layered_graph*
(11) | $L_0 := U_0, k^* := k := 0$;
(12) | **while** $L_k \neq \emptyset$ **do**
(13) | | **for** *each* $i \in L_k$ **do**
(14) | | | $N(i) := \{j : [i, j] \in E \setminus M, j \notin L_1 \cup L_3 \cup \cdots \cup L_{k-1}\}$;
(15) | | **end**
(16) | | $L_{k+1} := \bigcup_{i \in L_k} N(i)$;
(17) | | **if** $L_{k+1} = \emptyset$ **then**
(18) | | | **stop [comentário:** M **é um emparelhamento máximo]**
(19) | | **end**
(20) | | **if** $L_{k+1} \cap V_0 \neq \emptyset$ **then**
(21) | | | $k^* := k + 1$;
(22) | | | $L_{k+2} := \emptyset$
(23) | | **else**
(24) | | | $L_{k+2} := \{\bar{i} : [\bar{i}, j] \in M, j \in L_{k+1}\}$;
(25) | | **end**
(26) | | $k := k + 2$
(27) | **end**

Algoritmo 7: Algoritmo de Hopcroft-Karp: busca dos caminhos aumentantes disjuntos (BURKARD; DELL'AMICO; MARTELLO, 2009)

```
(1)  Procedure Find_set_ΔM
(2)      [comentário: encontra um conjunto maximal ΔM de caminhos
          aumentantes disjuntos]
(3)      delete all vertices in L_{k*} \ V_0;
(4)      mark all remaining vertices as unscanned;
(5)      k := 1; [comentário: contador de caminhos]
(6)      while L_0 ≠ ∅ do
(7)          choose x_0 := i ∈ L_0 and delete it in L_0;
(8)          ℓ := 0;
(9)          while ℓ ≥ 0 do
(10)             while x_ℓ has an unscanned neighbor in L_{ℓ+1} do
(11)                 choose unscanned neighbor x_{ℓ+1};
(12)                 mark x_{ℓ+1} as scanned;
(13)                 ℓ := ℓ + 1;
(14)                 if ℓ = k* then
(15)                     P_k := (x_0, x_1, ..., x_{k*});
(16)                     k := k + 1
(17)                 end
(18)             end
(19)             if ℓ < k* then
(20)                 ℓ := ℓ − 1
(21)             else
(22)                 ℓ := −1
(23)             end
(24)         end
(25)     end
(26)     return ΔM := (P_1, P_2, ..., P_{k−1})
```

O pseudocódigo é organizado em um laço principal e dois procedimentos, explicados a seguir com referência às linhas dos Algoritmos 6 e 7:

- **Laço principal (Algoritmo 6, linhas 2–8):** U_0 e V_0 guardam os vértices livres de cada lado (linhas 2 e 3). Cada iteração do laço da linha 4 é uma *fase*: constrói-se o grafo em camadas (linha 5), encontra-se um conjunto maximal ΔM de caminhos aumentantes mínimos e disjuntos (linha 6) e o emparelhamento é atualizado pela diferença simétrica $M \ominus \Delta M$ (linha 7). Ao longo de cada caminho, as arestas emparelhadas e as não emparelhadas trocam de papel, de modo que $|M|$ aumenta em $|\Delta M|$.

- **Construção do grafo em camadas (Algoritmo 6):** É uma busca em largura que parte de todos os vértices livres de U ao mesmo tempo, que formam a camada L_0 (linha 11). As camadas de índice ímpar contêm vértices de V, alcançados por arestas *não* emparelhadas (linha 14); a restrição $j \notin L_1 \cup \cdots \cup L_{k-1}$ impede que

um vértice seja revisitado. Se nenhum vértice novo é alcançado (linha 17), não há caminho aumentante e, pelo lema de Berge, M é máximo. Se a nova camada contém um vértice livre de V (linha 20), foi encontrado o comprimento k^* dos caminhos aumentantes mais curtos, e a construção termina. Caso contrário, a camada seguinte é formada pelos parceiros, em M, dos vértices recém-alcançados (linha 24): o caminho segue obrigatoriamente pela aresta emparelhada, o que garante a alternância.

- **Busca dos caminhos disjuntos (Algoritmo 7):** Primeiro, descartam-se os vértices da última camada que não são livres (linha 3), pois nenhum caminho aumentante termina neles. Em seguida, uma busca em profundidade parte de cada vértice livre $x_0 \in L_0$ (linha 7) e avança sempre para um vizinho ainda não visitado da camada seguinte (linha 10). Ao atingir a camada k^*, o caminho $P_k = (x_0, \ldots, x_{k^*})$ é registrado (linha 14) e a busca a partir dessa raiz termina; se a busca fica sem vizinhos antes disso, recua uma camada (linha 19). Como cada vértice é visitado no máximo uma vez por fase, os caminhos encontrados são disjuntos em vértices, e o conjunto ΔM é maximal.

- **Complexidade:** Cada fase examina cada aresta um número constante de vezes, com custo $O(E)$. Após $\sqrt{V}$ fases, todo caminho aumentante restante tem mais de $\sqrt{V}$ arestas; como os caminhos aumentantes de $M \ominus M^*$ são disjuntos em vértices, restam menos de $\sqrt{V}$ deles, e cada fase seguinte elimina ao menos um. O número de fases é, portanto, $O(\sqrt{V})$, o que leva à complexidade total $O(E\sqrt{V})$ (BURKARD; DELL'AMICO; MARTELLO, 2009).

3.1.3 Algoritmos aproximados

Por mais que o algoritmo de Hopcroft-Karp descrito nos Algoritmos 6 e 7 seja considerado relativamente eficiente, com complexidade $O(E\sqrt{V})$, seu custo ainda cresce rapidamente em instâncias muito grandes do problema de emparelhamento de cardinalidade máxima. Por isso, foram propostos métodos que abrem mão da garantia de otimalidade para resolver o problema com complexidade linear $O(E)$. Os dois métodos desta seção garantem um emparelhamento com pelo menos metade da cardinalidade máxima e, por isso, são classificados como algoritmos aproximados (Seção 2.6).

3.1.3.1 Método guloso simples

Um método que itera sobre as arestas do grafo em ordem arbitrária. Se a aresta $\{u, v\}$ conecta dois vértices livres, adicione essa aresta ao emparelhamento e remova os vértices u e v. Tem complexidade de tempo linear $O(E)$ e garante um emparelhamento maximal, que conforme visto na seção 3.1.1.1, é um emparelhamento

ao qual não é possível adicionar mais arestas, mas que não é necessariamente o de maior cardinalidade. A solução também garante um emparelhamento de pelo menos $1/2$ (metade) da cardinalidade máxima (VAZIRANI, 2001).

Algoritmo 8: Método guloso simples (VAZIRANI, 2001)

Input: bipartite graph $G = (U, V; E)$
Output: maximal matching M

```
(1) M := ∅
(2) for each edge {u, v} ∈ E, in arbitrary order do
(3)     if u and v are both unmatched then
(4)         M := M ∪ {{u, v}}
(5)         mark u and v as matched
(6)     end
(7) end
(8) return M
```

O Algoritmo 8 é explicado a seguir, linha a linha:

- **Linha 1:** O emparelhamento começa vazio, com todos os vértices livres.
- **Linhas 2–3:** Cada aresta é examinada uma única vez, em ordem arbitrária, verificando-se se seus dois extremos ainda estão livres.
- **Linhas 4–5:** Se estão, a aresta entra no emparelhamento e seus extremos são marcados como emparelhados, o que exclui todas as demais arestas incidentes a eles. Uma aresta rejeitada nunca é reconsiderada.
- **Linha 8:** Ao final, nenhuma aresta de G tem os dois extremos livres, pois ela teria sido aceita quando foi examinada. O emparelhamento retornado é, portanto, maximal.
- **Complexidade:** Com um vetor de marcas por vértice, o teste da linha 3 e as atualizações das linhas 4–5 custam $O(1)$. Como o laço examina cada aresta uma vez, o tempo total é $O(V + E)$, ou seja, linear no tamanho do grafo.
- **Garantia de aproximação:** Seja M o emparelhamento maximal retornado e M^* um emparelhamento máximo. Toda aresta de M^* tem pelo menos um extremo coberto por M, pois, caso contrário, ela poderia ser acrescentada a M, que não seria maximal. Cada aresta de M cobre 2 vértices, e cada vértice toca no máximo uma aresta de M^*, já que M^* é um emparelhamento. Logo, as arestas de M cobrem no máximo $2|M|$ arestas de M^*, e como todas as arestas de M^* são cobertas, $|M^*| \leq 2|M|$, ou seja, $|M| \geq \frac{1}{2}|M^*|$ (VAZIRANI, 2001).

3.1.3.2 Algoritmo de Karp-Sipser

Algoritmo descrito por Karp e Sipser (1981) para grafos esparsos que toma decisões com base na topologia do grafo ao invés de decisões aleatórias, tornando essas decisões mais seguras.

O algoritmo funciona reduzindo dinamicamente o grafo $G = (V, E)$ através da aplicação repetida de duas regras, nesta ordem de prioridade:

1. **Redução de folhas de grau 1:** Se existir algum vértice v no grafo atual com **grau 1** (apenas um vizinho u), a aresta $\{u, v\}$ é selecionada imediatamente para o emparelhamento. Isso é seguro pois, se u é o único vizinho de v, ele também é a única opção para emparelhar v. Se isso não for feito, v ficará isolado.

2. **Escolha Aleatória:** Se **não** existirem vértices de grau 1 (ou seja, todos os vértices têm grau ≥ 2), o algoritmo escolhe uma aresta $\{u, v\}$ aleatoriamente, adiciona ao emparelhamento e remove u e v do grafo.

   Em grafos esparsos, isso tende a quebrar ciclos e formar novas folhas (vértices de grau 1), permitindo que o algoritmo volte a usar a regra 1. Porém, em grafos densos, essa escolha aleatória pode comprometer a precisão da solução.

Para grafos esparsos, a regra 2 é raramente aplicada, pois a remoção de vértices na regra 1 frequentemente cria novas folhas, provocando um efeito cascata, fazendo com que a maior parte do grafo seja resolvida de maneira ótima. Com estruturas de dados adequadas, a complexidade do algoritmo é linear ($O(E)$). A implementação avaliada neste trabalho, por simplicidade, localiza os vértices de grau 1 por varredura, o que eleva sua complexidade a $O(V \cdot E)$.

Algoritmo 9: Algoritmo de Karp-Sipser (KARP; SIPSER, 1981)

Input: bipartite graph $G = (U, V; E)$
Output: maximal matching M

(1) $M := \emptyset$
(2) **while** $E \neq \emptyset$ **do**
(3) | **if** *there is a vertex* v *with* $\deg(v) = 1$ **then**
(4) | | let u be the only neighbor of v
(5) | | **[comentário: seguro: algum emparelhamento máximo contém** $\{u, v\}$**]**
(6) | **else**
(7) | | choose an arbitrary edge $\{u, v\} \in E$
(8) | | **[comentário: único passo que pode perder a otimalidade]**
(9) | **end**
(10) | $M := M \cup \{\{u, v\}\}$
(11) | remove u, v and all their incident edges from G
(12) **end**
(13) **return** M

O Algoritmo 9 é explicado a seguir, linha a linha:

- **Linhas 1–2:** O emparelhamento começa vazio, e o laço principal se repete enquanto o grafo ainda tiver arestas.
- **Linhas 3–4:** Regra 1. Se há um vértice v de grau 1, a aresta escolhida é a que o liga ao seu único vizinho u. Essa escolha nunca piora a solução: dado um emparelhamento máximo qualquer, se ele não contém $\{u, v\}$, então v está livre nele, e trocar a aresta que cobre u (se houver) por $\{u, v\}$ produz outro emparelhamento máximo que a contém.
- **Linha 7:** Regra 2. Se não há vértices de grau 1, escolhe-se uma aresta arbitrária. É o único passo em que a otimalidade pode ser perdida.
- **Linhas 10–11:** A aresta escolhida entra no emparelhamento, e seus dois extremos são removidos do grafo com todas as arestas incidentes. A remoção reduz o grau dos vizinhos de u e v, o que pode criar novos vértices de grau 1 e reativar a regra 1 na próxima iteração.
- **Linha 13:** O laço só termina quando não restam arestas no grafo (linha 2). Como só são removidas arestas incidentes a vértices emparelhados, nenhuma aresta de G tem os dois extremos livres ao final. O emparelhamento retornado é, portanto, maximal e, pelo mesmo argumento do método guloso simples, tem pelo menos metade da cardinalidade máxima.
- **Complexidade:** Cada iteração emparelha dois vértices, então há no máximo $V/2$ iterações, e cada aresta é removida uma única vez. Com uma estrutura que

mantém os vértices de grau 1 em uma lista atualizada a cada remoção, encontrar o vértice da linha 3 custa $O(1)$, e o total é $O(V + E)$. Na implementação avaliada, a linha 3 varre todos os vértices a cada iteração, com custo $O(V + E)$ por iteração, o que resulta em $O(V(V + E))$, ou seja, $O(V \cdot E)$ em grafos sem vértices isolados.

3.2 PROBLEMA DE ATRIBUIÇÃO (ASSIGNMENT PROBLEM)

O problema de atribuição, também conhecido como problema do emparelhamento ponderado, consiste em encontrar uma combinação ótima de atribuições entre dois conjuntos disjuntos, minimizando o custo total associado a essas atribuições. Exemplo: Considere N trabalhadores e N tarefas, onde cada trabalhador pode ser designado a exatamente uma tarefa, e cada tarefa deve ser atribuída a exatamente um trabalhador. O custo de atribuir o trabalhador i à tarefa j é representado por uma matriz de custos $W = [w_{ij}]$ (LAWLER, 1976). O objetivo é encontrar um conjunto de atribuições que minimize o custo total.

3.2.1 Algoritmos exatos

A seguir serão apresentados os principais algoritmos conhecidos com soluções exatas para o problema de atribuição.

3.2.1.1 Método Húngaro

O método Húngaro é um algoritmo eficiente para resolver o problema de atribuição em tempo polinomial. Ele foi desenvolvido por Harold Kuhn em 1955 e é baseado no trabalho de Dénes Kőnig e Jenő Egerváry sobre emparelhamentos em grafos bipartidos (JUNGNICKEL, 2008).

O método se baseia em manter um conjunto de potenciais para os vértices dos dois conjuntos do grafo bipartido, sendo u_i o potencial do vértice i no conjunto esquerdo e v_j o potencial do vértice j no conjunto direito. A regra que o algoritmo mantém é que para cada aresta $\{i, j\}$, a soma dos potenciais deve ser maior ou igual ao custo da aresta, ou seja, $u_i + v_j \geq w_{ij}$.

Cria-se um subgrafo de igualdade $H_{u,v}$ contendo apenas as arestas onde a soma dos potenciais é igual ao custo da aresta, ou seja, $u_i + v_j = w_{ij}$. O algoritmo então tenta encontrar um emparelhamento máximo neste subgrafo de igualdade. Se o emparelhamento encontrado cobre todos os vértices do conjunto esquerdo, então ele é ótimo para o problema de atribuição original. Caso contrário, o algoritmo ajusta os potenciais para criar novas arestas de igualdade e repete o processo até encontrar um emparelhamento máximo que cubra todos os vértices do conjunto esquerdo (JUNGNICKEL, 2008).

Tabela 1 – Algoritmos para emparelhamento de cardinalidade máxima em grafos bipartidos

Algoritmo	**Complexidade**	**Tipo**	**Avaliado**	**Descrição**
Caminhos aumentantes	$O(VE)$	Exato	Sim	Busca iterativa (DFS/BFS) por caminhos alternantes entre vértices livres, invertendo arestas para aumentar a cardinalidade.
Edmonds-Karp	$O(VE^2)$	Exato	Sim	Implementação de Ford-Fulkerson que usa BFS para encontrar sempre o caminho aumentante mais curto (em nº de arestas) na rede residual.
Dinic	$O(V^2E)$	Exato	Não	Algoritmo de fluxo que constrói grafos de níveis (BFS) e satura múltiplos caminhos simultaneamente via fluxo de bloqueio (DFS). Em redes de capacidade unitária, como a do emparelhamento bipartido, cai para $O(E\sqrt{V})$.
Hopcroft-Karp	$O(E\sqrt{V})$	Exato	Sim	Busca múltiplos caminhos aumentantes disjuntos por fase usando BFS e DFS.
Guloso simples	$O(E)$	Aproximado	Sim	Itera sobre arestas arbitrariamente,

Em seguida o pseudo-código do método Húngaro:

Algoritmo 10: Algoritmo Húngaro (JUNGNICKEL, 2008)

```
(1)  Procedure HUNGARIAN(n, w; mate)
(2)      for v ∈ V do mate(v) ← 0
(3)      for i = 1 to n do u_i ← max{w_ij : j = 1, ..., n}; v_i ← 0
(4)      nrex ← n
(5)      while nrex ≠ 0 do
(6)          for i = 1 to n do m(i) ← false; p(i) ← 0; δ_i ← ∞
(7)          aug ← false; Q ← {i ∈ S : mate(i) = 0}
(8)          repeat
(9)              remove an arbitrary vertex i from Q; m(i) ← true; j ← 1
(10)             while aug = false and j ≤ n do
(11)                 if mate(i) ≠ j' then
(12)                     if u_i + v_j − w_ij < δ_j then
(13)                         δ_j ← u_i + v_j − w_ij; p(j) ← i
(14)                         if δ_j = 0 then
(15)                             if mate(j') = 0 then
(16)                                 AUGMENT(mate, p, j'; mate)
(17)                                 aug ← true; nrex ← nrex − 1
(18)                             else
(19)                                 Q ← Q ∪ {mate(j')}
(20)                             end
(21)                         end
(22)                     end
(23)                 end
(24)                 j ← j + 1
(25)             end
(26)             if aug = false and Q = ∅ then
(27)                 J ← {i ∈ S : m(i) = true}; K ← {j' ∈ T : δ_j = 0}
(28)                 δ ← min{δ_j : j' ∈ T \ K}
(29)                 for i ∈ J do u_i ← u_i − δ
(30)                 for j' ∈ K do v_j ← v_j + δ
(31)                 for j' ∈ T \ K do δ_j ← δ_j − δ
(32)                 X ← {j' ∈ T \ K : δ_j = 0}
(33)                 if mate(j') ≠ 0 for all j' ∈ X then
(34)                     for j' ∈ X do Q ← Q ∪ {mate(j')}
(35)                 else
(36)                     choose j' ∈ X with mate(j') = 0
(37)                     AUGMENT(mate, p, j'; mate)
(38)                     aug ← true; nrex ← nrex − 1
(39)                 end
(40)             end
(41)         until aug = true
(42)     end
```

Algoritmo 11: Método AUGMENT (JUNGNICKEL, 2008)

```
(1) Procedure AUGMENT(mate, p, j'; mate)
(2)     repeat
(3)         i ← p(j); mate(j') ← i; next ← mate(i); mate(i) ← j'
(4)         if next ≠ 0 then
(5)             j' ← next
(6)         end
(7)     until next = 0
```

Na versão de Jungnickel (2008), o algoritmo *maximiza* o peso do emparelhamento; para minimizar custos, basta usar $w_{ij} = -c_{ij}$. As linhas da matriz formam o conjunto $S = \{1, \ldots, n\}$ e as colunas, o conjunto $T = \{1', \ldots, n'\}$, em que j' denota a coluna j como vértice do grafo. A seguir, cada parte do Algoritmo 10 é explicada com referência às suas linhas:

- **Inicialização (linhas 2–4):** Nenhum vértice começa emparelhado. Cada potencial de linha u_i recebe o maior peso da linha i, e os potenciais de coluna v_j recebem zero, o que satisfaz $u_i + v_j \geq w_{ij}$ para toda aresta. O contador $nrex$ guarda o número de linhas ainda livres.

- **Fases (linha 5):** Cada iteração do laço externo aumenta o emparelhamento em uma aresta, até que todas as linhas estejam emparelhadas. No início da fase, as marcas são reiniciadas (linha 6): $m(i)$ indica se a linha i já entrou na árvore de busca, $p(j)$ guarda a linha pela qual a coluna j foi alcançada e δ_j guarda a menor folga $u_i + v_j - w_{ij}$ da coluna j em relação às linhas da árvore. A fila Q começa com as linhas livres, que são as raízes da busca (linha 7).

- **Busca no subgrafo de igualdade:** Uma linha i é retirada da fila e entra na árvore (linha 9). Para cada coluna que não é a sua parceira atual (linha 11), calcula-se a folga da aresta $\{i, j\}$ e, se ela for menor que δ_j, a coluna passa a ter i como predecessora (linha 12). Folga nula significa que a aresta pertence ao subgrafo de igualdade $H_{u,v}$ (linha 14). Nesse caso, se a coluna está livre, foi encontrado um caminho aumentante, e o procedimento AUGMENT o aplica (linha 16); senão, a parceira da coluna entra na fila (linha 19), e a árvore cresce pela aresta emparelhada.

- **Ajuste dos potenciais (linhas 26–36):** Se a fila esvazia sem que um caminho aumentante seja encontrado, as linhas e colunas da árvore formam os conjuntos J e K. Calcula-se δ, a menor folga entre as colunas fora da árvore (linha 28). Diminuir u_i em δ nas linhas da árvore e aumentar v_j em δ nas colunas da árvore preserva as arestas da árvore no subgrafo de igualdade e mantém a viabilidade dos potenciais. Ao mesmo tempo, a folga das colunas de fora diminui em δ

(linha 31), o que coloca ao menos uma nova aresta no subgrafo de igualdade (o conjunto X, linha 32). Se todas as colunas de X estão emparelhadas, suas parceiras entram na fila e a busca continua; senão, uma coluna livre de X completa um caminho aumentante (linha 36).

- **Aumento (Algoritmo 11):** A partir da coluna livre encontrada, o procedimento AUGMENT segue os predecessores p de volta até a raiz, invertendo o emparelhamento ao longo do caminho:
  - Linha 3: $i = p(j)$ é a linha pela qual a coluna j' foi alcançada na busca. A coluna j' passa a ser parceira de i, e a antiga parceira de i é guardada em $next$ antes de ser substituída. Essa troca é a inversão de uma aresta emparelhada e uma não emparelhada do caminho aumentante.
  - Linhas 4–5: se i tinha parceira ($next \neq 0$), essa coluna acabou de ficar livre e o processo continua a partir dela, um passo mais perto da raiz.
  - Linha 7: o laço termina quando i não tinha parceira, ou seja, quando se chega à raiz da árvore, que era uma linha livre. Ao final, o emparelhamento tem uma aresta a mais.
  - Como o caminho tem no máximo n arestas emparelhadas, o procedimento custa $O(n)$.
- **Complexidade:** Em cada fase, cada linha entra na árvore no máximo uma vez, e sua varredura custa $O(n)$. Cada ajuste de potenciais custa $O(n)$ e acrescenta ao menos uma coluna à árvore, de modo que há no máximo n ajustes por fase. Cada fase custa, portanto, $O(n^2)$, e as n fases resultam em $O(n^3)$.

3.2.1.2 Jonker-Volgenant

O algoritmo de Jonker-Volgenant é uma evolução otimizada para a resolução do problema de atribuição linear, proposto por Roy Jonker e Anton Volgenant em 1987. Ele foi desenvolvido para superar o desempenho prático do método Húngaro, especialmente em grafos densos, combinando a teoria de dualidade com estratégias de inicialização e busca mais eficientes (JONKER; VOLGENANT, 1987).

Assim como o método Húngaro, este algoritmo baseia-se na manutenção de potenciais duais (u_i e v_j) e respeita as condições de folga complementar. No entanto, sua principal distinção conceitual é o uso da estratégia de Caminho Aumentante Mais Curto (Shortest Augmenting Path). Ao invés de construir emparelhamentos máximos em fases distintas, o algoritmo foca em encontrar o caminho de custo mínimo que conecta uma linha não atribuída a uma coluna livre.

O processo inicia com heurísticas avançadas (conhecidas como redução de colunas e transferência de redução) para resolver rapidamente as atribuições triviais. Para os vértices restantes, o algoritmo realiza uma busca similar ao algoritmo de Dijkstra: ele explora o grafo para encontrar o caminho aumentante mais curto, atualizando os potenciais duais simultaneamente durante a busca. Isso garante que, a cada iteração, o emparelhamento seja aumentado pelo menor custo possível até que a solução ótima completa seja atingida.

A implementação completa deste algoritmo é extensa; o Algoritmo 12 apresenta suas quatro fases em alto nível. Sua eficiência prática o torna uma escolha preferida para muitos problemas de atribuição em aplicações reais, especialmente quando comparado ao método Húngaro tradicional (JUNGNICKEL, 2008).

Algoritmo 12: Algoritmo de Jonker-Volgenant (LAPJV), em alto nível (JONKER; VOLGENANT, 1987)

Input: cost matrix $C = (c_{ij})_{n \times n}$
Output: optimal assignment x (row i is assigned to column x_i) and column duals v

(1) **[comentário: fase 1: redução de colunas]**
(2) **for** $j := n$ **to** 1 **do**
(3) | $i^* := \arg\min_i c_{ij}$; $v_j := c_{i^* j}$
(4) | **if** *row* i^* *is unassigned* **then** assign i^* to j
(5) **end**
(6) **[comentário: fase 2: transferência de redução]**
(7) **for** *each assigned row* i *that is the minimum of a single column* $j_1 = x_i$ **do**
(8) | $v_{j_1} := v_{j_1} - \min_{j \neq j_1}(c_{ij} - v_j)$
(9) **end**
(10) **[comentário: fase 3: redução de linhas aumentante (aplicada duas vezes)]**
(11) **for** *each free row* i **do**
(12) | let j_1, j_2 be the columns with the smallest and second smallest values $u_1 \leq u_2$ of $c_{ij} - v_j$
(13) | **if** $u_1 < u_2$ **then**
(14) | | $v_{j_1} := v_{j_1} - (u_2 - u_1)$
(15) | **else**
(16) | | **if** j_1 *is assigned* **then** $j_1 := j_2$
(17) | **end**
(18) | assign i to j_1; the row previously assigned to j_1, if any, becomes free
(19) **end**
(20) **[comentário: fase 4: aumento por caminhos mínimos]**
(21) **for** *each remaining free row* i **do**
(22) | find a shortest augmenting path from i to a free column with a Dijkstra-like search on the reduced costs $c_{kj} - v_j$
(23) | $v_j := v_j + d_j - \mu$ for every scanned column j, where d_j is its distance and μ the length of the path
(24) | augment the assignment along the path
(25) **end**
(26) **return** x, v

Diferentemente do Algoritmo 10, o LAPJV minimiza diretamente o custo e mantém explícitos apenas os potenciais de coluna v_j; o potencial de cada linha fica implícito como o menor custo reduzido $\min_j(c_{ij} - v_j)$ da linha. Uma aresta $\{i, j\}$ é *justa* quando seu custo reduzido $c_{ij} - v_j$ é o mínimo da linha i. As fases 1 a 3 são heurísticas de inicialização, que atribuem rapidamente boa parte das linhas; apenas a fase 4 garante a otimalidade (JONKER; VOLGENANT, 1987):

- **Fase 1, redução de colunas (linhas 1–4):** As colunas são percorridas da última para a primeira (linha 2). Cada potencial v_j recebe o menor custo da coluna, e i^* é a linha onde ele ocorre (linha 3). Se essa linha ainda está livre, ela é

atribuída à coluna (linha 4); se já foi atribuída a outra coluna, a coluna j fica livre por enquanto.

- **Fase 2, transferência de redução (linhas 6–8):** Para cada linha atribuída na fase 1 cuja coluna j_1 só tem essa linha como mínimo (linha 7), o potencial v_{j_1} diminui pela menor diferença entre o custo reduzido da linha nas outras colunas e o de j_1 (linha 8). Isso transfere a folga da coluna para a linha: a atribuição atual continua justa, mas a coluna j_1 fica menos atrativa para as linhas livres, o que favorece a fase 3.

- **Fase 3, redução aumentante de linhas (linhas 10–18):** Cada linha livre i (linha 11) busca as duas colunas de menor custo reduzido, j_1 e j_2 (linha 12).

  – Se o mínimo é estrito (linha 13), v_{j_1} diminui em $u_2 - u_1$ (linha 14), o que iguala o custo reduzido de j_1 ao de j_2 para a linha i e mantém a aresta $\{i, j_1\}$ justa.

  – Em caso de empate e se j_1 já está atribuída, a linha escolhe j_2 (linha 16), para não desalojar outra linha sem necessidade.

  – A linha i é atribuída a j_1; a linha que ocupava j_1, se houver, volta a ser livre (linha 18) e será processada novamente, agora diante de um v_{j_1} menor.

  A fase é aplicada duas vezes, como recomendado pelos autores, pois as linhas desalojadas podem disputar as colunas indefinidamente.

- **Fase 4, aumento por caminhos mínimos (linhas 20–24):** Para cada linha ainda livre (linha 21), uma busca semelhante ao algoritmo de Dijkstra sobre os custos reduzidos encontra o caminho aumentante mais barato até uma coluna livre (linha 22). Os custos reduzidos são não negativos, o que permite usar Dijkstra. Os potenciais das colunas examinadas são ajustados pela sua distância d_j em relação ao comprimento μ do caminho (linha 23), o que torna justas as arestas do caminho e mantém os demais custos reduzidos não negativos. Por fim, a atribuição é invertida ao longo do caminho, como no procedimento AUGMENT (linha 24).

- **Retorno (linha 26):** Devolve a atribuição x e os potenciais de coluna v, que comprovam a otimalidade pela folga complementar.

- **Complexidade:** As fases 1 e 2 percorrem a matriz uma vez, em $O(n^2)$. A fase 3 tem custo limitado na prática por ser aplicada apenas duas vezes. Na fase 4, há no máximo n linhas livres, e cada busca de caminho mínimo em uma matriz densa custa $O(n^2)$, o que leva ao limite total de $O(n^3)$, o mesmo do método Húngaro. A vantagem do LAPJV é prática: a inicialização deixa poucas linhas para a fase 4, que é a mais cara.

3.2.1.3 Redução para problema de fluxo de custo mínimo

Similarmente ao que foi feito para o problema de emparelhamento de cardinalidade máxima, o problema de atribuição pode ser resolvido através de uma redução ao problema de fluxo de custo mínimo. Assim como para o problema anterior, essa redução é feita criando dois novos vértices: um vértice fonte s e um vértice sorvedouro t. Em seguida, são adicionadas arestas do vértice fonte s para cada vértice do conjunto esquerdo do grafo bipartido, com capacidade 1 e custo 0. Também são adicionadas arestas de cada vértice do conjunto direito para o vértice sorvedouro t, também com capacidade 1 e custo 0. Finalmente, as arestas do grafo bipartido original são adicionadas com capacidade 1 e custo igual ao custo de atribuição correspondente.

Para resolver problemas de fluxo podemos utilizar algoritmos como o de caminhos de custo mínimo sucessivos (Successive Shortest Paths) ou o algoritmo de cancelamento de ciclos (Cycle canceling) (SCHRIJVER, 2004).

O primeiro algoritmo, o de caminhos de custo mínimo sucessivos, foi inicialmente proposto por Jewell, Busacker e Gowen e é uma adaptação do algoritmo de Ford-Fulkerson para incorporar o custo das arestas. Em poucas palavras, ele encontra o caminho de s para t de custo mínimo, envia o máximo fluxo possível ao longo desse caminho, e repete esse processo até que não seja mais possível aumentar o fluxo sem violar as capacidades das arestas (SCHRIJVER, 2004).

Algoritmo 13: Algoritmo de caminhos de custo mínimo sucessivos (JUNGNICKEL, 2008)

```
(1)  Procedure OPTFLOW(G, c, s, t, γ, v; f, sol)
(2)      for e ∈ E do f(e) ← 0
(3)      sol ← true, val ← 0
(4)      while sol = true and val < v do
(5)          construct the auxiliary network N' = (G', c', s, t) with cost function γ'
(6)          if t is not accessible from s in G' then
(7)              sol ← false
(8)          else
(9)              determine a shortest path P from s to t in (G', γ')
(10)             δ ← min{c'(e) : e ∈ P}; δ' ← min(δ, v − val); val ← val + δ'
(11)             augment f along P by δ'
(12)         end
(13)     end
```

O procedimento recebe a rede G com capacidades c, fonte s, sorvedouro t e custos γ, além do valor de fluxo desejado v; devolve o fluxo f e o indicador sol, que diz se esse valor pôde ser atingido. As linhas do Algoritmo 13 funcionam da seguinte forma (JUNGNICKEL, 2008):

- Linhas 2–3: o fluxo começa nulo em todos os arcos, o que já é um fluxo de custo

mínimo para o valor 0; val guarda o valor do fluxo atual.

- Linha 4: cada iteração aumenta o valor do fluxo, até atingir v ou até se descobrir que isso é impossível.
- Linha 5: constrói a rede auxiliar (residual) N'. Para cada arco (x, y) com $f(x, y) < c(x, y)$, há um arco (x, y) com capacidade residual $c(x, y) - f(x, y)$ e custo $\gamma(x, y)$; para cada arco com $f(x, y) > 0$, há um arco reverso (y, x) com capacidade $f(x, y)$ e custo $-\gamma(x, y)$, que representa a possibilidade de desfazer fluxo e recuperar seu custo.
- Linhas 6–7: se t não é alcançável a partir de s na rede auxiliar, o fluxo atual é máximo e o valor v não pode ser atingido; o algoritmo para com sol $=$ false.
- Linha 9: encontra o caminho P de menor custo de s a t na rede auxiliar. Como há arcos de custo negativo, o método usado aqui precisa tratá-los (ver as opções a seguir).
- Linha 10: δ é a menor capacidade residual do caminho, o máximo que se pode enviar por ele; δ' limita esse valor ao que ainda falta para atingir v.
- Linha 11: aumenta o fluxo em δ' nos arcos de P que são arcos originais e o diminui nos que são reversos. Como P é um caminho mínimo, o novo fluxo continua sendo de custo mínimo para o seu valor, o que garante a otimalidade ao final.
- **Complexidade:** Na redução do problema de atribuição, todas as capacidades são 1 e $v = n$, de modo que cada iteração envia uma unidade de fluxo e há no máximo n iterações. O custo de cada uma é dominado pela busca do caminho mínimo da linha 9; com Bellman-Ford, ela custa $O(VE)$, o que resulta em $O(nVE) \subseteq O(V^2E)$.

A complexidade do algoritmo é dependente do método utilizado para encontrar o caminho P. A implementação padrão utilizaria o algoritmo de Dijkstra, que tem uma complexidade de $O(E + V \log V)$ em uma implementação com filas de prioridade, onde V é o número de vértices e E o número de arestas. Porém, como o grafo residual pode conter arestas com custos negativos, essa implementação não funcionaria corretamente. Para contornar esse problema, temos duas opções principais:

- **Utilizar Bellman-Ford:** A solução mais simples e direta é utilizar, ao invés do algoritmo de Dijkstra, o algoritmo de Bellman-Ford para encontrar o caminho de custo mínimo P. O Bellman-Ford é mais lento, com uma complexidade de $O(VE)$, mas lida corretamente com arestas de custo negativo.

- **Utilizar a técnica de potenciais:** Permite que utilizemos o algoritmo de Dijkstra mesmo na presença de arestas de custo negativo.

  A cada iteração do loop, encontra-se o caminho de custo mínimo P utilizando Dijkstra com os custos reduzidos. Após encontrar o caminho, os potenciais são atualizados para todos os vértices v alcançáveis a partir de s no grafo residual, ajustando-os conforme a distância mínima encontrada pelo Dijkstra, de acordo com a fórmula $h(v) \leftarrow h(v) + d(s, v)$, onde $d(s, v)$ é a distância mínima de s até v no grafo residual com custos reduzidos.

  Abaixo uma adaptação do algoritmo 13 utilizando a técnica de potenciais:

Algoritmo 14: Adaptação do algoritmo 13 utilizando Dijkstra com técnica de potenciais

```
(1)  Procedure OPTFLOW_DIJKSTRA(G, c, s, t, γ, v; f, sol)
(2)      for e ∈ E do f(e) ← 0
(3)      for x ∈ V do π(x) ← 0
(4)      sol ← true, val ← 0
(5)      while sol = true and val < v do
(6)          construct the auxiliary network N' = (G', c', s, t)
(7)          for e = (u, v) ∈ E(G') do
(8)              define reduced cost: w(e) ← γ(e) + π(u) − π(v)
(9)          end
(10)         Run Dijkstra on N' using weights w to find shortest path distances d(·)
              from s
(11)         if t is not accessible from s (i.e., d(t) = ∞) then
(12)             sol ← false
(13)         else
(14)             for x ∈ V such that d(x) < ∞ do π(x) ← π(x) + d(x)
(15)             Identify shortest path P from s to t based on d(·)
(16)             δ ← min{c'(e) : e ∈ P}; δ' ← min(δ, v − val); val ← val + δ'
(17)             augment f along P by δ'
(18)         end
(19)     end
```

A estrutura é a mesma do Algoritmo 13; as diferenças estão no cálculo do caminho mínimo:

- Linhas 2–4: além do fluxo nulo, os potenciais π começam em 0. Isso é válido porque, na rede inicial, só há arcos originais, cujos custos de atribuição são não negativos.
- Linhas 5–6: como no Algoritmo 13, a cada iteração constrói-se a rede auxiliar, com seus arcos reversos de custo negativo.
- Linhas 7–8: cada arco (u, v) da rede auxiliar recebe o custo reduzido $w(e) = \gamma(e) + \pi(u) - \pi(v)$. Como os potenciais são as distâncias da iteração anterior,

que respeitam a desigualdade triangular, esses custos são não negativos, inclusive nos arcos reversos.

- Linha 10: executa Dijkstra com os custos reduzidos, o que é correto por eles serem não negativos. Em todo caminho de s a t, os potenciais intermediários se cancelam, e o custo reduzido difere do custo original apenas pela constante $\pi(s) - \pi(t)$. Por isso, o caminho mínimo é o mesmo nos dois custos.
- Linhas 11–12: se $d(t) = \infty$, t não é alcançável e o algoritmo para, como antes.
- Linha 14: os potenciais dos vértices alcançados são somados às distâncias encontradas. Com isso, os arcos do caminho mínimo passam a ter custo reduzido 0, e os arcos reversos criados pelo aumento também, o que mantém os custos reduzidos não negativos na próxima iteração.
- Linhas 15–17: o caminho é recuperado a partir das distâncias, e o aumento do fluxo segue exatamente as linhas 10–11 do Algoritmo 13.

A complexidade do algoritmo utilizando Dijkstra com a técnica de potenciais é $O(V(E + V \log V))$, onde V é o número de vértices e E o número de arestas do grafo. Isso ocorre porque, em cada iteração do loop principal, executamos o algoritmo de Dijkstra, que tem complexidade $O(E + V \log V)$ com filas de prioridade (heap de Fibonacci), e o cálculo dos custos reduzidos e a atualização dos potenciais custam apenas $O(E)$ e $O(V)$. O número de iterações é limitado pelo valor do fluxo, $v = n$, que no problema de atribuição não passa de V, o número de vértices do grafo bipartido.

3.2.2 Heurísticas

Assim como no problema de emparelhamento de cardinalidade máxima, as soluções exatas, apesar de apresentarem tempo polinomial, se tornam extremamente lentas para instâncias muito grandes. A seguir algumas heurísticas que objetivam resolver o problema de atribuição em tempo até mesmo linear ($O(E)$).

3.2.2.1 Heurística gulosa ordenada

Similar ao método guloso simples descrito na seção 3.1.3.1, com a diferença de que a ordem das arestas não é mais arbitrária, e sim decrescente por peso (ou crescente por custo, na atribuição de custo mínimo). Tem complexidade $O(E \log E)$ por conta da ordenação.

Diferentemente do problema de peso máximo (Seção 4.2.5), porém, na atribuição de custo mínimo o método não tem garantia de aproximação. Como a solução

precisa ser um emparelhamento perfeito, as arestas baratas escolhidas no início podem obrigar o uso de arestas arbitrariamente caras no final. Por exemplo, na matriz de custos $\begin{bmatrix} 0 & 1 \\ 1 & K \end{bmatrix}$, o método escolhe primeiro a aresta de custo 0 e é forçado a completar a atribuição com a de custo K, enquanto o ótimo custa 2; a razão $K/2$ cresce sem limite com K. Por isso, nesse problema, o método é classificado como heurística (Seção 2.6).

3.3 PROBLEMA DE ATRIBUIÇÃO GARGALO

O problema de atribuição gargalo, também conhecido como **Problema de Emparelhamento Min-Max** ou **Bottleneck Assignment problem**, é uma variação do problema de atribuição tradicional. O objetivo deste problema é encontrar, em um grafo bipartido ponderado, um emparelhamento de cardinalidade máxima (maior número possível de arestas) no qual o mínimo dos pesos das arestas do emparelhamento seja maximizado (LAWLER, 1976).

Por exemplo. Considere N trabalhadores e N estações de trabalho. w_{ij} representa a eficiência do trabalhador i na estação de trabalho j. A eficiência total da produção é limitada pela eficiência do trabalhador menos eficiente. O objetivo é atribuir trabalhadores às estações de trabalho de forma a maximizar a eficiência mínima entre todas as atribuições. (LAWLER, 1976).

3.3.1 Soluções exatas

A seguir serão apresentadas alguns dos principais métodos para encontrar uma solução exata para o problema de atribuição gargalo.

3.3.1.1 Métodos de threshold

Métodos de threshold funcionam alternando entre duas fases. A primeira fase consiste em selecionar um valor limite, chamado de **threshold**, descrito por $c*$. Na segunda fase, constrói-se um grafo bipartido com todas as arestas cujo peso é menor ou igual a $c*$. Em seguida, verifica-se se existe um emparelhamento perfeito nesse grafo. O menor valor de $c*$ que permite um emparelhamento perfeito é a solução ótima para o problema de atribuição gargalo (BURKARD; DELL'AMICO; MARTELLO, 2009).

Para selecionar o valor limite $c*$, podemos utilizar uma busca binária entre o menor e o maior peso das arestas do grafo original. A cada iteração, ajustamos o valor de $c*$ com base na existência ou não de um emparelhamento perfeito no grafo construído. Esse processo continua até que o valor ótimo seja encontrado. Burkard, Dell'Amico e Martello (2009), que descreve esse método, utiliza um algoritmo baseado em matrizes para decidir se um emparelhamento é perfeito ou não, porém podemos utilizar qualquer algoritmo clássico de emparelhamento em grafos bipartidos, como os

Tabela 2 – Algoritmos para problema de atribuição

Algoritmo	**Complexidade**	**Tipo**	**Avaliado**	**Descrição**
Método Húngaro	$O(n^3)$	Exato	Sim	Método primal-dual que mantém folga complementar para custo mínimo em bipartidos.
Jonker-Volgenant	$O(n^3)$	Exato	Sim	Algoritmo primal-dual eficiente que combina redução de custos inicial e busca de caminhos aumentantes com atualização otimizada de duais.
Custo mínimo (Bellman-Ford)	$O(V^2E)$	Exato	Sim	Método de caminhos sucessivos que usa Bellman-Ford para encontrar caminhos na rede residual, supor-

definidos na seção 3.1.2, como por exemplo o algoritmo de Hopcroft-Karp. Também segundo o autor, o algoritmo tem uma complexidade de tempo de $O(V^{2.5}/\sqrt{\log V})$.

Lawler (1976) menciona um método de threshold para solução do problema de atribuição gargalo, porém a definição de threshold dele é levemente diferente, referindo-se ao valor dinâmico do rótulo de um vértice durante busca de caminhos. O algoritmo que ele descreve se assemelha mais aos algoritmos de caminhos aumentantes, que serão discutidos na seção 3.3.1.3.

Algoritmo 15: Algoritmo Threshold para o LBAP (BURKARD; DELL'AMICO; MARTELLO, 2009)

(1) **let** $C = (c_{ij})$ be a given $n \times n$ cost matrix;
(2) $c_0^* := \min_{ij}\{c_{ij}\}, c_1^* := \max_{ij}\{c_{ij}\}$
(3) **if** $c_0^* = c_1^*$ **then**
(4) | $z := c_0^*$ **[comentário: qualquer permutação de** $\{1, 2, \ldots, n\}$ **é ótima]**
(5) **else**
(6) | **while** $C^* = \{c_{ij} : c_0^* < c_{ij} < c_1^*\} \neq \emptyset$ **do**
(7) | | **[comentário: encontra a mediana** c^* **de** C^***;]**
(8) | | $c^* := \min\{c \in C^* : |\{c_{ij} \in C^* : c_{ij} \leq c\}| \geq |C^*|/2\}$;
(9) | | Feasibility_check(c^*, c_0^*, c_1^*)
(10) | **end**
(11) | **if** *Feasibility_check has not yet been executed for* c_0^* **then**
(12) | | Feasibility_check(c_0^*, c_0^*, c_1^*)
(13) | **end**
(14) | find a perfect matching in the current bipartite graph $G[c_1^*]$
(15) | $z := c_1^*$ **[comentário: valor do emparelhamento ótimo]**
(16) **end**

(17) **Procedure** *Feasibility_check*(c^*, c_0^*, c_1^*)
(18) | define the current graph $G[c^*]$
(19) | **if** $G[c^*]$ *contains a perfect matching* **then**
(20) | | $c_1^* := c^*$
(21) | **else**
(22) | | $c_0^* := c^*$
(23) | **end**

O pseudocódigo segue a formulação de Burkard, Dell'Amico e Martello (2009), que minimiza o maior custo c_{ij} da atribuição, e é explicado a seguir com referência às linhas do Algoritmo 15. Nele, $G[c]$ denota o grafo bipartido que contém apenas as arestas $\{i, j\}$ com $c_{ij} \leq c$:

- **Inicialização (linhas 1–4):** O ótimo está sempre entre o menor e o maior custo da matriz, que formam o intervalo inicial de busca $[c_0^*, c_1^*]$ (linha 2). O algoritmo mantém a invariante de que $G[c_1^*]$ contém um emparelhamento perfeito, o que vale de início porque $G[\max c_{ij}]$ é o grafo bipartido completo. Se os dois extremos coincidem (linha 3), todos os custos são iguais e qualquer permutação é ótima

(linha 4).

- **Busca binária (linhas 6–9):** Enquanto existirem custos estritamente entre os dois extremos (conjunto C^* da linha 6), escolhe-se a mediana c^* desses custos como novo limiar (linha 8) e testa-se esse limiar (linha 9). Usar a mediana, em vez da média dos extremos, garante que cada teste descarte pelo menos metade dos valores candidatos, independentemente de como os custos estão distribuídos.

- **Teste de viabilidade (linhas 18–22):** O procedimento Feasibility_check constrói $G[c^*]$ (linha 18) e verifica se ele tem um emparelhamento perfeito (linha 19), o que pode ser feito com qualquer algoritmo de cardinalidade máxima, como o de Hopcroft-Karp. Se tem, o ótimo é no máximo c^*, e o extremo superior desce para c^* (linha 20); caso contrário, o ótimo é maior que c^*, e o extremo inferior sobe para c^* (linha 22).

- **Finalização (linhas 11–15):** Quando C^* fica vazio, o ótimo é c_0^* ou c_1^*. Como c_0^* pode nunca ter sido testado (é o mínimo inicial da matriz), ele é testado nesse momento, se necessário (linha 11); se for viável, o teste rebaixa c_1^* para ele. Por fim, extrai-se um emparelhamento perfeito de $G[c_1^*]$ (linha 14), cujo valor ótimo é $z = c_1^*$ (linha 15).

- **Complexidade:** Há no máximo n^2 custos distintos e cada teste descarta metade dos candidatos, de modo que o laço executa $O(\log n^2) = O(\log n)$ vezes. Cada iteração encontra a mediana em tempo linear no tamanho de C^*, $O(n^2)$, e faz um teste de emparelhamento perfeito, que custa $O(E\sqrt{V}) = O(n^{2.5})$ com Hopcroft-Karp, já que $G[c^*]$ tem $2n$ vértices e até n^2 arestas. O total é $O(n^{2.5} \log n)$; o limite de $O(n^{2.5}/\sqrt{\log n})$ citado por Burkard, Dell'Amico e Martello (2009) exige técnicas mais elaboradas para o teste de emparelhamento.

3.3.1.2 Método dual

Assim como os métodos de threshold, o método dual também define um valor limite $c*$. No entanto, ao invés de 'chutar' um valor usando busca binária, este método define o valor inicial de $c*$ como sendo o menor valor possível que ele pode assumir. Olhando para a matriz de custos, esse valor é o máximo entre os mínimos de cada linha e cada coluna da matriz:

$$c* = \max \left\{ \max_i (\min_j c_{ik}), \max_j (\min_i c_{kj}) \right\} \tag{3.2}$$

Com o limite definido, verificamos se é possível encontrar um emparelhamento perfeito no grafo bipartido formado pelas arestas cujo peso é menor ou igual a $c*$,

usando os mesmos algoritmos de antes. Se um emparelhamento perfeito for encontrado, então $c*$ é a solução ótima. Caso contrário, é necessário aumentar o valor de $c*$, e para isso nos baseamos no teorema de Kőnig, definido na seção 3.1.1.3.

Sabemos que todas as arestas $c_{ij} > c*$ estão conectadas a um conjunto de linhas I e um conjunto de colunas J. Essas linhas e colunas "cobrem"todas as arestas do grafo atual. Para aumentar o valor de $c*$, precisamos adicionar uma nova aresta que não esteja coberta por esses conjuntos. Procuramos o menor peso entre as arestas não cobertas, ou seja, aquelas que não pertencem às linhas em I nem às colunas em J. Esse valor se torna o novo $c*$:

$$Novoc* = \min_{i \notin I, j \notin J} c_{ij} \tag{3.3}$$

A partir daqui, o processo se repete. Com o novo valor de $c*$ e a nova aresta adicionada ao grafo, verificamos novamente a existência de um emparelhamento perfeito. Esse ciclo continua até que um emparelhamento perfeito seja encontrado, momento em que o valor atual de $c*$ será a solução ótima para o problema de atribuição gargalo.

Algoritmo 16: Dual_LBAP: algoritmo dual para o LBAP (BURKARD; DELL'AMICO; MARTELLO, 2009)

```
(1)  let C = (c_ij) be a given n × n cost matrix;
(2)  c* := max_{k=1,2,...,n}(min_{i=1,2,...,n} c_ik, min_{j=1,2,...,n} c_kj)
(3)  M := ∅
(4)  while |M| < n do
(5)      define the bipartite graph G[c*]
(6)      find a maximum matching M in G[c*]
(7)      if |M| < n then
(8)          let I ⊆ U and J ⊆ V be vertex sets in a minimum vertex cover of G[c*]
(9)          c* := min_{i∉I, j∉J} c_ij
(10)     end
(11) end
```

Com referência às linhas do Algoritmo 16:

- **Limite inferior (linhas 1–3):** Toda linha i precisa ser atribuída a alguma coluna, então o ótimo é pelo menos o menor custo dessa linha; o mesmo vale para cada coluna. O maior desses mínimos é, portanto, um limite inferior para o ótimo e é o valor inicial de c^* (linha 2). O emparelhamento começa vazio (linha 3).
- **Teste do limiar (linhas 4–6):** Enquanto o emparelhamento não for perfeito (linha 4), constrói-se o grafo $G[c^*]$ com as arestas de custo até c^* (linha 5) e calcula-se nele um emparelhamento máximo (linha 6). Se esse emparelhamento

tiver n arestas, o laço termina e c^* é o valor ótimo, porque nenhum valor menor que o atual foi viável.

- **Aumento do limiar (linhas 7–9):** Se o emparelhamento não é perfeito (linha 7), toma-se uma cobertura mínima de vértices de $G[c^*]$, formada pelas linhas I e colunas J (linha 8); pelo teorema de Kőnig, ela tem exatamente $|M| < n$ vértices. Só uma aresta fora da cobertura, isto é, com $i \notin I$ e $j \notin J$, pode aumentar o emparelhamento máximo, então o novo limiar é o menor custo entre essas arestas (linha 9). Como nenhum valor intermediário pode tornar o emparelhamento perfeito, o limiar nunca ultrapassa o ótimo.
- **Complexidade:** O limiar cresce estritamente a cada iteração e só assume valores da matriz, de modo que há no máximo n^2 iterações; cada uma calcula um emparelhamento máximo e uma cobertura mínima de vértices em um grafo com $2n$ vértices e até n^2 arestas. Reaproveitando o emparelhamento da iteração anterior, que continua válido em $G[c^*]$ após o aumento do limiar, cada iteração precisa apenas de uma busca alternante a partir dos vértices livres, em $O(n^2)$, que encontra um caminho aumentante ou, se não houver, a cobertura mínima. Isso dá um limite simples de $O(n^4)$ para o método.

O método dual não foi implementado no ambiente de testes deste trabalho. Para o problema de atribuição gargalo, foram comparados o algoritmo Threshold (Algoritmo 15) e o método de caminhos aumentantes (Algoritmo 17), descrito a seguir.

3.3.1.3 Método de caminhos aumentantes

Ao contrário do método dual, que ajusta um limiar global para todo o grafo, o método de caminhos aumentantes é construtivo. Ele inicia com um emparelhamento vazio e busca aumentar sua cardinalidade iterativamente, adicionando um par de vértices de cada vez. O objetivo central em cada passo é encontrar um caminho aumentante P que conecte uma linha livre a uma coluna livre, de tal forma que a capacidade desse caminho seja mínima. A capacidade aqui é definida pelo peso da aresta mais cara contida no caminho:

$$cap(P) = \max_{\{i,j\} \in P} c_{ij} \tag{3.4}$$

Para encontrar tal caminho, utiliza-se uma adaptação do algoritmo de Dijkstra. Em vez de somar os pesos das arestas, mantemos para cada vértice um valor de rótulo que representa o "gargalo"mínimo necessário para alcançá-lo. Durante a busca, ao tentar estender o caminho de um nó u (com rótulo d_u) para um vizinho v através da

aresta $\{u, v\}$, o custo não é somado, mas sim comparado. O novo custo potencial para atingir v é o máximo entre o gargalo já existente até u e o peso da nova aresta:

$$NovoCusto_v = \max(d_u, c_{uv}) \tag{3.5}$$

O algoritmo seleciona, a cada passo, o nó não visitado com o menor rótulo atual, garantindo uma exploração gulosa que minimiza o gargalo. Assim que uma coluna livre é alcançada por essa busca, o caminho aumentante é identificado e o emparelhamento é expandido, invertendo-se as arestas ao longo do caminho. Esse processo se repete (reiniciando as distâncias) até que um emparelhamento perfeito seja obtido. A solução do problema será determinada pela maior capacidade encontrada entre todos os caminhos aumentantes utilizados na construção.

Algoritmo 17: LBAP usando caminhos aumentantes (BURKARD; DELL'AMICO; MARTELLO, 2009)

(1) **let** $G = (U, V; E)$ be a bipartite graph with edge lengths c_{ij} for $[i, j] \in E$;
(2) find a lower bound c^* for the optimum objective function value;
(3) **[comentário: por exemplo,** $c^* := \max(\max_{i \in U} \min_{j \in V} c_{ij},\ \max_{j \in V} \min_{i \in U} c_{ij})$**]**
(4) find a maximum matching M in the graph $G[c^*]$ having an edge $[i, j]$ **iff** $c_{ij} \leq c^*$;
(5) **if** $|M| = |U|$ **then**
(6) | **stop [comentário:** M **é um emparelhamento ótimo de custo** c^***]**
(7) **else**
(8) | **let** L be the set of unmatched vertices of U
(9) **end**
(10) **while** L *is nonempty* **do**
(11) | choose an arbitrary vertex $i \in L$;
(12) | $L := L \setminus \{i\}$;
(13) | Dijkstra(i); **[comentário: o procedimento retorna um caminho** P **que começa em** i**]**
(14) | **if** $P \neq \text{nil}$ **then**
(15) | | $M := M \ominus P$;
(16) | | $c^* := \max(c^*, \ell(P))$
(17) | **end**
(18) **end**
(19) **[comentário:** M **é um emparelhamento máximo de custo mínimo** c^***.]**

Com referência às linhas do Algoritmo 17:

- **Ponto de partida (linhas 1–8):** Calcula-se um limite inferior c^* para o ótimo (linha 2), o mesmo do método dual: o maior entre os menores custos de cada linha e de cada coluna. Como arestas de custo até c^* não pioram a solução, encontra-se de uma vez um emparelhamento máximo em $G[c^*]$ (linha 4), com qualquer algoritmo de cardinalidade máxima. Se ele já é perfeito (linha 5), o limite

inferior foi atingido e a solução é ótima; senão, L recebe os vértices de U que ficaram livres (linha 8).

- **Laço de aumentos (linhas 10–13):** Cada vértice livre $i \in L$ é retirado de L uma única vez (linha 11) e, a partir dele, o procedimento Dijkstra (Algoritmo 18) busca o caminho aumentante de menor gargalo, isto é, aquele cuja aresta mais cara é a mais barata possível (linha 13).
- **Atualização (linhas 14–16):** Se existe caminho (linha 14), o emparelhamento é aumentado pela diferença simétrica com ele (linha 15) e o valor da solução passa a ser o maior entre o valor atual e o gargalo $\ell(P)$ do caminho usado (linha 16). Como cada caminho escolhido tem o menor gargalo possível, o valor final c^* é o ótimo.
- **Complexidade:** O emparelhamento inicial custa no máximo $O(n^{2.5})$ com Hopcroft-Karp. O laço executa no máximo n vezes, uma por vértice livre, e cada chamada do Dijkstra modificado custa $O(n^2)$ em uma matriz densa (ver a seguir), de modo que o total é $O(n^3)$.

Algoritmo 18: Dijkstra modificado para caminhos aumentantes em LBAP (BURKARD; DELL'AMICO; MARTELLO, 2009)

```
(1)  Procedure Dijkstra(i)
(2)      label all vertices j ∈ V by (α(j), β(j)) := (∞, nil);
(3)      R := V; [comentário: R contém os vértices de V ainda não
          examinados]
(4)      ᾱ(i) := c*, P := nil;
(5)      Label(i);
(6)      while R ≠ ∅ do
(7)          find a vertex j_1 ∈ R with minimum α(j_1);
(8)          if α(j_1) = ∞ then
(9)              R := ∅
(10)         else
(11)             if j_1 is unmatched then
(12)                 find the path P induced by the vertex sequence
                      (i, ..., β̄(β(j_1)), β(j_1), j_1);
(13)                 ℓ(P) := α(j_1);
(14)                 R := ∅
(15)             else
(16)                 let [i_1, j_1] be the matching edge;
(17)                 label i_1 with (ᾱ(i_1), β̄(i_1)) = (α(j_1), j_1);
(18)                 R := R \ {j_1};
(19)                 Label(i_1);
(20)             end
(21)         end
(22)     end

(23) Procedure Label(i)
(24)     for each neighbor j ∈ R of i do
(25)         if α(j) > max(ᾱ(i), c_ij) then
(26)             α(j) := max(ᾱ(i), c_ij);
(27)             β(j) := i
(28)         end
(29)     end
```

O procedimento mantém dois rótulos para cada vértice: para $j \in V$, $\alpha(j)$ é o menor gargalo conhecido de um caminho alternante de i até j, e $\beta(j)$ é o vértice de U que o precede nesse caminho; para $i' \in U$, $\overline{\alpha}(i')$ e $\overline{\beta}(i')$ têm o mesmo papel. Com referência às linhas do Algoritmo 18:

- **Inicialização (linhas 2–5):** Todos os vértices de V começam com gargalo infinito e sem predecessor (linha 2) e ficam no conjunto R dos ainda não examinados (linha 3). A raiz i recebe o gargalo c^* (linha 4), e não 0, porque arestas de custo até o valor atual da solução não a pioram; assim, entre caminhos de mesmo efeito sobre o resultado, qualquer um serve. Em seguida, os vizinhos de i são rotulados (linha 5).

- **Escolha do próximo vértice (linhas 6–8):** Como no Dijkstra tradicional, escolhe-se o vértice de R de menor rótulo (linha 7). Se esse rótulo é infinito (linha 8), nenhum vértice restante é alcançável a partir de i, a busca termina e o caminho retornado é $P = \mathrm{nil}$.

- **Coluna livre encontrada (linhas 11–13):** Se j_1 não está emparelhado (linha 11), o caminho aumentante foi encontrado. Ele é reconstruído de trás para frente, seguindo os predecessores β e $\overline{\beta}$ até i (linha 12), e seu gargalo é o rótulo de j_1 (linha 13).

- **Extensão pela aresta emparelhada (linhas 16–19):** Se j_1 está emparelhado com i_1 (linha 16), o caminho alternante segue obrigatoriamente pela aresta $\{i_1, j_1\}$, que já pertence ao emparelhamento e não altera o gargalo. Por isso, i_1 herda o rótulo de j_1 e o tem como predecessor (linha 17); j_1 sai de R e os vizinhos de i_1 são rotulados (linha 19).

- **Procedimento Label (linhas 24–27):** Para cada vizinho j de i ainda em R (linha 24), o gargalo de chegar a j por i é o maior entre o gargalo até i e o custo c_{ij}. Se esse valor é menor que o rótulo atual de j (linha 25), o rótulo é atualizado e i passa a ser o predecessor de j (linha 27). É o relaxamento do Dijkstra, com o máximo no lugar da soma.

- **Complexidade:** Cada vértice de V sai de R no máximo uma vez, e cada vértice de U é rotulado no máximo uma vez. Com a matriz densa, a busca do mínimo (linha 7) percorre R em $O(n)$ e cada chamada de Label percorre os $O(n)$ vizinhos, o que dá $O(n^2)$ por chamada do procedimento.

3.3.2 Heurísticas

Assim como na atribuição linear, os métodos gulosos para o problema de atribuição gargalo não têm garantia de aproximação e são, portanto, heurísticas (Seção 2.6). Esta seção apresenta a heurística construtiva gulosa, que adapta o critério de escolha das arestas ao objetivo de minimizar o maior custo.

3.3.2.1 Heurística Construtiva Gulosa

Assim como no problema de atribuição, o guloso simples não oferece garantia de qualidade; no problema de gargalo, ele deve ainda ser adaptado para evitar criar “bicos de garrafa” muito altos no final do processo.

O algoritmo seleciona arestas iterativamente. Para cada vértice livre, ele verifica não apenas a aresta de menor peso incidente, mas também o "custo de oportunidade"de não escolhê-la.

Se não escolhermos a aresta $\{u, v\}$ de peso 10, a próxima melhor opção para u tem peso 100? Se sim, a prioridade de $\{u, v\}$ aumenta drasticamente (MARTELLO; TOTH, 1987).

3.4 PROBLEMA DO EMPARELHAMENTO ESTÁVEL

O problema do emparelhamento estável, também conhecido como *Stable Marriage Problem* (SMP), é um problema clássico na teoria dos grafos e na ciência da computação, proposto inicialmente por Gale e Shapley em 1962 (KLEINBERG; TARDOS, 2005). Ele envolve a combinação de dois conjuntos distintos de elementos e, apesar de semelhante aos problemas de emparelhamento em grafos bipartidos estudados anteriormente, possui características e objetivos diferentes.

Imagine dois conjuntos distintos de mesmo tamanho, N homens e N mulheres. Cada indivíduo em ambos os conjuntos tem uma lista de preferências ordenadas para todos os membros do outro conjunto. O objetivo do SMP é encontrar um emparelhamento entre homens e mulheres de tal forma que não existam dois indivíduos, um homem e uma mulher, que prefeririam estar juntos em vez de com seus parceiros atuais. Se tal par existir, o emparelhamento é considerado instável.

O problema pode ser resolvido utilizando o algoritmo de Gale-Shapley, que usa um processo iterativo de propostas e rejeições para garantir que o emparelhamento final seja estável.

3.4.1 Propriedades

- **Estabilidade**: Um emparelhamento é estável se não houver dois indivíduos que prefeririam estar juntos em vez de com seus parceiros atuais. A estabilidade é a característica central do problema (LAWLER, 1976).
- **Existência de solução**: Gale e Shapley provaram que sempre existe pelo menos um emparelhamento estável para qualquer conjunto de preferências (AHUJA; MAGNANTI; ORLIN, 1993).
- **Otimalidade para quem propõe**: O emparelhamento resultante do algoritmo de Gale-Shapley é ótimo para o conjunto que faz as propostas (por exemplo, os homens), significando que cada indivíduo nesse conjunto recebe o melhor parceiro possível dentro dos emparelhamentos estáveis (AHUJA; MAGNANTI; ORLIN, 1993).
- **Pessimismo para quem recebe propostas**: Por outro lado, o emparelhamento é o pior possível para o conjunto que recebe as propostas (por exemplo, as mulheres), significando que cada indivíduo nesse conjunto recebe o pior parceiro

Tabela 3 – Algoritmos para problema de atribuição gargalo

Algoritmo	**Complexidade**	**Tipo**	**Avaliado**	**Descrição**
Threshold	$O(V^{2.5}/\sqrt{\log V})$	Exato	Sim*	Aplica busca binária sobre os pesos das arestas, testando a existência de emparelhamento perfeito no subgrafo restrito ao limiar.
Método Dual	$O(K \cdot E\sqrt{V})$	Exato	Não	Expande um limiar inferior iterativamente baseado no Teorema de König, adicionando apenas arestas necessárias para cobrir componentes.
Caminhos aumentantes	$O(EV)$	Exato	Sim	Abordagem construtiva que busca caminhos aumentantes minimizando a aresta de peso máximo (gargalo) no trajeto (variação de Dijkstra).
Construtiva Gulosa	$O(E \log E)$	Heurística	Não	Constrói a solução selecionando arestas de baixo peso, usando critérios locais ou de "arrependimento"para evitar gargalos futuros.

Avaliado: incluído no ambiente de testes; a origem de cada implementação consta na Tabela 10. * A implementação avaliada testa cada limiar por caminhos aumentantes, com custo $O(n^3)$ por teste e $O(n^3 \log n)$ no total.

possível dentro dos emparelhamentos estáveis (AHUJA; MAGNANTI; ORLIN, 1993).

- **Não-unicidade**: Pode haver múltiplos emparelhamentos estáveis para um dado conjunto de preferências, dependendo das listas de preferências dos indivíduos (KLEINBERG; TARDOS, 2005).

3.4.2 Algoritmo de Gale-Shapley

A solução clássica para o problema do emparelhamento estável é o algoritmo de Gale-Shapley, também conhecido como o algoritmo de casamento estável. O algoritmo funciona sob a lógica de propostas e rejeições, onde um dos conjuntos (por exemplo, os homens) faz propostas às mulheres com base em suas listas de preferências.

O Algoritmo 19 apresenta o pseudocódigo, em que M é o conjunto dos homens e W o das mulheres, cada um com n elementos:

Algoritmo 19: Algoritmo de Gale-Shapley para o problema do emparelhamento estável (KLEINBERG; TARDOS, 2005)

```
(1)  Initially all m ∈ M and w ∈ W are free
(2)  while there is a man m who is free and hasn't proposed to every woman do
(3)      Choose such a man m
(4)      Let w be the highest-ranked woman in m's preference list to whom m has
           not yet proposed
(5)      if w is free then
(6)          (m, w) become engaged
(7)      else
(8)          w is currently engaged to m'
(9)          if w prefers m' to m then
(10)             m remains free
(11)         else
(12)             (m, w) become engaged
(13)             m' becomes free
(14)         end
(15)     end
(16) end
(17) Return the set S of engaged pairs
```

O algoritmo funciona sob a lógica de propostas e rejeições (KLEINBERG; TARDOS, 2005):

- Linha 1: todos começam livres, sem nenhum par formado.
- Linhas 2–3: enquanto houver um homem m livre que ainda não propôs a todas as mulheres, escolhe-se um deles. A ordem de escolha não altera o resultado final.

- Linha 4: m propõe à mulher w mais bem colocada na sua lista de preferências entre as que ainda não recebeu proposta dele. Assim, cada homem percorre sua lista em ordem e nunca propõe duas vezes à mesma mulher.
- Linhas 5–6: se w está livre, ela aceita a proposta e os dois ficam noivos. Uma mulher que recebeu alguma proposta nunca volta a ficar livre.
- Linhas 8–10: se w já está noiva de m', ela compara os dois segundo a sua lista. Se prefere m', rejeita m, que continua livre e, numa próxima iteração, proporá à mulher seguinte da sua lista.
- Linhas 12–13: se w prefere m, ela troca de noivo: m e w ficam noivos e m' volta a ficar livre. Por isso, o parceiro de cada mulher só melhora ao longo da execução.
- Linha 17: o laço termina quando todos os homens estão noivos, e o conjunto S dos pares noivos é devolvido. S é um emparelhamento perfeito e estável: se um homem m preferisse uma mulher w ao seu par final, ele teria proposto a w antes, e w só o teria rejeitado ou trocado por alguém que ela prefere; logo, não existe par instável (KLEINBERG; TARDOS, 2005).
- **Complexidade:** Cada homem propõe no máximo uma vez a cada mulher, de modo que há no máximo n^2 iterações. Cada iteração custa $O(1)$ quando se mantém: uma lista dos homens livres, um ponteiro para a próxima mulher da lista de cada homem, o noivo atual de cada mulher e uma matriz de posições $\mathrm{rank}[w][m]$, calculada uma vez em $O(n^2)$, que permite comparar dois homens em tempo constante. O total é $O(n^2)$, linear no tamanho da entrada, que é formada pelas $2n$ listas de n preferências (KLEINBERG; TARDOS, 2005).

Neste trabalho, o algoritmo de Gale-Shapley foi avaliado em três implementações, cuja origem consta na Tabela 10.

4 PROBLEMAS DE EMPARELHAMENTO EM GRAFOS GERAIS

Ao contrário dos problemas em grafos bipartidos, problemas de emparelhamento em grafos gerais podem ser consideravelmente mais complexos. Alguns dos problemas apresentados neste capítulo, como o emparelhamento de cardinalidade máxima e o de peso máximo, são resolvidos por algoritmos exatos em tempo polinomial. Outros, como o emparelhamento induzido máximo, são NP-difíceis: não se conhece algoritmo de tempo polinomial que os resolva, e a existência de um implicaria P = NP.

4.1 EMPARELHAMENTO DE CARDINALIDADE MÁXIMA

A definição do problema de emparelhamento de cardinalidade máxima é, fundamentalmente, a mesma da sua contraparte bipartida, descrita na seção 3.1. No entanto, a solução deste problema é significativamente mais complexa que a versão bipartida, mesmo ainda sendo em tempo polinomial.

4.1.1 Propriedades

A seguir serão apresentadas algumas propriedades importantes para a compreensão dos algoritmos relacionados ao problema de emparelhamento de cardinalidade máxima em grafos gerais.

4.1.1.1 Ciclos ímpares

Previamente, na seção 3.1.1.2, vimos a definição de caminhos aumentantes, que vieram a ser fundamentais para a solução de grande parte dos problemas de emparelhamento em grafos bipartidos. Em grafos gerais, a base da solução ainda é a mesma, se baseando no lema de Berge, descrito por Jungnickel (2008) como:

> Um emparelhamento M em um grafo G é máximo se, e somente se, não existe nenhum caminho aumentante em G relativo a M.

Em grafos bipartidos, ciclos sempre têm tamanho par. Isso garante que, ao realizar uma busca (BFS ou DFS) a partir de um vértice livre, nunca encontraremos uma aresta que conecte dois vértices que estejam à mesma distância ("paridade") da raiz na árvore de busca alternante (AHUJA; MAGNANTI; ORLIN, 1993).

No entanto, em grafos gerais, a presença de ciclos de comprimento ímpar quebra essa lógica. Schrijver (2004) explica que, ao explorar o grafo, o algoritmo pode encontrar uma aresta conectando dois vértices que possuem paridade "par"em relação

à raiz da busca. Essa estrutura forma um **ciclo ímpar** que "confunde"a identificação de um caminho aumentante simples, pois um vértice poderia ser alcançado por dois caminhos alternantes de paridades diferentes.

4.1.1.2 Flor (Blossom)

A solução para lidar com ciclos ímpares foi proposta por Edmonds (1965), e consiste em uma estrutura denominada **Blossom**.

Uma Blossom B é definida como um ciclo de comprimento ímpar $2k+1$, onde k arestas pertencem ao emparelhamento M, de tal forma que existe apenas um vértice v_B no ciclo (a base da flor) que não é coberto por uma aresta de M dentro do próprio ciclo. A base v_B é conectada ao resto do grafo por um caminho alternante de comprimento par (o **caule** ou *stem*).

Edmonds provou que uma *blossom* pode ser contraída em um único vértice, com base no seguinte teorema:

> Existe um caminho aumentante no grafo original G se, e somente se, existe um caminho aumentante no grafo contraído $G' = G/B$, onde a flor B foi reduzida a um único vértice. (SCHRIJVER, 2004)

4.1.2 O algoritmo de Edmonds

Baseando-se na propriedade de Blossom e no teorema da contração, Lawler (1976) descreve o algoritmo para encontrar o emparelhamento máximo em grafos gerais da seguinte forma:

1. **Busca:** Inicie uma busca (geralmente BFS) a partir dos vértices livres para encontrar um caminho aumentante, construindo uma Floresta de Árvores Alternantes. Na versão de Jungnickel (2008) (Algoritmo 20), a busca parte de um vértice livre r de cada vez (linha 12) e cresce a árvore alternando uma aresta não emparelhada $\{x, y\}$ com a aresta emparelhada $\{y, mate(y)\}$ (linhas 33–35).

2. **Detecção:** Se a busca encontrar uma aresta que fecha um ciclo ímpar, uma **blossom** foi identificada. Isso ocorre quando a aresta $\{x, y\}$ liga dois vértices de profundidade par da mesma árvore (linha 23). O procedimento BLOSSOM (Algoritmo 21) identifica o ciclo, subindo de x e de y até a raiz e tomando como base w o ancestral comum mais profundo.

3. **Contração:** Contraia todos os vértices da **blossom** em um único super-vértice e modifique o grafo temporariamente. O procedimento CONTRACT (Algoritmo 22)

cria o super-vértice $b = n + m$, que herda o pai, a profundidade e o parceiro da base, recebe a união das adjacências da flor e desativa os vértices originais.

4. **Recursão:** Continue a busca no grafo contraído. Na versão de Jungnickel (2008), isso é feito sem recursão explícita: o super-vértice entra na fila de busca (linha 4 do Algoritmo 22) e passa a ser tratado como um vértice comum, podendo ele próprio fazer parte de novas flores.

5. **Expansão:** Se um caminho aumentante for encontrado no grafo contraído passando pelo super-vértice, "abra"(expanda) a flor novamente para traduzir esse caminho para o grafo original. O caminho dentro da flor será roteado de forma a manter a alternância correta. É o primeiro laço do procedimento AUGMENT (Algoritmo 23, linha 6), que substitui cada super-vértice do caminho, do mais recente para o mais antigo, por um caminho alternante de comprimento par dentro da flor.

6. **Aumento:** Use o caminho encontrado para aumentar o emparelhamento através da diferença simétrica (inversão das arestas). O AUGMENT é chamado quando a busca alcança um vértice livre y (linha 30 do Algoritmo 20), e seu último laço (linha 16) inverte as arestas ao longo do caminho.

Schrijver (2004) detalha que, a depender da forma como o algoritmo é implementado, principalmente no que trata da contração, a complexidade do algoritmo ingênuo pode chegar a $O(V^3)$. No entanto, melhorias subsequentes nesse algoritmo culminaram em uma implementação em $O(E\sqrt{V})$, descrita por Micali e Vazirani (1980). No entanto, por conta da complexidade de implementação da solução mais rápida, a maioria das implementações práticas do algoritmo utilizam variantes com complexidade próxima a $O(V^3)$.

Os pseudocódigos a seguir, de Jungnickel (2008), organizam esses passos em quatro procedimentos: MAXMATCH, que conduz a busca; BLOSSOM, que identifica a flor; CONTRACT, que a contrai; e AUGMENT, que expande as flores e aumenta o emparelhamento. Cada passo acima indica a parte correspondente.

Algoritmo 20: Procedimento MAXMATCH (JUNGNICKEL, 2008)

```
(1)  Procedure MAXMATCH(G; mate, nrex)
(2)      INMATCH(G; mate, nrex)
(3)      r ← 0
(4)      while nrex ≥ 2 and r < n do
(5)          r ← r + 1
(6)          if mate(r) = 0 then
(7)              Q ← ∅, aug ← false, m ← 0
(8)              for v ∈ V do
(9)                  p(v) ← 0, d(v) ← −1, a(v) ← true
(10)                 CA(v) ← A_v
(11)             end
(12)             d(r) ← 0; append r to Q
(13)             while aug = false and Q ≠ ∅ do
(14)                 remove the first vertex x of Q
(15)                 if a(x) = true then
(16)                     cont ← false
(17)                     for y ∈ CA(x) do
(18)                         u(y) ← false
(19)                     end
(20)                     repeat
(21)                         choose y ∈ CA(x) with u(y) = false; u(y) ← true
(22)                         if a(y) = true then
(23)                             if d(y) ≡ 0 (mod 2) then
(24)                                 m ← m + 1
(25)                                 BLOSSOM(x, y; B(m), w(m))
(26)                                 CONTRACT(B(m), m, w)
(27)                             end
(28)                             else if d(y) = −1 then
(29)                                 if mate(y) = 0 then
(30)                                     AUGMENT(x, y)
(31)                                 else
(32)                                     z ← mate(y)
(33)                                     p(y) ← x; d(y) ← d(x) + 1
(34)                                     p(z) ← y; d(z) ← d(y) + 1
(35)                                     insert z with priority d(z) into Q
(36)                                 end
(37)                             end
(38)                         end
(39)                     until u(y) = true for all y ∈ CA(x) or aug = true or
                             cont = true
(40)                 end
(41)             end
(42)         end
(43)     end
```

Algoritmo 21: Algoritmo de BLOSSOM (JUNGNICKEL, 2008)

```
(1)  Procedure BLOSSOM(x, y; B, w)
(2)      P ← {x}; P′ ← {y}; u ← x; v ← y
(3)      repeat
(4)          P ← P ∪ {p(u)}; u ← p(u)
(5)      until p(u) = r
(6)      repeat
(7)          P′ ← P′ ∪ {p(v)}; v ← p(v)
(8)      until v = r
(9)      S ← P ∩ P′
(10)     let w be the element of S for which d(w) ≥ d(z) for all z ∈ S
(11)     B ← ((P ∪ P′) \ S) ∪ {w}
```

O procedimento BLOSSOM é chamado pelo MAXMATCH quando a aresta $\{x, y\}$ liga dois vértices de profundidade par da mesma árvore alternante, de raiz r (linha 23 do Algoritmo 20). Seu papel é identificar o conjunto de vértices B da flor e a sua base w:

- **Linha 2:** Os conjuntos P e P' guardarão os caminhos de x e de y até a raiz, e começam contendo apenas o próprio vértice de partida.
- **Linhas 4 e 7:** Dois laços sobem na árvore pelos ponteiros de pai $p(\cdot)$, o primeiro a partir de x e o segundo a partir de y, até alcançar a raiz r. Ao final, P e P' contêm todos os vértices dos caminhos de x e de y até r.
- **Linha 9:** A interseção S dos dois caminhos é o trecho que eles têm em comum, de algum vértice até a raiz.
- **Linha 10:** A base w é o vértice de S de maior profundidade $d(\cdot)$, ou seja, o ancestral comum mais profundo de x e y: é nele que os dois caminhos se encontram e se fecha o ciclo ímpar.
- **Linha 11:** A flor B é formada pelos vértices dos dois caminhos que estão abaixo de w, mais a própria base. Os vértices de S acima de w ficam de fora, pois pertencem ao caule, e não ao ciclo. Em seguida, o MAXMATCH chama o CONTRACT (Algoritmo 22) com B e w.
- **Complexidade:** Cada caminho até a raiz tem no máximo $O(V)$ vértices, e a interseção e a união dos conjuntos podem ser feitas em tempo linear no tamanho deles; cada chamada custa, portanto, $O(V)$. Como cada contração reduz o número de vértices ativos, há no máximo $O(V)$ flores por busca, o que é compatível com o custo total $O(V^3)$ do algoritmo citado acima.

Algoritmo 22: Procedimento CONTRACT (JUNGNICKEL, 2008)

```
(1)  Procedure CONTRACT(B, m, w)
(2)      b ← n + m; a(b) ← true
(3)      p(b) ← p(w); d(b) ← d(w); mate(b) ← mate(w)
(4)      insert b into Q with priority d(b)
(5)      CA(b) ← ⋃_{z∈B} CA(z)
(6)      for z ∈ CA(b) do
(7)          CA(z) ← CA(z) ∪ {b}
(8)      end
(9)      for z ∈ B do
(10)         a(z) ← false
(11)     end
(12)     for z ∈ CA(b) do
(13)         if a(z) = true and p(z) ∈ B then
(14)             d(z) ← d(b) + 1; p(z) ← b
(15)             d(mate(z)) ← d(z) + 1
(16)         end
(17)     end
(18)     cont ← true
```

Algoritmo 23: Procedimento AUGMENT (JUNGNICKEL, 2008)

```
(1)  Procedure AUGMENT(x, y)
(2)      P ← {y, x}; v ← x
(3)      while p(v) ≠ 0 do
(4)          P ← P ∪ {p(v)}; v ← p(v)
(5)      end
(6)      while there exists b ∈ P with b > n do
(7)          choose the largest b ∈ P with b > n
(8)          B ← B(b − n); w ← w(b − n); z ← mate(w)
(9)          let q be the neighbor of b on P which is different from z
(10)         choose some q′ ∈ B ∩ CA(q)
(11)         determine the alternating path B′ of even length in B from w to q′
(12)         replace b by w in P
(13)         insert B′ into P between w and q
(14)     end
(15)     u ← y; v ← x
(16)     while v ≠ r do
(17)         z ← mate(v); mate(v) ← u; mate(u) ← v
(18)         u ← z; let v be the successor of z on P
(19)     end
(20)     mate(v) ← u; mate(u) ← v
(21)     nrex ← nrex − 2; aug ← true
```

4.1.2.1 Teorema de Tutte-Berge

O teorema de Tutte-Berge, apesar de não aparecer explicitamente na descrição do algoritmo, é fundamental para sua corretude, pois ele fornece um certificado de

otimalidade. Por exemplo, digamos que a etapa de busca do algoritmo de Blossom não encontrou nenhum caminho aumentante. Como sabemos que o emparelhamento atual é máximo? Para descobrir isso, utilizamos a fórmula de Tutte-Berge, descrita por Jungnickel (2008) da seguinte forma:

$$v(G) = \frac{1}{2} \min_{U \subseteq V} (|V| + |U| - \mathrm{odd}(G - U)) \tag{4.1}$$

Onde $v(G)$ é o tamanho do emparelhamento máximo, e $\mathrm{odd}(G - U)$ é o número de componentes conexos com número ímpar de vértices no grafo resultante após a remoção de U. Se o grafo possui um emparelhamento perfeito, o mínimo desta equação é $|V|/2$, o que ocorre se e somente se $odd(G - U) \leq |U|$ para todo U.

4.1.3 Algoritmos aproximados

Alguns dos algoritmos aproximados que resolvem este problema são os mesmos vistos na seção 3.1, como o método guloso simples e o algoritmo de Karp-Sipser, que também funcionam para grafos gerais e mantêm a garantia de metade da cardinalidade máxima, já que ambos produzem um emparelhamento maximal. Além desses, existem outros métodos que podem ser utilizados neste problema.

4.1.3.1 Algoritmo de crescimento de caminho (Path Growing)

Uma variação inteligente do método guloso, descrita por Drake e Hougardy (2003), que usa DFS e tenta evitar a "miopia"de escolher uma aresta isolada. Assim como os anteriores, garante um emparelhamento com pelo menos metade da cardinalidade máxima, caso particular da garantia de metade do peso máximo da versão ponderada (Seção 4.2.5).

1. Escolha um vértice livre inicial v.
2. Inicie uma busca em profundidade (DFS) para construir um caminho simples o mais longo possível.
3. Ao longo desse caminho, selecione arestas alternadas para o emparelhamento (a 1ª sim, a 2ª não, a 3ª sim...).
4. Remova os vértices usados e repita.

Ao contrário do guloso simples (que escolhe arestas desconexas), o Path Growing tende a cobrir cadeias longas de vértices, o que é estruturalmente vantajoso.

Tabela 4 – Algoritmos para problema de emparelhamento de cardinalidade máxima em grafos gerais

Algoritmo	**Complexidade**	**Tipo**	**Avaliado**	**Descrição**
Edmonds	$O(V^3)$	Exato	Sim	Algoritmo exato (Blossom) que contrai ciclos ímpares temporariamente em supervértices para identificar caminhos aumentantes.
Micali e Vazirani	$O(E\sqrt{V})$	Exato	Não	Variação otimizada do algoritmo de Edmonds que utiliza estruturas de dados avançadas para reduzir drasticamente a complexidade.
Guloso simples	$O(E)$	Aproximado	Não	Itera sobre arestas arbitrariamente, selecionando aquelas que conectam dois vértices livres.
Karp-Sipser	$O(E)$	Aproximado	Não	Prioriza remoção de vértices de grau 1; usa escolha aleatória quando não há folhas.
Crescimento de caminho	$O(E)$	Aproximado	Não	Método linear que constrói caminhos simples via DFS e seleciona arestas alternadas para garantir um emparelhamento maximal.

Avaliado: incluído no ambiente de testes; a origem de cada implementação consta na Tabela 10.

4.2 EMPARELHAMENTO PONDERADO MÁXIMO

Enquanto o problema de cardinalidade máxima busca o maior número de arestas, o problema do **Emparelhamento de Peso Máximo** (Maximum Weight Matching) introduz uma função de peso $w : E \to \mathbb{R}$ nas arestas, e o objetivo torna-se encontrar um emparelhamento M que maximize $\sum_{e \in M} w(e)$. Este problema é a generalização natural do Problema da Atribuição para grafos não-bipartidos, descrito em 3.2.

A complexidade deste problema em grafos gerais decorre, assim como no problema de emparelhamento de cardinalidade máxima descrito em 4.1, da presença de ciclos ímpares. Se tentarmos modelar o problema usando Programação Linear (PL) apenas com restrições de grau ($\sum x_e \leq 1$), a solução ótima do sistema relaxado pode ser fracionária (ex: $x_e = 0.5$ para as três arestas de um triângulo), o que não corresponde a um emparelhamento válido.

4.2.1 Formulação poliédrica e Restrições de Blossom

Conforme detalhado por Schrijver (2004), para descrever o **Poliedro do Emparelhamento** (Matching Polytope) em grafos gerais, Edmonds (1965) provou que é necessário adicionar uma família exponencial de restrições conhecidas como **Desigualdades de Blossom** (ou restrições de subconjuntos ímpares):

$$\sum_{e \in E(U)} x_e \leq \frac{|U| - 1}{2}, \quad \forall U \subseteq V, |U| \text{ ímpar}, |U| \geq 3 \tag{4.2}$$

Onde $E(U)$ é o conjunto de arestas com ambas as extremidades em U. Essas inequações garantem que, em qualquer subconjunto ímpar de vértices, o número de arestas selecionadas não exceda a capacidade máxima inteira possível.

4.2.2 A abordagem Primal-Dual

Diferente do caso de cardinalidade, onde buscamos apenas caminhos aumentantes, o algoritmo para o caso ponderado é um método **Primal-Dual**. Ele é uma generalização do Método Húngaro, mas adaptado para lidar com as blossoms.

Segundo Lawler (1976), o algoritmo mantém variáveis duais não apenas para os vértices (y_v), mas também para cada subconjunto ímpar de vértices (z_B) que forma uma blossom contraída. A condição de otimalidade baseia-se na folga complementar. Uma aresta $\{u, v\}$ é considerada "admissível"ou "justa"(*tight*) se a soma dos duais correspondentes iguala o peso da aresta. Simplificadamente:

$$y_u + y_v + \sum_{B : \{u,v\} \subseteq B} z_B = w_{uv} \tag{4.3}$$

O algoritmo opera iterativamente em dois estágios, descritos por Jungnickel (2008) e Schrijver (2004):

1. **Estágio Primal (Busca):** Restringindo-se apenas às arestas admissíveis (onde a equação acima é satisfeita), o algoritmo busca um caminho aumentante usando a técnica de contração de blossoms de Edmonds. Se encontrado, o emparelhamento é aumentado.
2. **Estágio Dual (Ajuste de Preços):** Se a busca falha em encontrar um caminho aumentante no grafo de arestas admissíveis, o algoritmo não termina (ao contrário do caso de cardinalidade). Em vez disso, ele realiza uma atualização das variáveis duais y e z.
    - O algoritmo calcula um valor δ (delta), que é a menor "folga"entre as arestas que conectam a árvore de busca atual a vértices externos.
    - Os duais dos vértices na árvore são ajustados por $\pm\delta$ e os duais das blossoms (z_B) também são ajustados.
    - Esse ajuste tem o efeito de "destravar"novas arestas (tornando-as admissíveis) ou permitir a expansão/contração de blossoms, possibilitando que a busca prossiga.

4.2.3 O algoritmo de Edmonds Ponderado

O procedimento resultante é frequentemente chamado de **Algoritmo de Blossom Ponderado** ou simplesmente Algoritmo de Edmonds para Emparelhamento Ponderado.

A implementação original de Edmonds era polinomial, $O(V^4)$. Lawler (1976) apresentou uma implementação $O(V^3)$, que por muito tempo foi o padrão em livros didáticos.

A variante $O(V^3)$ descrita por Lawler (1976) para o emparelhamento ponderado utiliza uma abordagem primal-dual rigorosa. O algoritmo mantém um conjunto de arestas admissíveis (custo reduzido nulo) baseadas em variáveis duais de vértices (y_i) e de subconjuntos ímpares (z_B). A cada iteração onde a busca por caminhos aumentantes falha, o algoritmo calcula um escalar crítico δ baseado nas folgas das arestas incidentes à árvore alternante atual (conjunto S). A atualização dual subsequente ($y_i \leftarrow y_i \pm \delta$, $z_B \leftarrow z_B \pm 2\delta$) garante o surgimento de novas arestas admissíveis ou a expansão de **blossoms** com dual nulo, mantendo a complexidade cúbica através do gerenciamento eficiente dos valores de folga mínima (**slack**) para vértices fora da árvore.

Lawler também destaca que a manipulação das variáveis duais das blossoms (z_B) é o que permite ao algoritmo "pagar"o custo de usar arestas dentro de estruturas

cíclicas ímpares. Quando uma blossom é formada durante a busca, ela recebe um valor dual $z_B > 0$; quando ela precisa ser desfeita (expandida) para encontrar um caminho, seu valor dual retorna a zero.

O Algoritmo 24 resume esse esquema primal-dual em alto nível, omitindo as estruturas de dados de contração e expansão de blossoms. Nele, $\pi_{uv} = y_u + y_v + \sum_{B:\{u,v\}\subseteq B} z_B - w_{uv}$ denota a folga da aresta $\{u, v\}$; uma aresta é justa quando $\pi_{uv} = 0$.

Algoritmo 24: Esquema primal-dual do algoritmo de Edmonds ponderado, em alto nível (LAWLER, 1976)

Input: graph $G = (V, E)$ with weights $w : E \to \mathbb{R}$
Output: maximum weight matching M

(1) $M := \emptyset$; $y_v := \frac{1}{2}\max_{e\in E} w(e)$ for all $v \in V$; $z_B := 0$ for every blossom B
(2) **while true do**
(3) grow alternating trees rooted at the free vertices using only tight edges, labeling vertices as outer (S) or inner (T)
(4) **if** *a tight edge joins outer vertices of two different trees* **then**
(5) augment M along the augmenting path found; remove all labels
(6) **else if** *a tight edge joins two outer vertices of the same tree* **then**
(7) contract the blossom B formed by the edge, with $z_B := 0$
(8) **else**
(9) $\delta_1 := \min\{y_v : v \text{ outer}\}$
(10) $\delta_2 := \min\{\pi_{uv} : u \text{ outer}, v \text{ unlabeled}\}$
(11) $\delta_3 := \min\{\pi_{uv}/2 : u, v \text{ outer, in different blossoms}\}$
(12) $\delta_4 := \min\{z_B/2 : B \text{ inner blossom}\}$
(13) $\delta := \min(\delta_1, \delta_2, \delta_3, \delta_4)$
(14) $y_v := y_v - \delta$ for outer v; $y_v := y_v + \delta$ for inner v
(15) $z_B := z_B + 2\delta$ for outer blossoms; $z_B := z_B - 2\delta$ for inner blossoms
(16) **if** $\delta = \delta_1$ **then stop [comentário:** M **é ótimo]**
(17) expand every inner blossom B with $z_B = 0$
(18) **end**
(19) **end**
(20) **return** M, after expanding all remaining blossoms

O Algoritmo 24 alterna entre o estágio primal e o estágio dual descritos acima, e pode ser lido linha a linha da seguinte forma:

- **Linha 1:** O emparelhamento começa vazio, todos os vértices recebem o mesmo dual y_v, igual à metade do maior peso, e nenhuma blossom tem dual. Com essa escolha, $y_u + y_v \geq w_{uv}$ para toda aresta, ou seja, todas as folgas π_{uv} começam não negativas, e a solução dual é viável desde o início.

- **Linhas 2–3:** A cada iteração, cresce-se uma floresta de árvores alternantes, enraizadas nos vértices livres, usando apenas arestas justas ($\pi_{uv} = 0$). Os vértices a distância par da raiz são rotulados como externos (S) e os a distância ímpar, como internos (T), como na busca do caso de cardinalidade.

- **Linhas 4–5:** Uma aresta justa entre vértices externos de árvores diferentes liga duas raízes livres por um caminho alternante, isto é, um caminho aumentante. O emparelhamento é aumentado ao longo dele, e os rótulos são apagados para uma nova busca.

- **Linhas 6–7:** Uma aresta justa entre dois vértices externos da mesma árvore fecha um ciclo ímpar, que é contraído em uma blossom, como no Algoritmo 20. A nova blossom começa com dual $z_B = 0$ e se comporta como um vértice externo.

- **Linhas 9–13:** Se nenhuma das situações anteriores ocorre, a busca está travada e é preciso ajustar os duais. Calcula-se o maior passo δ que mantém a solução dual viável, como o menor de quatro limites: δ_1 é o menor dual de vértice externo, que não pode ficar negativo; δ_2 é a menor folga de aresta entre um vértice externo e um não rotulado, que diminui em δ; δ_3 é a metade da menor folga entre vértices externos de blossoms diferentes, pois essa folga diminui em 2δ, uma vez que os dois extremos são externos; e δ_4 é a metade do menor dual de blossom interna, que diminui em 2δ e também não pode ficar negativo.

- **Linhas 14–15:** Os duais são atualizados: os vértices externos perdem δ e os internos ganham δ, de modo que as arestas justas da árvore (entre um externo e um interno) continuam justas; as blossoms externas ganham 2δ e as internas perdem 2δ, o que compensa a variação dos duais dos vértices dentro delas.

- **Linha 16:** Se o passo foi limitado por δ_1, algum vértice externo chegou a dual zero. Como os vértices livres são raízes, e portanto externos, e todos os externos perdem o mesmo δ a cada ajuste, os vértices livres têm o menor dual e chegam a zero; nesse ponto, as condições de folga complementar valem e M é ótimo.

- **Linha 17:** Uma blossom interna cujo dual chegou a zero (limite δ_4) é expandida, devolvendo seus vértices à busca. Nos casos δ_2 e δ_3, o ajuste torna justa uma nova aresta, e a busca da próxima iteração consegue crescer a árvore, contrair uma blossom ou aumentar o emparelhamento.

- **Linha 20:** Ao final, as blossoms que restaram contraídas são expandidas para que o emparelhamento seja expresso em termos das arestas do grafo original.

- **Complexidade:** Há no máximo $V/2$ aumentos. Entre dois aumentos consecutivos, cada ajuste dual rotula um novo vértice, cria uma blossom, expande uma blossom ou encerra o algoritmo, de modo que ocorrem $O(V)$ ajustes por aumento. Calculando δ diretamente pelas definições das linhas 9–12, cada ajuste custa $O(E)$, o que dá $O(V^2E)$ no total. A implementação de Lawler (1976) mantém, para cada vértice fora da árvore, a menor folga até ela (*slack*), o que reduz o cálculo de δ a $O(V)$ e leva ao total de $O(V^3)$.

4.2.4 Melhorias

Desenvolvimentos posteriores por Galil, Micali e Gabow (1986) e Gabow (1990) melhoraram a estrutura de dados para gerenciar a prioridade das atualizações duais e a contração de blossoms, atingindo complexidades próximas a $O(EV \log V)$ ou $O(E\sqrt{V\alpha(E,V)})$ em implementações de escalonamento (SCHRIJVER, 2004).

Apesar de ser polinomial (classe P), a implementação de emparelhamento ponderado em grafos gerais é considerada uma das rotinas mais complexas da otimização combinatória clássica, sendo significativamente mais difícil de implementar corretamente do que algoritmos para grafos bipartidos ou fluxo máximo.

4.2.5 Algoritmos aproximados

Aqui também podemos utilizar o método guloso ordenado, descrito na seção 3.2.2.1. No problema de peso máximo, ao contrário da atribuição de custo mínimo, ele tem garantia de aproximação: o emparelhamento obtido tem pelo menos metade do peso máximo, pois cada aresta da solução ótima que fica de fora é bloqueada por uma aresta escolhida antes e, portanto, de peso maior ou igual, e cada aresta escolhida bloqueia no máximo duas arestas da solução ótima (DRAKE; HOUGARDY, 2003). Além dele, temos:

4.2.5.1 Algoritmo de Crescimento de Caminho (Path Growing)

Similar ao algoritmo descrito na seção 4.1.3.1, esta também faz uso de Path Growing para encontrar um emparelhamento. A vantagem desta é que não é necessário ordenação, tornando possível a execução em tempo linear ($O(E)$). Foi descrito por Drake e Hougardy (2003):

1. O algoritmo constrói caminhos simples disjuntos no grafo.
2. Comece em um vértice livre v qualquer. Estenda o caminho escolhendo a aresta de maior peso conectada a v que leve a um vértice ainda não visitado neste caminho.
3. Repita até que o caminho não possa crescer mais.
4. Dado um caminho $P = (e_1, e_2, e_3, \ldots, e_k)$, temos dois emparelhamentos possíveis formados por arestas alternadas:
5.
   - $M_1 = \{e_1, e_3, e_5, \ldots\}$ (Arestas ímpares)
   - $M_2 = \{e_2, e_4, e_6, \ldots\}$ (Arestas pares)
6. Compare o peso total de M_1 e M_2. Adicione o conjunto mais pesado à solução final e remova os vértices do grafo.

A diferença para a versão de emparelhamento de cardinalidade máxima é o passo 5, onde os pesos são somados para escolher o melhor conjunto. Garante emparelhamento de pelo menos $1/2$ do peso máximo (DRAKE; HOUGARDY, 2003).

O Algoritmo 25 corresponde à versão original de Drake e Hougardy (2003), que é a implementada neste trabalho. Nela, os conjuntos M_1 e M_2 são mantidos ao longo de todos os caminhos e o mais pesado é retornado ao final, em vez de a escolha ser feita caminho a caminho como na descrição acima; ambas as versões têm a mesma garantia de aproximação.

Algoritmo 25: Algoritmo Path Growing (DRAKE; HOUGARDY, 2003)

Input: graph $G = (V, E)$ with weights $w : E \to \mathbb{R}$
Output: matching M with $w(M) \geq \frac{1}{2} w(M^*)$

(1) $M_1 := \emptyset$; $M_2 := \emptyset$; $i := 1$
(2) **while** $E \neq \emptyset$ **do**
(3) | choose a vertex x with $\deg(x) \geq 1$
(4) | **while** x *has a neighbor* **do**
(5) | | let $\{x, y\}$ be the heaviest edge incident to x
(6) | | $M_i := M_i \cup \{\{x, y\}\}$; $i := 3 - i$
(7) | | remove x and all its incident edges from G
(8) | | $x := y$
(9) | **end**
(10) **end**
(11) **return** the heavier of M_1 and M_2

O Algoritmo 25 pode ser lido linha a linha da seguinte forma:

- **Linha 1:** Os dois emparelhamentos candidatos começam vazios, e o índice i indica em qual deles entra a próxima aresta.
- **Linhas 2–3:** Enquanto restarem arestas no grafo, inicia-se um novo caminho a partir de qualquer vértice que ainda tenha vizinhos.
- **Linhas 4–5:** O caminho cresce a partir do vértice atual x pela aresta mais pesada incidente a ele, que leva ao vizinho y.
- **Linha 6:** A aresta escolhida entra em M_i, e o índice alterna entre 1 e 2 ($i := 3 - i$). Assim, as arestas do caminho são distribuídas alternadamente entre M_1 e M_2; como duas arestas consecutivas do caminho vão para conjuntos diferentes, cada um deles continua sendo um emparelhamento.
- **Linhas 7–8:** O vértice x é removido do grafo com todas as suas arestas, o que impede que o caminho volte a ele, e o caminho continua a partir de y. Quando y não tem mais vizinhos, o laço interno termina e um novo caminho é iniciado na linha 3.

- **Linha 11:** Retorna-se o mais pesado dos dois emparelhamentos. Para a garantia de aproximação, cada aresta do grafo é removida junto com algum vértice x na linha 7, e a aresta escolhida nesse passo é a mais pesada entre as que são removidas com ele. Como cada vértice é removido uma única vez e cada aresta da solução ótima M^* pode ser atribuída a um extremo distinto removido, $w(M_1) + w(M_2) \geq w(M^*)$, e o mais pesado dos dois tem pelo menos metade do peso ótimo (DRAKE; HOUGARDY, 2003).
- **Complexidade:** Cada vértice é removido uma vez, e a escolha da aresta mais pesada percorre a lista de adjacência do vértice atual, de modo que cada aresta é examinada um número constante de vezes. Mantendo uma lista dos vértices com grau positivo para a linha 3, o custo total é $O(V + E)$, linear no tamanho do grafo, sem a ordenação $O(E \log E)$ do método guloso ordenado.

4.3 EMPARELHAMENTO INDUZIDO MÁXIMO

Diferentemente dos problemas de emparelhamento em grafos gerais vistos até o momento, que pertencem à classe de complexidade P, o problema do Emparelhamento Induzido Máximo (MIM - *Maximum Induced Matching*), ou emparelhamento induzido de cardinalidade máxima, frequentemente referido na literatura como **Emparelhamento Forte** (*Strong Matching*), pertence à classe dos problemas NP-difíceis. Apesar de ter uma variação estruturalmente simples quando comparado aos problemas clássicos, esse problema impõe uma restrição topológica severa sobre quais arestas podem coexistir na solução.

4.3.1 Propriedades

Seja $G = (V, E)$ um grafo geral. Um subconjunto de arestas $M \subseteq E$ é um **emparelhamento induzido** se satisfaz duas condições:

1. **Condição de Emparelhamento:** Nenhuma aresta em M compartilha um vértice (são disjuntas par a par).
2. **Condição Induzida:** O subgrafo induzido pelos vértices saturados por M, denotado por $G[V(M)]$, contém **apenas** as arestas de M.

Em termos mais simples, se tomarmos duas arestas quaisquer $\{u, v\}$ e $\{x, y\}$ pertencentes ao emparelhamento induzido, não pode existir no grafo original nenhuma aresta conectando um vértice do par $\{u, v\}$ a um vértice do par $\{x, y\}$. Isso implica que as arestas do emparelhamento devem estar "topologicamente isoladas"umas das outras.

Tabela 5 – Algoritmos para problema de emparelhamento ponderado máximo em grafos gerais

Algoritmo	**Complexidade**	**Tipo**	**Avaliado**	**Descrição**
Edmonds	$O(V^3)$	Exato	Sim	Método primal-dual que gerencia variáveis duais para vértices e blossoms, ajustando preços para encontrar caminhos no grafo de igualdade.
Edmonds otimizado	$O(EV \log V)$	Exato	Não	Melhoria do Edmonds original através de uma série de otimizações
Guloso ordenado	$O(E \log E)$	Aproximado	Não	Ordena as arestas por peso decrescente e seleciona iterativamente as disponíveis.
Crescimento de caminho	$O(E)$	Aproximado	Sim	Método que estende caminhos pela aresta mais pesada local e seleciona o subconjunto de arestas (par ou ímpar) de maior soma total.

Avaliado: incluído no ambiente de testes; a origem de cada implementação consta na Tabela 10.

Cameron (1989) oferece uma caracterização alternativa baseada em distância: um emparelhamento induzido é um conjunto de arestas onde a distância entre quaisquer duas arestas distintas é de pelo menos 2 (ou seja, não há aresta de ligação entre elas).

4.3.2 Complexidade computacional

Diferentemente do emparelhamento clássico (cardinalidade ou ponderado), que possui algoritmos polinomiais eficientes, o problema de encontrar o emparelhamento induzido de cardinalidade máxima é **NP-Difícil**.

Este resultado foi estabelecido no trabalho fundamental de Stockmeyer e Vazirani (1982). Eles demonstraram que o problema permanece NP-Difícil mesmo quando restrito a grafos bipartidos com grau máximo 3. Isso é um fato surpreendente e crucial: a propriedade de ser bipartido, que geralmente torna problemas de emparelhamento fáceis (como visto no Teorema de König), **não** ajuda no caso do emparelhamento induzido.

4.3.3 Redução ao Problema do Conjunto Independente

A intratabilidade do MIM pode ser explicada através de uma transformação de grafos. O problema de encontrar um emparelhamento induzido em G é equivalente a encontrar um **Conjunto Independente Máximo** (Maximum Independent Set - MIS) em um grafo transformado.

A transformação ocorre em dois passos:

1. Construímos o **Grafo de Linha** $L(G)$, onde cada vértice representa uma aresta de G e dois vértices são adjacentes se as arestas correspondentes em G compartilham uma ponta. (Emparelhamento clássico em $G \equiv$ Conjunto Independente em $L(G)$).
2. Construímos o **Quadrado do Grafo de Linha** $L(G)^2$. Neste grafo, conectamos dois vértices se a distância entre eles em $L(G)$ for 1 ou 2.

Encontrar um emparelhamento induzido máximo em G equivale exatamente a encontrar um Conjunto Independente Máximo em $L(G)^2$ (CAMERON, 1989). Como o problema do Conjunto Independente é NP-Difícil para grafos gerais (GAREY; JOHNSON, 1990), essa relação estrutural justifica a dificuldade do MIM.

4.3.4 Abordagens de solução e heurísticas

Dada a natureza NP-Difícil do problema e a ineficácia de métodos exatos para grafos densos ou de grande escala, a literatura concentra-se majoritariamente em

algoritmos construtivos gulosos e metaheurísticas.

4.3.4.1 Algoritmo exato

Para instâncias pequenas, a redução acima permite resolver o MIM de forma exata. Constrói-se o grafo de conflitos $H = L(G)^2$ e busca-se nele um conjunto independente máximo por *branch-and-bound*: a cada passo, escolhe-se uma aresta candidata e exploram-se dois ramos, incluí-la ou não na solução. Um ramo é podado quando, mesmo incluindo todas as candidatas restantes, não superaria a melhor solução já encontrada. É a abordagem usada neste trabalho como referência exata (Algoritmo 26), viável apenas para algumas dezenas de vértices.

Algoritmo 26: Emparelhamento induzido máximo exato, via conjunto independente máximo em $L(G)^2$ (CAMERON, 1989)

Input: graph $G = (V, E)$
Output: maximum induced matching M^*
(1) build $H = L(G)^2$: one vertex per edge of G; $e \sim f$ in H iff e and f share a vertex or are joined by an edge of G
(2) $M^* := \emptyset$
(3) BRANCH$(\emptyset, E)$
(4) **return** M^*

(5) **Procedure** BRANCH(M, C)
(6) | **[comentário: M: solução atual; C: arestas candidatas sem conflito com M]**
(7) | **if** $|M| + |C| \leq |M^*|$ **then return [comentário: poda]**
(8) | **if** $C = \emptyset$ **then**
(9) | | $M^* := M$
(10) | | **return**
(11) | **end**
(12) | choose $e \in C$
(13) | BRANCH$(M \cup \{e\}, C \setminus (N_H(e) \cup \{e\}))$ **[comentário: inclui e]**
(14) | BRANCH$(M, C \setminus \{e\})$ **[comentário: exclui e]**

O Algoritmo 26 pode ser lido linha a linha da seguinte forma:

- **Linha 1:** Constrói-se o grafo de conflitos $H = L(G)^2$, com um vértice para cada aresta de G e uma aresta entre duas arestas de G que não podem estar juntas em um emparelhamento induzido: as que compartilham um vértice ou as que são ligadas por uma terceira aresta. Um emparelhamento induzido de G é exatamente um conjunto independente de H.
- **Linhas 2–4:** A melhor solução conhecida M^* começa vazia, e a busca parte de uma solução vazia com todas as arestas como candidatas. Ao final, M^* contém o emparelhamento induzido máximo.

- **Linha 7:** Poda (*bound*): o procedimento BRANCH recebe a solução parcial M e o conjunto C das arestas que ainda podem ser adicionadas a ela sem conflito. Mesmo que todas as candidatas fossem incluídas, a solução teria no máximo $|M| + |C|$ arestas; se isso não supera a melhor solução já encontrada, o ramo é abandonado.
- **Linhas 8–9:** Sem candidatas restantes, a solução parcial está completa e, como passou pela poda, é maior que M^*, que é então substituída.
- **Linhas 12–14:** Ramificação (*branch*): escolhe-se uma candidata e e exploram-se os dois casos possíveis. No primeiro, e entra na solução, e saem das candidatas e e todos os seus vizinhos em H, que passariam a conflitar com ela. No segundo, e é descartada e as demais candidatas são mantidas. Todo emparelhamento induzido cai em exatamente um dos dois ramos, o que garante que o ótimo é encontrado.
- **Complexidade:** A construção de H compara pares de arestas, com custo $O(E^2)$. A árvore de busca tem profundidade no máximo E, pois cada chamada remove ao menos uma candidata, e até 2^E folhas no pior caso; cada chamada faz operações de conjunto de custo $O(E)$. O total é exponencial, $O(E \cdot 2^E)$ no pior caso, o que é esperado para um problema NP-difícil; a poda reduz bastante a busca na prática, mas não altera esse limite, e o algoritmo só é viável em instâncias pequenas.

4.3.4.2 Heurísticas gulosas

Dada a equivalência entre o problema do Emparelhamento Induzido Máximo em G e o Conjunto Independente Máximo no grafo transformado $L(G)^2$, as heurísticas gulosas clássicas da literatura de conjuntos independentes são naturalmente adaptadas para este contexto.

O procedimento geral consiste em selecionar iterativamente uma aresta $e \in E$, adicioná-la à solução e remover sua vizinhança induzida do grafo. As variantes diferem no critério de seleção da aresta:

- **Guloso Aleatório (Random Greedy):** Seleciona a próxima aresta uniformemente ao acaso dentre as disponíveis. Duckworth, Wormald e Zito (2002) analisaram o comportamento assintótico desta abordagem em grafos cúbicos aleatórios, demonstrando que, apesar de sua simplicidade, ela fornece uma linha de base (*baseline*) essencial para a avaliação de métodos mais sofisticados.
- **Guloso de Grau Mínimo (Min-Degree Greedy):** A heurística prioriza a seleção da aresta que possui o **menor grau induzido** (ou seja, a aresta cuja remoção implica a eliminação do menor número de outras arestas candidatas). Esta

abordagem é uma adaptação direta da heurística *Min-Degree* (ou *Greedy*) para o problema do Conjunto Independente Máximo. Halldórsson e Radhakrishnan (1994) estabeleceram os fundamentos teóricos para essa estratégia, demonstrando que priorizar vértices de baixo grau fornece garantias de aproximação superiores em grafos esparsos e de grau limitado. No contexto do MIM, essa lógica traduz-se em escolher a aresta $e = \{u, v\}$ que minimiza a soma dos graus de seus extremos ou o tamanho de sua vizinhança em $L(G)^2$.

As duas variantes compartilham a mesma estrutura e diferem apenas no critério de seleção, conforme o Algoritmo 27. Ambas são classificadas como heurísticas (Seção 2.6): as garantias conhecidas para os gulosos de conjunto independente dependem do grau máximo do grafo (HALLDóRSSON; RADHAKRISHNAN, 1994) e, em $L(G)^2$, esse grau cresce com o quadrado do grau de G, de modo que não há razão de aproximação constante para grafos gerais.

Algoritmo 27: Heurísticas gulosas para emparelhamento induzido: aleatória e de grau mínimo (DUCKWORTH; WORMALD; ZITO, 2002; HALLDóRSSON; RADHAKRISHNAN, 1994)

Input: graph $G = (V, E)$; selection rule RANDOM or MINDEGREE
Output: maximal induced matching M

(1) build $H = L(G)^2$
(2) $M := \emptyset$; $C := E$
(3) **while** $C \neq \emptyset$ **do**
(4) | **if** *rule is* RANDOM **then**
(5) | | choose $e \in C$ uniformly at random
(6) | **else**
(7) | | choose $e \in C$ with minimum degree in $H[C]$
(8) | **end**
(9) | $M := M \cup \{e\}$
(10) | $C := C \setminus (N_H(e) \cup \{e\})$
(11) **end**
(12) **return** M

O Algoritmo 27 pode ser lido linha a linha da seguinte forma:

- **Linha 1:** Constrói-se o grafo de conflitos $H = L(G)^2$, o mesmo do Algoritmo 26.
- **Linha 2:** A solução começa vazia, e todas as arestas de G são candidatas.
- **Linhas 3–7:** Enquanto houver candidatas, escolhe-se uma delas segundo a regra da variante: ao acaso, no guloso aleatório (linha 5), ou a de menor grau no subgrafo de H induzido pelas candidatas, no guloso de grau mínimo (linha 7). O grau de e em $H[C]$ é o número de candidatas que deixarão de estar disponíveis se e for escolhida; minimizá-lo preserva o maior número possível de opções para os passos seguintes.

- **Linhas 9–10:** A aresta escolhida entra na solução e, junto com todos os seus vizinhos em H, deixa de ser candidata, o que mantém M um emparelhamento induzido a cada passo.
- **Linha 12:** Quando não há mais candidatas, nenhuma aresta pode ser adicionada a M sem violar a restrição induzida: o resultado é um emparelhamento induzido maximal, mas não necessariamente máximo.
- **Complexidade:** A construção de H custa $O(E^2)$, e H tem $O(E^2)$ arestas no pior caso. O laço executa no máximo E iterações, e cada aresta de H é percorrida um número constante de vezes ao remover os vizinhos das escolhidas, com custo total $O(E^2)$. No guloso aleatório, cada escolha custa $O(1)$; no de grau mínimo, mantendo os graus em $H[C]$ atualizados a cada remoção, a escolha do mínimo custa $O(E)$ por iteração, também $O(E^2)$ no total. As duas variantes têm, portanto, complexidade $O(E^2)$, dominada pelo grafo de conflitos, como indicado na Tabela 6.

4.3.4.3 Metaheurísticas

Para superar os ótimos locais frequentemente encontrados pelos métodos gulosos, abordagens iterativas de busca local têm sido aplicadas.

Fürst, Leichter e Rautenbach (2017) investigaram formalmente o poder da busca local para o Emparelhamento Induzido Máximo. A estratégia analisada consiste em iniciar com um emparelhamento induzido qualquer (ou vazio) e tentar aumentá-lo iterativamente através de melhorias locais simples. Eles focaram na análise de grafos k-regulares e demonstraram que, embora a busca local possa ficar presa em ótimos locais longe da solução global em grafos gerais, ela oferece garantias de desempenho interessantes para classes específicas (como grafos livres de garras ou C_4-free). O algoritmo de busca local proposto tenta adicionar arestas que não violem a restrição induzida e, se necessário, remover um número pequeno de arestas existentes para acomodar a nova inclusão, um processo análogo às trocas (*k-swaps*) em outros problemas combinatórios.

Tabela 6 – Algoritmos para problema de emparelhamento induzido máximo em grafos gerais

Algoritmo	**Complexidade**	**Tipo**	**Avaliado**	**Descrição**
Guloso Aleatório	$O(E^2)$*	Heurística	Sim	Seleciona arestas uniformemente ao acaso e remove suas vizinhanças induzidas.
Guloso de grau mínimo	$O(E^2)$*	Heurística	Sim	Seleciona a aresta cuja remoção elimina o menor número de arestas vizinhas.
Exato (conjunto independente)	exponencial	Exato	Sim	Busca por conjunto independente máximo em $L(G)^2$ via *branch-and-bound*; viável apenas em instâncias pequenas.

Avaliado: incluído no ambiente de testes; a origem de cada implementação consta na Tabela 10. * Complexidade das implementações avaliadas, que constroem explicitamente o grafo de conflitos $L(G)^2$.

5 METODOLOGIA

Este capítulo descreve o ambiente de testes construído para implementar e comparar os algoritmos dos capítulos anteriores. Todo o código do ambiente, os geradores de instâncias e os resultados brutos estão disponíveis no repositório do trabalho[1]. São apresentados a arquitetura do ambiente, a interface padrão de cada problema, os geradores de instância, o protocolo de medição, os critérios de comparação, a máquina utilizada e a origem e a adaptação de cada algoritmo, de modo que os resultados do capítulo seguinte sejam reprodutíveis e as comparações, defensáveis.

5.1 ARQUITETURA DO AMBIENTE DE TESTES

O ambiente é separado em três camadas:

- **Repositório de código de terceiros:** Contém os códigos de algoritmos obtidos de repositórios públicos, em sua maioria do GitHub. Estes códigos estão intactos, exatamente da forma que estão publicados. Nenhuma alteração foi feita a eles, nem mesmo com intuito de corrigir possíveis problemas, os quais serão mencionados posteriormente. As bibliotecas maduras usadas como referência (NetworkX, SciPy, igraph e lap) não ficam nesta camada: são instaladas pelo gerenciador de pacotes do Python, com versões fixadas.
- **Adaptadores/*wrappers*:** Como os algoritmos divergem quanto aos formatos de entrada e saída, e até mesmo quanto à linguagem, essa camada foi criada com o intuito de traduzir cada um dos algoritmos para o formato padrão do trabalho, a fim de manter o ambiente homogêneo para fim de testes. Quando o código original não pode ser usado diretamente, a reimplementação também fica nesta camada, de modo que o original permanece intacto no repositório.
- **Módulo de medição:** É a parte que executa e mede os testes, responsável por cronometragem, verificação e gravação dos resultados. Para cada problema, um *script* de *benchmark* gera as instâncias e aciona este módulo, que executa todos os algoritmos por meio dos seus *wrappers*. Isso garante que todos sejam submetidos ao mesmo protocolo, e que possam ser feitas comparações entre algoritmos sob condições idênticas.

[1] <https://github.com/talisbreda/TCC>

5.2 INTERFACE PADRÃO DOS PROBLEMAS

Cada problema possui uma interface padrão à qual todos os seus algoritmos são adaptados (Tabela 7). É ela que torna a comparação justa: todo algoritmo de um mesmo problema recebe exatamente a mesma instância e produz a saída no mesmo formato, de modo que a única variável entre eles é o próprio método.

Tabela 7 – Interface padrão de entrada e saída por problema

Problema	**Entrada**	**Saída**
Cardinalidade máxima (bipartido)	vértices de cada lado e lista de arestas	tamanho e emparelhamento
Atribuição linear	matriz de custos $n \times n$	custo total e atribuição
Atribuição gargalo	matriz de custos $n \times n$	custo gargalo e atribuição
Casamento estável	listas de preferência de ambos os lados	emparelhamento
Cardinalidade máxima (geral)	número de vértices e lista de arestas	tamanho e emparelhamento
Peso máximo (geral)	número de vértices e arestas com peso	peso total e emparelhamento
Emparelhamento induzido máximo	número de vértices e lista de arestas	tamanho e emparelhamento

5.3 GERADORES DE INSTÂNCIA

As instâncias são produzidas por construtores determinísticos, um por família, parametrizados pelo tamanho n. Cada construtor que usa aleatoriedade cria, a cada chamada, um gerador pseudoaleatório próprio, iniciado com a semente fixa 42 (`random.Random(42)`, do Python). Como esse gerador não é compartilhado com o restante do programa, uma mesma chamada produz sempre a mesma instância, independentemente da ordem em que os algoritmos são executados. Além disso, a instância de cada tamanho é construída uma única vez e entregue a todos os algoritmos da célula, de modo que todos resolvem exatamente a mesma entrada. Instâncias de tamanhos diferentes partem da mesma semente, mas não são aninhadas: a de tamanho 100 não contém a de tamanho 50.

Para cada problema, definiram-se famílias com características distintas, de modo a expor o comportamento dos algoritmos sob diferentes estruturas de entrada. Sempre que possível, a família foi construída de forma que o valor ótimo seja conhecido de antemão, o que permite conferir as soluções sem depender de outro algoritmo. A Tabela 8 resume as famílias, e os geradores são descritos a seguir.

Tabela 8 – Famílias de instâncias por problema

Família	**Construção e propriedade**	**Tamanhos n**
Cardinalidade máxima — grafos bipartidos (Algoritmo 28)		
Grafo 1 — esparso, sem perfeito	dois blocos fixos, grau médio 2,2; $\lvert M^*\rvert = 0{,}8n$	50–250
Grafo 2 — esparso, com perfeito	diagonal mais um vizinho $(7i + 3) \bmod n$; $\lvert M^*\rvert = n$	50–250
Grafo 3 — denso, com perfeito	diagonal mais arestas aleatórias até grau médio 15; $\lvert M^*\rvert = n$	50–250
Atribuição linear (Algoritmo 29)		
Matriz 1 — densa uniforme	custos inteiros uniformes em $[1, 100]$	10–100; 50–250[a]
Matriz 2 — permutação barata	permutação aleatória de custo 1, demais em $[50, 100]$; ótimo $= n$	idem
Matriz 3 — estruturada	$c_{ij} = \lvert i - j\rvert + 1$; ótimo $= n$	idem
Atribuição gargalo (Algoritmo 29)		
Matriz 1 — densa uniforme	custos inteiros uniformes em $[1, 100]$	10–200
Matriz 2 — estruturada	$c_{ij} = \lvert i - j\rvert + 1$; gargalo ótimo $= 1$	10–200
Matriz 3 — poucos valores	custos sorteados em $\{1, 5, 10, 50\}$; muitos empates	10–200
Casamento estável (Algoritmo 30)		
Perfil 1 — aleatório	listas independentes e uniformes	25–100; 100–400[b]
Perfil 2 — correlacionado	uma única lista para todos os homens e outra para todas as mulheres	idem
Perfil 3 — cíclico	m prefere $m, m + 1, \ldots$; w prefere $w, w - 1, \ldots$ (módulo n)	idem
Grafos gerais: cardinalidade e peso máximo (Algoritmo 31)		
Esparso (grau $\sim$3)	$G(n, p)$ com $p = 3/n$; na versão ponderada, pesos em $[1, 100]$	10–100
Denso ($p = 0{,}3$)	$G(n, p)$ com $p = 0{,}3$; idem	10–100
Emparelhamento induzido máximo (Algoritmo 31)		
Esparso, médio e denso	$G(n, p)$ com grau médio esperado 3, 6 e 12	10–40; 50–400[c]

Nota: (a) 10 a 100 com todos os algoritmos e 50 a 250 com todos exceto os caminhos mínimos sucessivos; (b) 25 a 100 com todos e 100 a 400 apenas com o Gale-Shapley numérico e o de Lattas; (c) 10 a 40 com o algoritmo exato e 50 a 400 apenas com as heurísticas. n é o número de vértices de cada lado (bipartidos e matrizes $n \times n$), o número de homens e de mulheres (casamento estável) ou o número de vértices (grafos gerais).

5.3.1 Grafos bipartidos

Para o emparelhamento de cardinalidade máxima, as três famílias diferem na densidade e na existência de emparelhamento perfeito, e o valor ótimo de todas é conhecido por construção (Algoritmo 28). Os vértices são $u_0, \ldots, u_{n-1}$ do lado U e $v_0, \ldots, v_{n-1}$ do lado V.

Algoritmo 28: Geradores de grafos bipartidos para cardinalidade máxima

```
(1)  Procedure Grafo1(n)
(2)      a_U := ⌊0,8n⌋; a_V := ⌊0,6n⌋; b_U := n − a_U; b_V := n − a_V
(3)      E := ∅
(4)      for i := 0 to a_U − 1 do
(5)          E := E ∪ {{u_i, v_{i mod a_V}}, {u_i, v_{(i+1) mod a_V}}}
(6)      end
(7)      for k := 0 to b_U − 1 do
(8)          for o := 0 to 2 do
(9)              E := E ∪ {{u_{a_U+k}, v_{a_V+(2k+o) mod b_V}}}
(10)         end
(11)     end
(12)     return E

(13) Procedure Grafo2(n)
(14)     E := ∅
(15)     for i := 0 to n − 1 do
(16)         j := (7i + 3) mod n
(17)         if j = i then j := (j + 1) mod n
(18)         E := E ∪ {{u_i, v_i}, {u_i, v_j}}
(19)     end
(20)     return E

(21) Procedure Grafo3(n, d = 15)
(22)     rng := new pseudo-random generator with seed 42
(23)     E := {{u_i, v_i} : 0 ≤ i < n}
(24)     while |E| < d · n do
(25)         draw i and j uniformly from {0, ..., n − 1} using rng
(26)         E := E ∪ {{u_i, v_j}} [comentário: arestas repetidas são
              descartadas]
(27)     end
(28)     return E
```

- **Grafo 1 — esparso, sem emparelhamento perfeito (linhas 2–9):** O grafo é a união de dois blocos sem arestas entre si. No bloco A, os primeiros $0{,}8n$ vértices de U ligam-se, cada um, a dois vértices consecutivos dos primeiros $0{,}6n$ vértices de V (linha 4). Como o bloco tem só $0{,}6n$ vértices em V, no máximo $0{,}6n$ de suas arestas podem estar num emparelhamento, e esse valor é atingido pelas arestas $\{u_i, v_i\}$, $i < a_V$. No bloco B, os $0{,}2n$ vértices restantes de U ligam-se a três dos $0{,}4n$ vértices restantes de V (linhas 7–9); como $2k < b_V$ para todo $k < b_U$, as arestas $\{u_{a_U+k}, v_{a_V+2k}\}$ cobrem todo o lado U do bloco. O emparelhamento máximo tem, portanto, exatamente $0{,}6n + 0{,}2n = 0{,}8n$ arestas, e nenhum é perfeito. O grau médio dos vértices de U é $(2 \cdot 0{,}8n + 3 \cdot 0{,}2n)/n = 2{,}2$. A construção não usa aleatoriedade.

- **Grafo 2 — esparso, com emparelhamento perfeito (linhas 15–18):** Cada

u_i liga-se a v_i e a um segundo vizinho v_j, com $j = (7i + 3) \bmod n$ (linha 16), deslocado em uma posição quando coincidiria com i. As arestas $\{u_i, v_i\}$ formam um emparelhamento perfeito, logo $|M^*| = n$ (linha 18). Como 7 não divide nenhum dos tamanhos usados, $i \mapsto (7i + 3) \bmod n$ é uma permutação, e os segundos vizinhos ficam espalhados por todo o lado V, sem a estrutura em blocos do Grafo 1. A construção também é determinística.

- **Grafo 3 — denso, com emparelhamento perfeito (linhas 22–26):** Parte da diagonal $\{u_i, v_i\}$, que garante um emparelhamento perfeito (linha 23), e acrescenta arestas sorteadas pelo gerador com semente fixa (linha 22) até o grafo ter $d \cdot n = 15n$ arestas (linha 24), isto é, grau médio 15. Sorteios repetidos não alteram o conjunto (linha 26). A densidade relativa é $15/n$, de 30% em $n = 50$ a 6% em $n = 250$; a família é "densa" em comparação com as duas anteriores, cujo grau médio fica entre 2 e 2,2.

- **Complexidade:** Os Grafos 1 e 2 são gerados em tempo $O(n)$, com um número constante de arestas por vértice. No Grafo 3, a fração de sorteios repetidos é no máximo d/n, de modo que o número esperado de sorteios é $O(d \cdot n)$.

5.3.2 Matrizes de custo

Os problemas de atribuição linear e de gargalo recebem uma matriz de custos C de dimensão $n \times n$, gerada por um dos quatro procedimentos do Algoritmo 29. A atribuição linear usa as famílias uniforme, permutação barata e estruturada; o gargalo usa as famílias uniforme, estruturada e de poucos valores.

Algoritmo 29: Geradores de matrizes de custo para atribuição linear e gargalo

```
(1)  Procedure Uniforme(n)
(2)      rng := new pseudo-random generator with seed 42
(3)      for i, j := 0 to n − 1 do c_ij := integer drawn uniformly from [1, 100]
(4)      return C

(5)  Procedure PermutacaoBarata(n)
(6)      rng := new pseudo-random generator with seed 42
(7)      π := random permutation of {0, ..., n − 1} using rng
(8)      for i, j := 0 to n − 1 do c_ij := integer drawn uniformly from [50, 100]
(9)      for i := 0 to n − 1 do c_iπ(i) := 1
(10)     return C

(11) Procedure Estruturada(n)
(12)     for i, j := 0 to n − 1 do c_ij := |i − j| + 1
(13)     return C

(14) Procedure PoucosValores(n)
(15)     rng := new pseudo-random generator with seed 42
(16)     for i, j := 0 to n − 1 do c_ij := value drawn uniformly from {1, 5, 10, 50}
(17)     return C
```

- **Matriz uniforme (linhas 2–3):** Cada custo é um inteiro sorteado de modo uniforme e independente em $[1, 100]$. É a família sem estrutura, em que o ótimo não é conhecido de antemão e é obtido pelo algoritmo de referência.
- **Permutação barata (linhas 7–9):** Sorteia-se uma permutação π (linha 7), todos os custos são preenchidos com valores entre 50 e 100 (linha 8) e, por fim, as posições $(i, \pi(i))$ recebem custo 1 (linha 9). A atribuição π custa n, e qualquer outra usa ao menos uma posição de custo maior ou igual a 50, custando no mínimo $(n-1) + 50 > n$. Logo, π é a única atribuição ótima e o valor ótimo n é conhecido por construção.
- **Matriz estruturada (linha 12):** O custo cresce com a distância à diagonal, $c_{ij} = |i - j| + 1$, sem aleatoriedade. A diagonal tem custo 1 em todas as linhas, de modo que a identidade é ótima tanto para a atribuição linear (custo n) quanto para o gargalo (custo máximo 1).
- **Poucos valores (linha 16):** Cada custo é sorteado entre apenas quatro valores. Com isso, há muitos empates: muitas atribuições têm o mesmo gargalo, e a busca pelo limiar do algoritmo Threshold (Algoritmo 15) tem só quatro candidatos distintos.
- **Complexidade:** Todos os procedimentos preenchem n^2 posições em tempo constante cada, totalizando $O(n^2)$; o sorteio da permutação custa $O(n)$.

5.3.3 Perfis de preferência

No casamento estável, uma instância é formada pelas listas de preferência P_m de cada homem m (uma ordenação das mulheres) e Q_w de cada mulher w (uma ordenação dos homens). As três famílias do Algoritmo 30 variam o grau de concordância entre as listas.

Algoritmo 30: Geradores de perfis de preferência para o casamento estável

(1) **Procedure** *Aleatorio(n)*
(2) $rng :=$ new pseudo-random generator with seed 42
(3) **for** $m := 0$ **to** $n-1$ **do** $P_m :=$ random permutation of $\{0,\dots,n-1\}$ using rng
(4) **for** $w := 0$ **to** $n-1$ **do** $Q_w :=$ random permutation of $\{0,\dots,n-1\}$ using rng
(5) **return** (P, Q)

(6) **Procedure** *Correlacionado(n)*
(7) $rng :=$ new pseudo-random generator with seed 42
(8) $\pi :=$ random permutation of $\{0,\dots,n-1\}$ using rng **[comentário: ranking das mulheres]**
(9) $\sigma :=$ random permutation of $\{0,\dots,n-1\}$ using rng **[comentário: ranking dos homens]**
(10) **for** $m := 0$ **to** $n-1$ **do** $P_m := \pi$
(11) **for** $w := 0$ **to** $n-1$ **do** $Q_w := \sigma$
(12) **return** (P, Q)

(13) **Procedure** *Ciclico(n)*
(14) **for** $m := 0$ **to** $n-1$ **do** $P_m := (m, m+1, \dots, m+n-1) \bmod n$
(15) **for** $w := 0$ **to** $n-1$ **do** $Q_w := (w, w-1, \dots, w-n+1) \bmod n$
(16) **return** (P, Q)

- **Perfil aleatório (linhas 3–4):** Cada lista é uma permutação uniforme e independente das demais. É o caso médio, em que as preferências não têm relação entre si.

- **Perfil correlacionado (linhas 8–11):** Sorteiam-se apenas duas permutações: π, adotada por todos os homens (linha 10), e σ, adotada por todas as mulheres (linha 11). É o extremo da contenção: todos os homens disputam as mesmas mulheres na mesma ordem. No Gale-Shapley, a cada rodada só um homem é aceito pela mulher mais disputada que resta, e o total de propostas é $n + (n-1) + \cdots + 1 = n(n+1)/2$, o pior caso quadrático do algoritmo.

- **Perfil cíclico (linhas 14–15):** As listas são deslocamentos circulares, sem aleatoriedade: o homem m prefere $m, m+1, m+2, \dots$ e a mulher w prefere $w, w-1, w-2, \dots$, sempre módulo n. A primeira escolha de cada homem m

é a mulher m, cuja primeira escolha é ele próprio; o Gale-Shapley termina, assim, com n propostas e devolve o emparelhamento $\{(m, m)\}$, o melhor caso do algoritmo.

- **Complexidade:** Cada procedimento escreve $2n$ listas de tamanho n, em tempo $O(n^2)$.

5.3.4 Grafos gerais aleatórios

Para os problemas em grafos gerais, inclusive o emparelhamento induzido máximo, as instâncias são grafos aleatórios no modelo $G(n, p)$ de Erdős–Rényi, em que cada um dos $\binom{n}{2}$ pares de vértices é ligado de forma independente com probabilidade p (Algoritmo 31).

Algoritmo 31: Gerador de grafos gerais aleatórios no modelo $G(n, p)$

```
(1)  Procedure GrafoAleatorio(n, p, ponderado)
(2)      rng := new pseudo-random generator with seed 42
(3)      E := ∅
(4)      for u := 0 to n − 1 do
(5)          for v := u + 1 to n − 1 do
(6)              if a number drawn uniformly from [0, 1) using rng is < p then
                     E := E ∪ {{u, v}}
(7)          end
(8)      end
(9)      if ponderado then
(10)         rng′ := new pseudo-random generator with seed 43
(11)         for each e ∈ E, in the order of insertion do w(e) := integer drawn
                 uniformly from [1, 100] using rng′
(12)     end
(13)     return E (and w, if ponderado)
```

- **Sorteio das arestas (linhas 2–6):** Os pares $\{u, v\}$ com $u < v$ são percorridos em ordem fixa (linha 4), e cada um vira aresta com probabilidade p (linha 6). O grau esperado de cada vértice é $p(n-1)$. As famílias usam: $p = 3/n$ ("esparso, grau $\sim$3") e $p = 0{,}3$ ("denso") para cardinalidade e peso máximo; e $p = d/(n-1)$, com grau médio esperado d igual a 3, 6 e 12 (grafos esparso, médio e denso), para o emparelhamento induzido máximo.

- **Pesos (linhas 9–11):** Na versão ponderada, os pesos são inteiros uniformes em $[1, 100]$, sorteados por um segundo gerador, com semente 43 (linha 10). Como as arestas continuam sendo sorteadas pelo primeiro gerador, a instância ponderada tem exatamente as mesmas arestas da instância de cardinalidade de mesmo tamanho e densidade; muda apenas a existência dos pesos.

- **Propriedades:** Nenhuma propriedade de emparelhamento é imposta: o grafo pode ter vértices isolados e não ter emparelhamento perfeito. O valor ótimo é obtido pelo algoritmo exato de referência (Blossom, van Rantwijk ou o exato do emparelhamento induzido) ou, nos tamanhos em que o exato não é executado, comparado ao melhor resultado conhecido.
- **Complexidade:** Todos os $\binom{n}{2}$ pares são testados, independentemente de p, o que leva a tempo $\Theta(n^2)$; o sorteio dos pesos acrescenta $O(|E|)$.

5.4 PROTOCOLO DE MEDIÇÃO

Um protocolo de medição foi criado a fim de reduzir o quanto fontes de ruído externas afetam os resultados dos testes. A unidade de medição é a **célula**: a combinação de um algoritmo, uma instância e um tamanho n. Cada célula é executada uma vez para aquecimento e, em seguida, cinco vezes com cronometragem. Algumas das decisões que compõem esse protocolo:

- **Execução de aquecimento:** A primeira execução de cada célula é descartada da cronometragem, para que as repetições medidas não paguem custos de importação, alocação inicial e caches frios. O resultado dessa execução é o utilizado na verificação de qualidade.
- **Coletor de lixo:** É desligado durante a execução de uma célula inteira, e religado entre células, a fim de não comprometer o resultado da execução.
- **Mediana:** Cada célula reporta a mediana dos tempos, ao invés da média, para dar mais robustez com relação a interrupções momentâneas.
- **Limite de tempo por célula:** Impede que um algoritmo defeituoso inviabilize toda a suíte de testes. Fez-se necessário por conta de uma situação observada durante os testes, que será discutida na seção de resultados. O limite, de 120 segundos, é aplicado à execução de aquecimento: se ela o excede, é interrompida, a célula é marcada como estourada e os tamanhos maiores daquele algoritmo não são executados, pois também estourariam. A exceção é o problema de atribuição, em que todos os tamanhos são executados mesmo após um estouro, pois um dos algoritmos falha de forma não-monótona no tamanho da entrada. Por decisão deliberada, o limite é verificado apenas em torno do aquecimento, e não a cada operação do algoritmo, para não adicionar custo à região cronometrada.

5.5 CRITÉRIOS DE COMPARAÇÃO

Comparar tempo de execução entre implementações em linguagens diferentes é intrinsecamente injusto: parte da diferença reflete a linguagem, não o algoritmo. Para que as comparações sejam válidas, adotaram-se quatro critérios complementares.

1. Implementações em linguagem interpretada (ex: Python) e as de núcleo compilado (ex: C++) são apresentadas em tabelas separadas, e o tempo absoluto só é comparado dentro de uma mesma linguagem. Manter as duas linguagens, em vez de uma só, foi necessário por dois motivos. Primeiro, não há implementações públicas confiáveis de todos os algoritmos em uma mesma linguagem, e portá-las significaria reimplementá-las, o que contraria o objetivo de avaliar o código como foi publicado (camada intacta, Seção 5.1). Segundo, as bibliotecas maduras usadas como referência de correção e de patamar de desempenho (igraph, SciPy e lap) têm núcleo compilado. O conjunto principal de comparação é o das implementações interpretadas, cujas tabelas compõem o Capítulo 6; as tabelas das compiladas ficam no Apêndice A, e a comparação entre as duas linguagens é feita apenas pelo expoente empírico (critério 3). Bibliotecas escritas inteiramente em linguagem interpretada, como o NetworkX, ficam na tabela das interpretadas, pois nesse caso a comparação de tempo com elas é legítima. Há ainda casos intermediários: algoritmos escritos como laços em Python sobre vetores da biblioteca NumPy, cujo núcleo é compilado, como os Húngaros `hunalgorithm` e `mayorx` e o Gale-Shapley numérico. Estes são classificados como interpretados, pois o algoritmo em si é executado pelo interpretador.

2. As tabelas mostram complexidade assintótica ao lado do tempo medido, permitindo a comparação do teórico vs real, principalmente entre grafos de diferentes densidades.

3. Para cada algoritmo, é estimado o **expoente empírico do crescimento**. Supondo que o tempo cresça como $t(n) \approx C \cdot n^k$, aplicar o logaritmo resulta em

$$\log t(n) = \log C + k \log n, \tag{5.1}$$

   isto é, uma reta de inclinação k no gráfico do logaritmo do tempo pelo logaritmo do tamanho. O expoente é estimado pela inclinação da reta ajustada por mínimos quadrados aos pontos medidos $(x_i, y_i) = (\log n_i, \log t_i)$:

$$k = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}. \tag{5.2}$$

   Esse dado, ao contrário do tempo absoluto, é comparável entre linguagens: a linguagem altera apenas a constante C, o que desloca a reta verticalmente

sem mudar sua inclinação. Junto ao expoente é reportado o coeficiente de determinação R^2, que indica o quanto os pontos de fato seguem uma reta; um R^2 baixo sinaliza que o crescimento não é bem descrito por uma única potência de n na faixa medida.

4. Em problemas com apenas soluções exatas, a qualidade da solução é idêntica entre todos, sendo o método de comparação o tempo de execução. Já nos que têm soluções aproximadas ou heurísticas, é apresentado o resultado obtido e a razão entre este e o resultado do algoritmo exato de referência (ou melhor resultado conhecido), permitindo calcular a qualidade da solução.

5.6 AMBIENTE DE EXECUÇÃO E REPRODUTIBILIDADE

Todos os resultados foram obtidos numa única máquina (Tabela 9); medir tudo no mesmo ambiente é o que torna os tempos comparáveis. Três fatores exigiram cuidados, dois da máquina e um da linguagem, todos documentados para permitir reprodução.

1. Por ser um **processador híbrido** (núcleos de desempenho e de eficiência, não equivalentes), a execução foi fixada a um núcleo de desempenho, evitando que a migração entre núcleos alterasse o tempo sem mudança no código.

2. Como a **frequência varia** dinamicamente com a carga, o regulador foi fixado no modo de desempenho, com a máquina conectada à energia e sem aplicações concorrentes.

3. Em Python, o *hash* de cadeias de caracteres é aleatorizado a cada processo, o que altera a ordem de iteração de conjuntos e dicionários entre execuções. Por isso, a semente dessa aleatorização foi fixada (variável `PYTHONHASHSEED`), tornando as execuções reprodutíveis. Isso foi necessário pois foi notada em um dos algoritmos uma grande sensibilidade a essa ordem, o que será detalhado no capítulo de resultados.

Tabela 9 – Especificação da máquina de medição

Componente	**Especificação**
Processador	Intel Core 7 150U (10 núcleos físicos, 12 *threads*)
Topologia	híbrida: 2 núcleos de desempenho e 8 de eficiência
Memória	33 GB
Sistema operacional	Ubuntu 24.04.2 LTS (kernel 7.0.0)
Linguagem	Python 3.12.3
Bibliotecas	NumPy 2.5.2, SciPy 1.18.0, NetworkX 3.6.1, igraph 1.0.0, lap 0.5.13

5.7 ORIGEM, ADAPTAÇÃO E VERIFICAÇÃO DOS ALGORITMOS

A maior parte dos algoritmos foi obtida de implementações públicas, em vez de reescrita. Isso é coerente com o objetivo de comparar métodos presentes na literatura, reduz o risco de erros próprios e permite avaliar a qualidade das implementações efetivamente disponíveis. Foram empregadas quatro estratégias de adaptação, conforme natureza do código:

1. Carregamento direto, quando o original é utilizável na linguagem do projeto;
2. Encapsulamento, para métodos disponíveis em bibliotecas maduras;
3. Reimplementação fiel, quando o original não é utilizável diretamente, seja por estar em outra linguagem ou versão, ou por conter efeitos colaterais, seguindo a lógica original com a fonte documentada;
4. Implementação própria a partir da fonte bibliográfica, para os métodos aproximados, as heurísticas e alguns exatos sem implementação pública adequada, algoritmos de descrição curta cuja reimplementação é mais confiável que um repositório de qualidade incerta.

A Tabela 10 registra o tipo e a origem de cada algoritmo, e aponta o pseudocódigo correspondente nos capítulos anteriores.

Tabela 10 – Proveniência dos algoritmos avaliados

Algoritmo	**Tipo**	**Alg.**	**Origem**
Cardinalidade máxima — grafos bipartidos			
Caminhos aumentantes	Exato	4	<wbchristerson/perfect-matchings>
Hopcroft-Karp	Exato	6 e 7	<sofiat-olaosebikan/hopcroftkarp>
Edmonds-Karp (fluxo)	Exato	5	reimpl. de `Maxflow-Algorithms`
Hopcroft-Karp (NetworkX)	Referência	6 e 7	NetworkX (interpretada)
Emparelhamento bipartido (igraph)	Referência	—	igraph (compilada)
Guloso simples	Aproximado	8	própria (CORMEN et al., 2009)
Karp-Sipser	Aproximado	9	própria (KARP; SIPSER, 1981)
Atribuição linear			
Húngaro (munkres)	Exato	10	<bmc/munkres>
Húngaro (benchaplin)	Exato	10	<benchaplin/hungarian-algorithm>
Húngaro (hunalgorithm)	Exato	10	<pbertoni/HungarianAlgorithm>
Húngaro (mayorx)	Exato	10	<mayorx/hungarian-algorithm>
Caminhos mínimos sucessivos	Exato	13	reimpl. a partir de repositório
SciPy	Referência	12	SciPy (compilada)
Jonker-Volgenant	Referência	12	lap (compilada)
Atribuição gargalo			
Threshold	Exato	15	reimpl. (original em JavaScript)
Caminhos aumentantes	Exato	17	própria (BURKARD; DELL'AMICO; MARTELLO, 2009)
Casamento estável			
Gale-Shapley (orientado a objetos)	Exato	19	pacote `gale-shapley-algorithm`
Gale-Shapley (numérico)	Exato	19	pacote `gale-shapley-algorithm`
Gale-Shapley (Lattas)	Exato	19	reimpl. de `StableMarriages`
Grafos gerais			
Blossom/Edmonds (cardinalidade)	Exato	20	<adharshkamath/Edmonds-Algorithm>
van Rantwijk (peso máximo)	Exato	24	`mwmatching.py` de J. van Rantwijk
NetworkX (cardinalidade e peso)	Referência	20 e 24	NetworkX (interpretada)
Path Growing	Aproximado	25	própria (DRAKE; HOUGARDY, 2003)
Emparelhamento induzido máximo			
Exato (conjunto independente)	Exato	26	própria (CAMERON, 1989)
Guloso aleatório	Heurística	27	própria
Guloso de grau mínimo	Heurística	27	própria

Nota: **Alg.**: número do pseudocódigo correspondente. **Origem**: `usuário/nome` indica um repositório do GitHub; NetworkX, igraph, SciPy e lap são bibliotecas, com núcleo interpretado (Python) ou compilado; "própria" indica implementação deste trabalho a partir da referência citada; "reimpl." indica código reimplementado a partir do original indicado.

Os endereços completos constam no cabeçalho de cada adaptador, no código-fonte. Cabe uma ressalva de linhagem: a rotina de emparelhamento de peso máximo em grafos gerais do NetworkX é uma adaptação do mesmo código de van Rantwijk também avaliado de forma independente; compará-las mede o efeito da engenharia de biblioteca sobre um mesmo algoritmo, e não duas abordagens distintas.

Por fim, nenhuma implementação própria ou reimplementada foi incorporada sem **verificação independente**: os métodos exatos foram confrontados com soluções por força bruta (em instâncias pequenas) e com bibliotecas consolidadas; os métodos aproximados e as heurísticas, quanto à validade da solução e a não superarem o ótimo, e os aproximados também quanto a respeitarem suas garantias de aproximação. Além disso, os benchmarks já existentes ao início do trabalho foram migrados para o ambiente unificado sob uma salvaguarda de não-regressão: a qualidade de saída de cada um foi registrada antes da migração e exigiu-se que a nova execução a reproduzisse exatamente, de modo que a unificação não alterasse nenhuma conclusão.

6 RESULTADOS E DISCUSSÃO

Este capítulo apresenta e discute os resultados obtidos com o ambiente de testes descrito no capítulo anterior. Para cada problema, apresentam-se **todas as famílias de instância** definidas na Seção 5.3, de modo a expor como a estrutura da entrada afeta o desempenho e a qualidade. As tabelas de tempo deste capítulo trazem as implementações em linguagem interpretada (Python); conforme a Seção 5.5, os tempos das implementações de núcleo compilado, que servem de referência, ficam no Apêndice A. Para cada instância, a tabela de tempo é seguida da tabela de qualidade. Todos os valores são gerados diretamente a partir dos dados medidos. A Tabela 10 relaciona cada algoritmo destas tabelas ao seu pseudocódigo e à sua origem. Ao final, discutem-se os defeitos encontrados em implementações públicas e o crescimento empírico dos tempos frente à complexidade teórica.

6.1 CARDINALIDADE MÁXIMA EM GRAFOS BIPARTIDOS

As tabelas a seguir apresentam tempo e qualidade para os três grafos: dois esparsos (um sem e um com emparelhamento perfeito) e um denso.

Tabela 11 – Tempo de execução em Python — Grafo 1 — Esparso | SEM perfeito | MCM = 80% de n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	$O(V + E)$	3.65	13.93	30.60	54.24	83.43
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.55	1.10	1.64	2.16	2.73
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	$O(V^2)$	9.56	65.27	216.18	519.03	1024.37
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.57	1.20	1.55	2.02	2.80
Gulosa	aproximado	$O(E)$	$O(V)$	0.01	0.01	0.02	0.03	0.03
Karp-Sipser	aproximado	$O(V \cdot E)$	$O(V + E)$	0.16	0.44	0.88	1.50	2.30

Tabela 12 – Qualidade da solução — Grafo 1 — Esparso | SEM perfeito | MCM = 80% de n

Algoritmo	**Tipo**	**Tempo**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	40	80	120	160	200
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	40	80	120	160	200
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	40	80	120	160	200
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	40	80	120	160	200
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	40	80	120	160	200
Gulosa	aproximado	$O(E)$	37	76	116	156	195
Karp-Sipser	aproximado	$O(V \cdot E)$	40	80	120	160	200

Tabela 13 – Tempo de execução em Python — Grafo 2 — Esparso | COM perfeito | MCM = n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	$O(V + E)$	6.08	25.24	58.92	75.27	419.70
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.58	1.08	1.67	2.17	3.72
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	$O(V^2)$	9.12	62.41	208.19	498.99	984.10
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.47	0.93	1.36	1.76	2.91
Gulosa	aproximado	$O(E)$	$O(V)$	0.01	0.01	0.02	0.02	0.03
Karp-Sipser	aproximado	$O(V \cdot E)$	$O(V + E)$	0.20	0.59	1.23	2.11	2.53

Tabela 14 – Qualidade da solução — Grafo 2 — Esparso | COM perfeito | MCM = n

Algoritmo	**Tipo**	**Tempo**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	50	100	150	200	250
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	50	100	150	200	250
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Gulosa	aproximado	$O(E)$	43	86	130	170	216
Karp-Sipser	aproximado	$O(V \cdot E)$	50	100	150	200	250

Tabela 15 – Tempo de execução em Python — Grafo 3 — Denso | COM perfeito | MCM = n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	$O(V + E)$	2.08	19.12	34.10	39.79	259.82
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	$O(V + E)$	1.14	2.41	3.99	6.19	7.53
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	$O(V^2)$	12.19	75.08	238.34	556.37	1077.90
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	1.51	3.15	4.84	6.94	8.66
Gulosa	aproximado	$O(E)$	$O(V)$	0.03	0.06	0.08	0.11	0.14
Karp-Sipser	aproximado	$O(V \cdot E)$	$O(V + E)$	0.49	1.35	3.87	4.65	6.86

Tabela 16 – Qualidade da solução — Grafo 3 — Denso | COM perfeito | MCM = n

Algoritmo	**Tipo**	**Tempo**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Aug. Paths (wbchristerson)	exato	$O(V \cdot E)$	50	100	150	200	250
Hopcroft-Karp (sofiat)	exato	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Edmonds-Karp (Maxflow-Algs)	exato	$O(V \cdot E^2)$	50	100	150	200	250
Hopcroft-Karp (NetworkX)	baseline	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	50	100	150	200	250
Gulosa	aproximado	$O(E)$	47	93	142	191	235
Karp-Sipser	aproximado	$O(V \cdot E)$	50	100	150	200	249

Entre os algoritmos exatos, a hierarquia de desempenho é a mesma nas três instâncias: o Edmonds-Karp, que reduz o emparelhamento a um problema de fluxo, é mais de duas ordens de grandeza mais lento que o Hopcroft-Karp, que agrupa os

caminhos aumentantes em fases. A distância é ainda maior no grafo denso, onde o número de arestas amplifica o custo de reconstruir a busca a cada aumento. A implementação de núcleo compilado (igraph, Apêndice A) é a mais rápida em todas as instâncias, como esperado.

Quanto aos algoritmos aproximados, o compromisso entre qualidade e tempo se confirma, mas com uma nuance revelada pela variação entre instâncias: o método guloso, embora seja a mais rápida de todo o conjunto, tem sua pior qualidade não no grafo denso, e sim no grafo esparso com emparelhamento perfeito, em que a ordem de varredura das arestas mais facilmente o prende em um emparelhamento maximal pequeno. O algoritmo de Karp-Sipser, por sua vez, encontra o ótimo em todas as instâncias esparsas e fica a menos de meio ponto percentual dele no grafo denso, resultado da regra dos vértices de grau um, que domina em grafos esparsos.

6.2 ATRIBUIÇÃO LINEAR

Apresentam-se as três matrizes de custo: densa uniforme (aleatória), permutação barata e estruturada.

Tabela 17 – Tempo de execução em Python — Matriz 1 — Densa uniforme | custo in [1,100] (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	$O(n^2)$	0.39	1.98	4.82	15.43	41.61	97.23	239.96	428.17	679.84
Húngaro (benchaplin)	exato	$O(n^3)$	$O(n^2)$	11.11	92.32	391.64	BugAttr	BugAttr	BugAttr	BugAttr	BugAttr	BugAttr
Húngaro (hunalgorithm)	exato	$O(n^3)$	$O(n^2)$	2.10	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc
Húngaro (mayorx/KM)	exato	$O(n^3)$	$O(n^2)$	0.99	2.25	7.49	12.30	21.11	29.30	58.44	58.55	86.71
SSP (flows)	exato	$O(n^4)$*	$O(n^2)$	0.45	2.65	6.92	29.97	98.01	235.13	—	—	—

Tabela 18 – Qualidade da solução — Matriz 1 — Densa uniforme | custo in [1,100]

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	102	146	168	217	238	244	258	275	300
Húngaro (benchaplin)	exato	$O(n^3)$	102	146	168	BugAttr	BugAttr	BugAttr	BugAttr	BugAttr	BugAttr
Húngaro (hunalgorithm)	exato	$O(n^3)$	102	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc	BugPerc
Húngaro (mayorx/KM)	exato	$O(n^3)$	102	146	168	217	238	244	258	275	300
SSP (flows)	exato	$O(n^4)$*	102	146	168	217	238	244	—	—	—
SciPy (linear_sum_assg)	baseline	$O(n^3)$	102	146	168	217	238	244	258	275	300
Jonker-Volgenant (lap)	baseline	$O(n^3)$	102	146	168	217	238	244	258	275	300

Tabela 19 – Tempo de execução em Python — Matriz 2 — Permutação barata | ótimo = n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	$O(n^2)$	0.11	0.32	0.62	1.61	3.44	5.87	13.12	23.18	35.94
Húngaro (benchaplin)	exato	$O(n^3)$	$O(n^2)$	2.13	7.57	15.76	41.53	91.41	170.38	398.76	728.52	1163.23
Húngaro (hunalgorithm)	exato	$O(n^3)$	$O(n^2)$	0.14	0.37	0.72	1.81	4.35	7.42	16.75	28.39	44.60
Húngaro (mayorx/KM)	exato	$O(n^3)$	$O(n^2)$	0.42	0.80	1.23	2.12	3.39	4.84	8.22	12.34	17.08
SSP (flows)	exato	$O(n^4)$*	$O(n^2)$	0.40	2.15	6.23	25.72	86.22	212.71	—	—	—

Tabela 20 – Qualidade da solução — Matriz 2 — Permutação barata | ótimo = n

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (benchaplin)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (hunalgorithm)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (mayorx/KM)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
SSP (flows)	exato	$O(n^4)$*	10	20	30	50	75	100	—	—	—
SciPy (linear_sum_assg)	baseline	$O(n^3)$	10	20	30	50	75	100	150	200	250
Jonker-Volgenant (lap)	baseline	$O(n^3)$	10	20	30	50	75	100	150	200	250

Tabela 21 – Tempo de execução em Python — Matriz 3 — Estruturada | custo = |i-j|+1 (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	$O(n^2)$	0.10	0.32	0.62	1.64	3.38	5.96	13.21	23.29	36.16
Húngaro (benchaplin)	exato	$O(n^3)$	$O(n^2)$	2.15	7.58	15.80	41.51	91.39	170.36	396.75	726.02	1157.10
Húngaro (hunalgorithm)	exato	$O(n^3)$	$O(n^2)$	0.13	0.36	0.71	1.78	4.31	7.31	16.45	27.90	43.83
Húngaro (mayorx/KM)	exato	$O(n^3)$	$O(n^2)$	0.41	0.80	1.21	2.12	3.40	4.83	8.27	12.35	17.01
SSP (flows)	exato	$O(n^4)$*	$O(n^2)$	0.39	2.17	6.28	26.00	87.68	214.55	—	—	—

Tabela 22 – Qualidade da solução — Matriz 3 — Estruturada | custo = |i-j|+1

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Húngaro (munkres)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (benchaplin)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (hunalgorithm)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
Húngaro (mayorx/KM)	exato	$O(n^3)$	10	20	30	50	75	100	150	200	250
SSP (flows)	exato	$O(n^4)$*	10	20	30	50	75	100	—	—	—
SciPy (linear_sum_assg)	baseline	$O(n^3)$	10	20	30	50	75	100	150	200	250
Jonker-Volgenant (lap)	baseline	$O(n^3)$	10	20	30	50	75	100	150	200	250

Todos os algoritmos que concluem atingem o custo ótimo, de modo que a comparação recai sobre o tempo. Entre as quatro implementações do método Húngaro, obtidas de autores distintos para exatamente o mesmo algoritmo, observa-se variação de mais de uma ordem de grandeza — evidência de que a qualidade da implementação, e não apenas o algoritmo, determina o desempenho na prática. As implementações de núcleo compilado (SciPy e Jonker-Volgenant, Apêndice A) são as mais rápidas em todas as matrizes.

O ponto mais instrutivo, porém, só aparece ao comparar as instâncias: os defeitos das implementações `benchaplin` e `hunalgorithm` manifestam-se **exclusivamente na matriz densa aleatória**. Nas matrizes estruturadas, que produzem padrões de custo mais regulares, ambas concluem normalmente. Essa sensibilidade ao conteúdo da entrada — em que a mesma implementação ora funciona, ora falha, conforme a estrutura da matriz — é discutida em conjunto na Seção 6.7.

6.3 ATRIBUIÇÃO GARGALO

Apresentam-se as três matrizes de custo, para os dois algoritmos exatos.

Tabela 23 – Tempo de execução em Python — Matriz 1 — Densa uniforme | custo in [1,100] (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	$O(n^2)$	0.10	0.50	1.44	4.48	13.50	32.06	109.54	257.24
Caminhos aumentantes	exato	$O(n^3)$	$O(n^2)$	0.05	0.23	0.62	2.10	5.43	11.32	32.65	68.21

Tabela 24 – Qualidade da solução — Matriz 1 — Densa uniforme | custo in [1,100]

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	26	18	18	13	8	7	5	4
Caminhos aumentantes	exato	$O(n^3)$	26	18	18	13	8	7	5	4

Tabela 25 – Tempo de execução em Python — Matriz 2 — Estruturada | custo = |i-j|+1 (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	$O(n^2)$	0.10	0.55	1.42	6.51	23.44	51.67	192.45	438.60
Caminhos aumentantes	exato	$O(n^3)$	$O(n^2)$	0.03	0.11	0.23	0.61	1.35	2.46	5.63	10.13

Tabela 26 – Qualidade da solução — Matriz 2 — Estruturada | custo = |i-j|+1

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	1	1	1	1	1	1	1	1
Caminhos aumentantes	exato	$O(n^3)$	1	1	1	1	1	1	1	1

Tabela 27 – Tempo de execução em Python — Matriz 3 — Poucos valores | muitos empates (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	$O(n^2)$	0.05	0.27	0.80	2.97	9.97	23.25	79.08	188.72
Caminhos aumentantes	exato	$O(n^3)$	$O(n^2)$	0.06	0.42	1.33	5.26	15.55	35.09	111.08	253.58

Tabela 28 – Qualidade da solução — Matriz 3 — Poucos valores | muitos empates

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$
Threshold (busca binária)	exato	$O(n^3 logn)$	5	1	1	1	1	1	1	1
Caminhos aumentantes	exato	$O(n^3)$	5	1	1	1	1	1	1	1

Ambos os algoritmos produzem sempre o mesmo custo gargalo ótimo, o que também serve como verificação cruzada de corretude. O tempo, no entanto, revela o

achado mais interessante deste problema, e que ilustra bem por que vale examinar todas as instâncias: **a ordem entre os dois algoritmos se inverte conforme a matriz**. Nas matrizes densa e estruturada, o método de caminhos aumentantes, construtivo, é muito mais rápido que o de *threshold*. Na matriz com poucos valores distintos, contudo, o *threshold* passa a ser o mais rápido: como sua busca binária opera sobre o conjunto de valores distintos da matriz, poucos valores tornam essa busca curtíssima, enquanto o método construtivo não se beneficia da mesma forma. A conclusão de qual método é preferível, portanto, depende da estrutura esperada da entrada — uma nuance que uma única instância representativa teria ocultado.

6.4 CASAMENTO ESTÁVEL

Apresentam-se os três perfis de preferência: aleatório, correlacionado e cíclico.

Tabela 29 – Tempo de execução em Python — Perfil 1 — Aleatório | permutações uniformes (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	$O(n^2)$	7.54	94.14	321.96	692.09	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	$O(n^2)$	0.44	1.57	3.36	5.52	21.70	50.59	95.89
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	$O(n^2)$	0.31	1.29	2.85	5.08	20.07	52.20	100.25

Tabela 30 – Qualidade da solução — Perfil 1 — Aleatório | permutações uniformes

Algoritmo	**Tipo**	**Tempo**	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	0 =ref	0 =ref	0 =ref	0 =ref	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref

Tabela 31 – Tempo de execução em Python — Perfil 2 — Correlacionado | ranking de popularidade (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	$O(n^2)$	43.60	589.68	2826.56	8665.93	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	$O(n^2)$	0.75	2.76	6.22	10.87	43.05	101.47	188.56
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	$O(n^2)$	0.66	3.48	10.24	21.99	144.99	482.24	1160.66

Tabela 32 – Qualidade da solução — Perfil 2 — Correlacionado | ranking de popularidade

Algoritmo	**Tipo**	**Tempo**	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	0 =ref	0 =ref	0 =ref	0 =ref	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref

Tabela 33 – Tempo de execução em Python — Perfil 3 — Cíclico | deslocamento estruturado (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	$O(n^2)$	2.44	14.89	46.06	104.25	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	$O(n^2)$	0.39	1.35	3.00	5.16	20.25	47.17	88.20
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	$O(n^2)$	0.21	0.74	1.60	2.80	10.95	27.22	51.22

Tabela 34 – Qualidade da solução — Perfil 3 — Cíclico | deslocamento estruturado

Algoritmo	**Tipo**	**Tempo**	$n = 25$	$n = 50$	$n = 75$	$n = 100$	$n = 200$	$n = 300$	$n = 400$
Gale-Shapley (OOP/pip)	exato	$O(n^2)$†	0 =ref	0 =ref	0 =ref	0 =ref	—	—	—
Gale-Shapley (numérico/pip)	exato	$O(n^2)$	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref
Gale-Shapley (Lattas/dict)	exato	$O(n^3)$*	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref	0 =ref

Pela unicidade do emparelhamento estável ótimo para os pretendentes, as três variantes produzem exatamente o mesmo resultado em todos os perfis — o que as tabelas de qualidade confirmam, com estabilidade e concordância em todas as células. A qualidade, portanto, não as distingue, e a comparação recai sobre o tempo. A variante orientada a objetos, embora teoricamente quadrática, é substancialmente mais lenta e inviável nos tamanhos maiores, em todos os perfis, por conta do sobrecusto de reconstruir estruturas a cada rodada — comportamento quantificado na Seção 6.8. As variantes que operam sobre vetores e dicionários escalam sem dificuldade até os maiores tamanhos avaliados.

6.5 EMPARELHAMENTO EM GRAFOS GERAIS

Cobrem-se os dois problemas de grafos gerais — cardinalidade máxima e peso máximo —, cada um em uma instância esparsa e uma densa.

Tabela 35 – Tempo de execução em Python — Cardinalidade — esparso (grau ~3) (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
Blossom (Edmonds)	exato	$O(V^3)$	$O(V^2)$	1.66	3.21	3.97	20.36	43.90	103.46
NetworkX (cardinalidade)	baseline	$O(V^3)$	$O(V+E)$	0.40	0.63	0.74	1.41	2.28	3.15

Tabela 36 – Qualidade da solução — Cardinalidade — esparso (grau ~3)

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
Blossom (Edmonds)	exato	$O(V^3)$	5	9	13	23	31	43
NetworkX (cardinalidade)	baseline	$O(V^3)$	5	9	13	23	31	43

Tabela 37 – Tempo de execução em Python — Cardinalidade — denso (p=0.3) (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
Blossom (Edmonds)	exato	$O(V^3)$	$O(V^2)$	1.60	13.79	35.81	342.60	2862.65	9526.65
NetworkX (cardinalidade)	baseline	$O(V^3)$	$O(V+E)$	0.38	0.47	1.07	1.94	12.25	7.46

Tabela 38 – Qualidade da solução — Cardinalidade — denso (p=0.3)

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
Blossom (Edmonds)	exato	$O(V^3)$	5	10	15	25	37	50
NetworkX (cardinalidade)	baseline	$O(V^3)$	5	10	15	25	37	50

Tabela 39 – Tempo de execução em Python — Peso — esparso (grau ~3) (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
van Rantwijk (mwmatching)	exato	$O(V^3)$	$O(V+E)$	0.26	0.54	0.86	2.19	3.49	6.30
NetworkX (peso máximo)	baseline	$O(V^3)$	$O(V+E)$	0.59	1.25	1.98	4.87	7.52	13.58
Path Growing	aproximado	$O(E)$	$O(V+E)$	0.03	0.05	0.07	0.11	0.17	0.23

Tabela 40 – Qualidade da solução — Peso — esparso (grau ~3)

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
van Rantwijk (mwmatching)	exato	$O(V^3)$	309	604	790	1284	1907	2929
NetworkX (peso máximo)	baseline	$O(V^3)$	309	604	790	1284	1907	2929
Path Growing	aproximado	$O(E)$	278	510	732	1030	1492	2407

Tabela 41 – Tempo de execução em Python — Peso — denso (p=0.3) (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
van Rantwijk (mwmatching)	exato	$O(V^3)$	$O(V+E)$	0.25	0.87	2.20	4.83	16.68	26.68
NetworkX (peso máximo)	baseline	$O(V^3)$	$O(V+E)$	0.59	2.38	5.83	13.56	41.85	78.35
Path Growing	aproximado	$O(E)$	$O(V+E)$	0.03	0.08	0.15	0.36	0.78	1.35

Tabela 42 – Qualidade da solução — Peso — denso (p=0.3)

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$
van Rantwijk (mwmatching)	exato	$O(V^3)$	309	765	1279	2247	3429	4710
NetworkX (peso máximo)	baseline	$O(V^3)$	309	765	1279	2247	3429	4710
Path Growing	aproximado	$O(E)$	278	642	1215	1934	3113	4329

No caso ponderado, um resultado é digno de nota e mantém-se nas duas densidades: a implementação independente de van Rantwijk é mais rápida que a

rotina correspondente do NetworkX, ainda que esta seja uma adaptação daquele mesmo código. A comparação, portanto, não opõe duas abordagens, e sim uma implementação a um descendente empacotado dela; a diferença mede o efeito das camadas de generalidade que a biblioteca acrescenta. O algoritmo aproximado Path Growing é ordens de grandeza mais rápida que os exatos e entrega uma fração elevada do peso ótimo em ambas as densidades, sempre bem acima da sua garantia teórica de metade; a qualidade que alcança é, aliás, um pouco superior nos grafos densos.

6.6 EMPARELHAMENTO INDUZIDO MÁXIMO

Sendo NP-difícil, o emparelhamento induzido máximo tem perfil distinto: o algoritmo exato só é viável em instâncias pequenas, e a qualidade das heurísticas é o eixo central. Apresentam-se as três densidades avaliadas. A referência de qualidade depende do tamanho: até $n = 40$, a razão é calculada contra o algoritmo exato; a partir de $n = 50$, em que o exato é inviável, contra o melhor resultado obtido entre as heurísticas. Nesse segundo regime, uma razão igual a 1 indica apenas a melhor heurística, e não necessariamente o ótimo.

Tabela 43 – Tempo de execução em Python — Grafo esparso (grau ~3) (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	$O(E^2)$	0.16	0.29	1.49	1.99	5.10	74.61	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	$O(V+E)$	0.05	0.05	0.08	0.13	0.18	0.29	0.47	1.50	8.28	27.29
Guloso grau mínimo	heuristica	$O(E^2)$	$O(E^2)$	0.18	0.36	0.78	1.47	1.83	2.78	4.27	18.48	85.90	378.36

Tabela 44 – Qualidade da solução — Grafo esparso (grau ~3)

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	2	3	5	5	8	10	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	2	2	3	3	5	7	10	18	41	75
Guloso grau mínimo	heuristica	$O(E^2)$	2	3	5	5	8	10	11	25	49	93

Tabela 45 – Tempo de execução em Python — Grafo médio (grau ~6) (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	$O(E^2)$	0.36	1.07	2.50	4.55	10.56	133.00	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	$O(V+E)$	0.06	0.11	0.17	0.25	0.28	0.40	0.60	2.43	9.02	32.93
Guloso grau mínimo	heuristica	$O(E^2)$	$O(E^2)$	0.47	1.55	3.48	5.50	5.88	11.08	21.08	72.77	327.12	1573.84

Tabela 46 – Qualidade da solução — Grafo médio (grau ~6)

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	2	3	4	4	6	8	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	1	2	3	3	5	6	7	15	30	55
Guloso grau mínimo	heuristica	$O(E^2)$	2	3	3	4	5	7	8	19	39	72

Tabela 47 – Tempo de execução em Python — Grafo denso (grau ~12) (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	$O(E^2)$	0.64	2.82	5.62	9.65	17.19	58.30	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	$O(V + E)$	0.07	0.14	0.20	0.29	0.38	1.06	0.77	2.82	11.87	39.37
Guloso grau mínimo	heuristica	$O(E^2)$	$O(E^2)$	0.85	4.00	8.80	14.85	23.40	48.81	74.74	326.03	1314.14	6086.44

Tabela 48 – Qualidade da solução — Grafo denso (grau ~12)

Algoritmo	**Tipo**	**Tempo**	$n = 10$	$n = 15$	$n = 20$	$n = 25$	$n = 30$	$n = 40$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Exato (MIS conflitos)	exato	$exp.$	1	2	3	3	4	5	—	—	—	—
Guloso aleatório	heuristica	$O(E^2)$	1	1	1	2	3	4	4	9	20	35
Guloso grau mínimo	heuristica	$O(E^2)$	1	2	3	3	4	5	6	14	25	55

O algoritmo exato, que reduz o problema a conjunto independente máximo no grafo de conflitos, resolve as instâncias pequenas em poucos milissegundos — inclusive nas densas, em que o emparelhamento induzido máximo é pequeno e a poda é eficaz —, mas seu custo cresce rapidamente e o torna inviável além de algumas dezenas de vértices, o que justifica empiricamente a necessidade das heurísticas. A comparação entre densidades revela um comportamento não-monótono digno de nota: a heurística de grau mínimo alcança o ótimo tanto nos grafos esparsos quanto nos densos, mas tem sua pior qualidade na densidade *intermediária*. A explicação é que, em grafos muito densos, quase toda aresta conflita com as demais, de modo que o emparelhamento induzido é pequeno e fácil de acertar; é na densidade média que há mais espaço para escolhas localmente boas mas globalmente subótimas. A escolha aleatória, por sua vez, degrada-se de forma mais acentuada com o aumento da densidade. Em todos os casos, a heurística de grau mínimo é consistentemente superior à aleatória, ao custo de um tempo maior, por manter e atualizar o grafo de conflitos.

6.7 DEFEITOS EM IMPLEMENTAÇÕES PÚBLICAS

Um resultado que só emerge da execução repetida dos algoritmos — e que um levantamento puramente bibliográfico não revelaria — diz respeito à confiabilidade de implementações públicas de algoritmos clássicos. Duas das quatro implementações do método Húngaro avaliadas apresentam defeitos, ambos manifestados apenas nas matrizes densas aleatórias, como visto nas tabelas de qualidade da atribuição.

A primeira (`hunalgorithm`) falha de forma determinística a partir de um certo tamanho, por exceder um limite interno de iterações previsto em seu próprio código.

A segunda (`benchaplin`) é mais notável, por ser **não-determinística entre execuções**. Sobre exatamente a mesma matriz de entrada, a implementação ora retorna o custo ótimo correto, ora falha com um erro de tipo, ora ingressa em um caminho de execução tão lento que não conclui em tempo praticável. A causa é a dependência

do defeito quanto à ordem de iteração de uma coleção de objetos, que a linguagem aleatoriza a cada processo. São três desfechos distintos para a mesma entrada, todos observados. Além de ilustrar de forma vívida o valor de *implementar e medir* sobre apenas catalogar, esse comportamento tem uma consequência metodológica: a fixação da semente de aleatorização, descrita no capítulo anterior, foi necessária justamente para tornar os resultados reprodutíveis.

Optou-se por **reportar** esses defeitos, e não corrigi-los: como o objeto de estudo é o conjunto de implementações efetivamente disponíveis, corrigi-los apagaria um resultado legítimo sobre a qualidade do que se encontra publicado.

6.8 CRESCIMENTO EMPÍRICO FRENTE À COMPLEXIDADE TEÓRICA

Como o tempo absoluto não é comparável entre linguagens, a análise de crescimento apoia-se no expoente empírico definido na Seção 5.5: a inclinação da reta ajustada ao gráfico do tempo pelo tamanho, em escala logarítmica. As Figuras 1 e 2 apresentam esses gráficos para o grafo e a matriz densos, e as Tabelas 49 a 53 confrontam o expoente medido com a complexidade teórica.

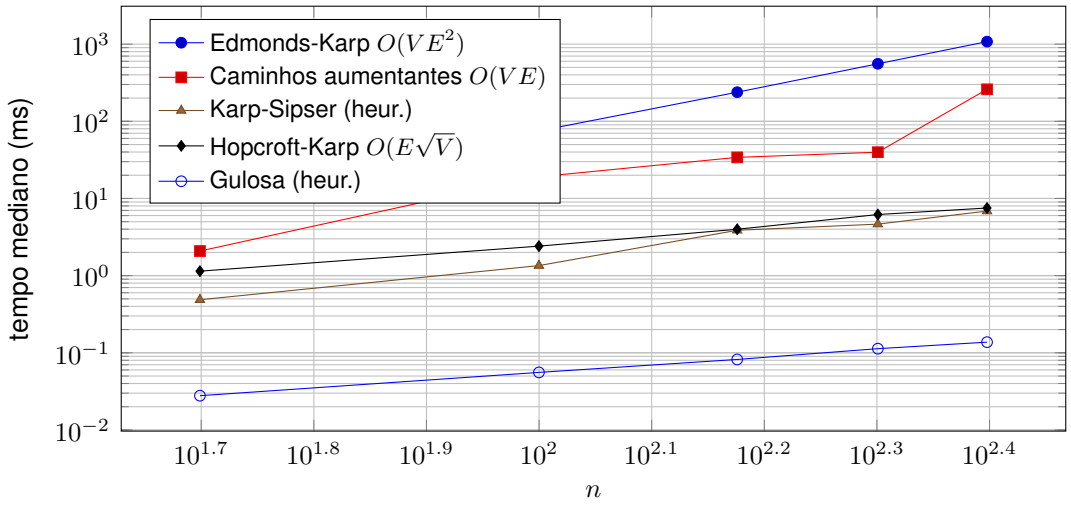


Figura 1 – Tempo de execução em escala log-log para o grafo denso (cardinalidade máxima bipartida). A inclinação de cada reta corresponde ao expoente empírico de crescimento; a de Edmonds-Karp, próxima de 3, destaca-se das demais.

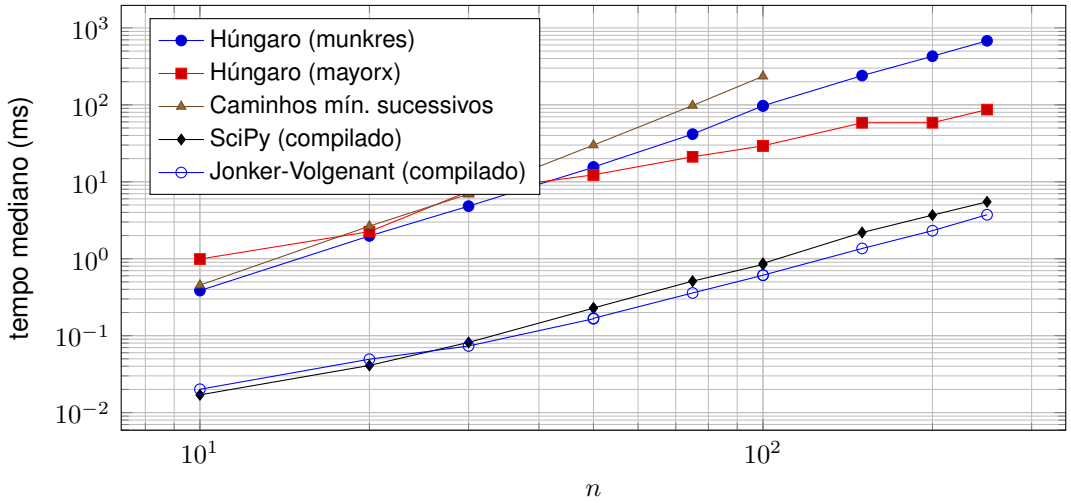


Figura 2 – Tempo de execução em escala log-log para a matriz densa (atribuição linear). As duas implementações de núcleo compilado situam-se bem abaixo das interpretadas, mas com inclinação semelhante.

Os expoentes medidos são, em geral, coerentes com a teoria. O caso mais nítido é o do Edmonds-Karp, cujo expoente próximo de três acompanha o comportamento cúbico previsto e o separa visivelmente dos demais na Figura 1. Nas implementações de núcleo compilado, a reta situa-se muito abaixo das interpretadas, mas com inclinação semelhante — exatamente o efeito esperado: a linguagem altera a constante (a posição da reta), não o expoente (a inclinação).

Tabela 49 – Expoente empírico de crescimento — Grafo 3 — Denso | COM perfeito | MCM = n

Algoritmo	**Compl. teórica**	**Expoente medido**	R^2
Aug. Paths (wbchristerson)	$O(V \cdot E)$	2.61	0.917
Hopcroft-Karp (sofiat)	$O(E \cdot \sqrt{V})$	1.20	0.996
Edmonds-Karp (Maxflow-Algs)	$O(V \cdot E^2)$	2.79	0.999
Hopcroft-Karp (NetworkX)	$O(E \cdot \sqrt{V})$	1.09	0.999
Bipartite Match. (igraph)	$O(E \cdot \sqrt{V})$	1.05	0.994
Gulosa	$O(E)$	1.00	1.000
Karp-Sipser	$O(V \cdot E)$	1.68	0.983

Tabela 50 – Expoente empírico de crescimento — Matriz 1 — Densa uniforme | custo in [1,100]

Algoritmo	**Compl. teórica**	**Expoente medido**	R^2
Húngaro (munkres)	$O(n^3)$	2.36	0.999
Húngaro (benchaplin)	$O(n^3)$	3.22	0.998
Húngaro (mayorx/KM)	$O(n^3)$	1.39	0.980
SSP (flows)	$O(n^4)*$	2.71	0.998
SciPy (linear_sum_assg)	$O(n^3)$	1.86	0.994
Jonker-Volgenant (lap)	$O(n^3)$	1.65	0.987

Observa-se ainda que, nos tamanhos avaliados, o expoente medido frequentemente fica *abaixo* do previsto pela complexidade de pior caso, que é conservadora: o pior caso raramente se materializa nas instâncias típicas. A exceção reveladora é a variante orientada a objetos do Gale-Shapley (Tabela 51), cujo expoente medido supera o teórico: o sobrecusto de implementação a torna, na prática, super-quadrática, confirmando por medição a ressalva já registrada sobre ela.

Tabela 51 – Expoente empírico de crescimento — Perfil 1 — Aleatório | permutações uniformes

Algoritmo	**Compl. teórica**	**Expoente medido**	R^2
Gale-Shapley (OOP/pip)	$O(n^2)$Ď	3.28	0.996
Gale-Shapley (numérico/pip)	$O(n^2)$	1.94	0.999
Gale-Shapley (Lattas/dict)	$O(n^3)*$	2.07	0.999

Tabela 52 – Expoente empírico de crescimento — Matriz 1 — Densa uniforme | custo in [1,100]

Algoritmo	**Compl. teórica**	**Expoente medido**	R^2
Threshold (busca binária)	$O(n^3 logn)$	2.61	0.996
Caminhos aumentantes	$O(n^3)$	2.41	0.999

Tabela 53 – Expoente empírico de crescimento — Peso — esparso (grau 3)

Algoritmo	**Compl. teórica**	**Expoente medido**	R^2
van Rantwijk (mwmatching)	$O(V^3)$	1.40	0.989
NetworkX (peso máximo)	$O(V^3)$	1.36	0.990
Path Growing	$O(E)$	0.88	0.998

7 TRABALHOS FUTUROS

Este trabalho construiu um ambiente de testes unificado e o aplicou a sete problemas de emparelhamento em grafos. Algumas extensões ficaram fora do escopo, por limitação de tempo ou por decisão metodológica, e constituem os próximos passos naturais:

- **Algoritmos descritos e não implementados:** Parte dos métodos apresentados nos capítulos de problemas não foi incorporada ao ambiente de testes: o algoritmo de Dinic, para cardinalidade máxima em grafos bipartidos; a variante de caminhos mínimos sucessivos com Dijkstra e potenciais, para a atribuição linear; o método dual, para a atribuição gargalo; o algoritmo de Micali e Vazirani, para cardinalidade máxima em grafos gerais; e a versão otimizada do algoritmo de Edmonds ponderado (GALIL; MICALI; GABOW, 1986). Como o ambiente já define a interface padrão e o protocolo de cada problema, incluí-los exige apenas um novo adaptador por algoritmo.
- **Métodos aproximados e heurísticas para os demais problemas:** A comparação entre métodos exatos, aproximados e heurísticos foi feita para cardinalidade máxima em grafos bipartidos, peso máximo em grafos gerais e emparelhamento induzido máximo. Ficaram de fora as heurísticas gulosas descritas para a atribuição linear e a atribuição gargalo, o método guloso ordenado para peso máximo em grafos gerais, o método guloso simples e os algoritmos de Karp-Sipser e de crescimento de caminho para cardinalidade máxima em grafos gerais, e as metaheurísticas de busca local para o emparelhamento induzido (FüRST; LEICHTER; RAUTENBACH, 2017).
- **Contagem de operações elementares:** Avaliou-se instrumentar os algoritmos com contadores de operações, como caminhos aumentantes percorridos e arestas examinadas, uma métrica independente de linguagem e de máquina. A ideia foi descartada porque exigiria manter uma cópia modificada de cada repositório ao lado do original intacto. Uma alternativa a investigar é a instrumentação externa, por ferramentas de rastreamento da própria linguagem, sem alterar o código de terceiros.
- **Separação do efeito da linguagem:** As implementações interpretadas e compiladas foram mantidas em tabelas separadas, e o expoente empírico permitiu compará-las apenas quanto ao crescimento. Implementar um mesmo algoritmo nas duas linguagens permitiria medir diretamente quanto da diferença de tempo se deve à linguagem, e não ao algoritmo.

- **Instâncias reais:** As instâncias utilizadas são sintéticas, geradas por famílias parametrizadas. Aplicar o ambiente a instâncias provenientes das aplicações citadas na introdução verificaria se as conclusões se mantêm fora dos cenários controlados.
- **Comunicação dos defeitos encontrados:** Os defeitos identificados em implementações públicas do método Húngaro (Seção 6.7) podem ser reportados aos seus mantenedores, junto com as instâncias que os reproduzem.
- **Problemas fora do escopo:** As variações NP-difíceis citadas na introdução, como o emparelhamento tridimensional e o problema da atribuição quadrática, os emparelhamentos em multigrafos e hipergrafos e as variações em grafos gerais excluídas na delimitação do estudo, como o b-emparelhamento, o emparelhamento em matroides e o *Stable Roommates Problem*.

REFERÊNCIAS

AHO, A. V.; HOPCROFT, J. E.; ULLMAN, J. D. *The Design And Analysis of Computer Algorithms*. 1st. ed. Reading, MA: Addison-Wesley, 1974. ISBN 9780201000290.

AHUJA, R. K.; MAGNANTI, T. L.; ORLIN, J. B. *Network flows: theory, algorithms, and applications*. USA: Prentice-Hall, Inc., 1993. ISBN 013617549X.

BURKARD, R.; DELL'AMICO, M.; MARTELLO, S. *Assignment Problems*. USA: Society for Industrial and Applied Mathematics, 2009. ISBN 0898716632.

CAETANO, T. S. et al. Learning graph matching. *IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE*, IEEE Computer Society, United States, v. 31, n. X, December 2009. ISSN 0162-8828.

CAMERON, K. Induced matchings. *Discrete Applied Mathematics*, v. 24, n. 1, p. 97–102, 1989. ISSN 0166-218X. Disponível em: <https://www.sciencedirect.com/science/article/pii/0166218X9290275F>.

CORMEN, T. H. et al. *Introduction to Algorithms*. 3rd. ed. Cambdrige, MA: The MIT Press, 2009. ISBN 978-0-262-53305-8.

DASGUPTA, C. H. P. S.; VAZIRANI, U. V. *Algorithms*. 1st. ed. New York, NY: McGraw-Hill Education, 2006. ISBN 978-0073523408.

DRAKE, D. E.; HOUGARDY, S. A simple approximation algorithm for the weighted matching problem. *Information Processing Letters*, v. 85, n. 4, p. 211–213, 2003. ISSN 0020-0190. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0020019002003939>.

DUCKWORTH, W.; WORMALD, N.; ZITO, M. Maximum induced matchings of random cubic graphs. *Journal of Computational and Applied Mathematics*, v. 142, n. 1, p. 39–50, 2002. ISSN 0377-0427. Probabilistic Methods in Combinatorics and Combinatorial Optimization. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0377042701004575>.

EDMONDS, J. Paths, trees, and flowers. *Canadian Journal of Mathematics*, v. 17, p. 449–467, 1965.

FAN, W. Graph pattern matching revised for social network analysis. In: *Proceedings of the 15th International Conference on Database Theory*. New York, NY, USA: Association for Computing Machinery, 2012. (ICDT '12), p. 8–21. ISBN 9781450307918. Disponível em: <https://doi.org/10.1145/2274576.2274578>.

FüRST, M.; LEICHTER, M.; RAUTENBACH, D. *Locally Searching for Large Induced Matchings*. 2017. Disponível em: <https://arxiv.org/abs/1708.02028>.

GABOW, H. N. Data structures for weighted matching and nearest common ancestors with linking. In: *Proceedings of the First Annual ACM-SIAM Symposium on Discrete Algorithms*. USA: Society for Industrial and Applied Mathematics, 1990. (SODA '90), p. 434–443. ISBN 0898712513.

GALIL, Z.; MICALI, S.; GABOW, H. An $o(ev \log v)$ algorithm for finding a maximal weighted matching in general graphs. *SIAM J. Comput.*, v. 15, p. 120–130, 02 1986.

GAREY, M. R.; JOHNSON, D. S. *Computers and Intractability; A Guide to the Theory of NP-Completeness*. USA: W. H. Freeman & Co., 1990. ISBN 0716710455.

HALIM, S.; HALIM, F. *Competitive Programming 3: The New Lower Bound of Programming Contests*. 3rd. ed. [S.l.]: Lulu.com, 2013. Self-published by the authors. ISBN 978-1482852483.

HALLDóRSSON, M.; RADHAKRISHNAN, J. Greed is good: approximating independent sets in sparse and bounded-degree graphs. In: *Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing*. New York, NY, USA: Association for Computing Machinery, 1994. (STOC '94), p. 439–448. ISBN 0897916638. Disponível em: <https://doi.org/10.1145/195058.195221>.

HALLER, S. et al. *A Comparative Study of Graph Matching Algorithms in Computer Vision*. 2022. Disponível em: <https://arxiv.org/abs/2207.00291>.

JONKER, R.; VOLGENANT, A. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, v. 38, n. 4, p. 325–340, 1987. Disponível em: <https://doi.org/10.1007/BF02278710>.

JUNGNICKEL, D. *Graphs, Networks and Algorithms*. 3rd. ed. Berlin, Germany: Springer, 2008. ISBN 978-354727798.

KARP, R. M.; SIPSER, M. Maximum matching in sparse random graphs. In: *Proceedings of the 22nd Annual Symposium on Foundations of Computer Science*. USA: IEEE Computer Society, 1981. (SFCS '81), p. 364–375. Disponível em: <https://doi.org/10.1109/SFCS.1981.21>.

KLEINBERG, J.; TARDOS Éva. *Algorithm Design*. 1st. ed. Boston, MA: Pearson, 2005. ISBN 978-0321295354.

LAWLER, E. L. *Combinatorial Optimization: Networks and Matroids*. New York, NY: Dover Publications, 1976. Reprint edition. ISBN 978-0486414539.

Basic terminology. In: LOVáSZ, L.; PLUMMER, M. (Ed.). *Matching Theory*. North-Holland, 1986, (North-Holland Mathematics Studies, v. 121). p. xxix–xxxiii. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0304020808736363>.

MANBER, U. *Introduction to Algorithms: A Creative Approach*. 1st. ed. Reading, MA: Addison-Wesley, 1989. ISBN 978-0201120370.

MANDULAK, M. et al. Efficient weighted graph matching on gpus. *SC24*, IEEE Computer Society, United States, November 2024. ISSN 0162-8828.

MARTELLO, S.; TOTH, P. Linear assignment problems. In: MARTELLO, S. et al. (Ed.). *Surveys in Combinatorial Optimization*. North-Holland, 1987, (North-Holland Mathematics Studies, v. 132). p. 259–282. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0304020808732389>.

MICALI, S.; VAZIRANI, V. An O(sqrt(|v|) |E|) Algorithm for Finding Maximum Matching in General Graphs. In: . [S.l.: s.n.], 1980. p. 17–27.

ROSEN, K. H.; KREHER, D. L.; STINSON, D. R. *Discrete Mathematics and Its Applications*. 1st. ed. Boca Raton, FL: CRC Press, 1998. ISBN 978-0849339882.

SCHRIJVER, A. *Combinatorial Optimization: Polyhedra and Efficiency*. 1st. ed. Berlin, Germany: Springer, 2004. ISBN 3540204563.

STOCKMEYER, L. J.; VAZIRANI, V. V. Np-completeness of some generalizations of the maximum matching problem. *Information Processing Letters*, v. 15, n. 1, p. 14–19, 1982. ISSN 0020-0190. Disponível em: <https://www.sciencedirect.com/science/article/pii/0020019082900771>.

SUSSMAN, D. L.; LI, L. *Overview of Graph Matching Challenges and Approaches*. 2021. Acessado em: 26 nov. 2024. Disponível em: <https://www.ll.mit.edu/>.

TAYLOR, W. R. Protein structure comparison using bipartite graph matching and its application to protein structure classification. *Molecular and Cellular Proteomics*, American Society for Biochemistry and Molecular Biology, United States, v. 1, n. 4, April 2002. ISSN 1535-9476.

VAZIRANI, V. V. *Approximation algorithms*. Berlin, Heidelberg: Springer-Verlag, 2001. ISBN 3540653678.

ZASLAVSKIY, M.; BACH, F.; VERT, J.-P. A path following algorithm for the graph matching problem. *IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE*, IEEE Computer Society, United States, v. 31, n. 12, December 2009. ISSN 0162-8828.

Apêndices

APÊNDICE A – TEMPOS DAS IMPLEMENTAÇÕES DE NÚCLEO COMPILADO

Este apêndice reúne as tabelas de tempo das implementações de núcleo compilado (C/C++), que no Capítulo 6 servem apenas de referência. Conforme os critérios da Seção 5.5, esses tempos não são comparados diretamente com os das implementações interpretadas; a comparação entre linguagens é feita somente pelo expoente empírico de crescimento (Seção 6.8). As instâncias são as mesmas das tabelas correspondentes do Capítulo 6.

A.1 CARDINALIDADE MÁXIMA EM GRAFOS BIPARTIDOS

As Tabelas 54 a 56 trazem os tempos da biblioteca igraph nos três grafos da Seção 6.1.

Tabela 54 – Tempo de execução em compilado (C/C++) — Grafo 1 — Esparso | SEM perfeito | MCM = 80% de n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.09	0.14	0.20	0.26	0.31

Tabela 55 – Tempo de execução em compilado (C/C++) — Grafo 2 — Esparso | COM perfeito | MCM = n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.08	0.13	0.20	0.25	0.31

Tabela 56 – Tempo de execução em compilado (C/C++) — Grafo 3 — Denso | COM perfeito | MCM = n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 50$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
Bipartite Match. (igraph)	baseline	$O(E \cdot \sqrt{V})$	$O(V + E)$	0.20	0.36	0.61	0.85	1.01

A.2 ATRIBUIÇÃO LINEAR

As Tabelas 57 a 59 trazem os tempos das bibliotecas SciPy e lap (Jonker-Volgenant) nas três matrizes da Seção 6.2.

Tabela 57 – Tempo de execução em compilado (C/C++) — Matriz 1 — Densa uniforme | custo in [1,100] (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
SciPy (linear_sum_assg)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.04	0.08	0.23	0.51	0.84	2.19	3.68	5.48
Jonker-Volgenant (lap)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.05	0.07	0.17	0.36	0.61	1.36	2.31	3.73

Tabela 58 – Tempo de execução em compilado (C/C++) — Matriz 2 — Permutação barata | ótimo = n (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
SciPy (linear_sum_assg)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.04	0.07	0.17	0.35	0.61	1.35	2.27	3.54
Jonker-Volgenant (lap)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.04	0.07	0.16	0.32	0.56	1.18	2.04	3.20

Tabela 59 – Tempo de execução em compilado (C/C++) — Matriz 3 — Estruturada | custo = |i-j|+1 (mediana, ms)

Algoritmo	**Tipo**	**Tempo**	**Espaço**	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 75$	$n = 100$	$n = 150$	$n = 200$	$n = 250$
SciPy (linear_sum_assg)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.04	0.07	0.17	0.35	0.58	1.30	2.23	3.50
Jonker-Volgenant (lap)	baseline	$O(n^3)$	$O(n^2)$	0.02	0.04	0.07	0.16	0.32	0.55	1.18	2.04	3.19